


**République Tunisienne**

**Ministère de l'Enseignement Supérieur et de la Recherche Scientifique**

**Université de Monastir**

**Faculté des Sciences de Monastir**




# *Rapport de Projet de Fin d'Études*

*Présenté en vue de l'obtention du*

***Mastère Professionnel en Sciences des Données***

**Elaboré Par**

**Rahma Charfeddine**

**Développement d'un Assistant IA pour l'Automatisation du Recrutement**

Soutenu le 08/10/2025 à 15 :15 devant le jury :

| | |
|---|---|
| Président | Pr. FEKI Ghada |
| Rapporteur | Pr. MAATOUK Mohamed Neji |
| Encadrant Académique | Pr. GHORBEL Leila |
| Encadrant Professionnel | Mme. ZRIDA Mariem |




Année Universitaire : 2024-2025

Code Sujet : MPDS-2-25-14

# Signatures des encadrants

Encadrant Académique (FSM)

Pr. GHORBEL Leila



Encadrant Professionnel (AstroLab Group)

Mme. ZRIDA Mariem



# Abstract

This final-year project, carried out as part of a professional Master's degree in Data Science, focuses on the development of an AI Assistant designed to automate the recruitment process. The project is structured around three modules : the first two handle the analysis and intelligent matching of resumes and job offers, while the third provides a conversational chatbot.



**Keywords : Recruitment, Parsing, Matching, Chatbot, LLM, NLP**



# Résumé

Ce projet de fin d'études, dans le cadre d'un mastère professionnel en Science des Données, porte sur le développement d'un Assistant IA qui vise à automatiser le processus de rectrutement. Ce processus repose sur la conception et le réaliastion de trois modules. Le premier et le deuxieme modules permettent l'analyse et la mise en correspondance intelligente des CV et des offres d'emploi. Le troisieme module offre un chatbot conversationnel.



**Mots clés : Recrutement, Parsing, Mise en Correspondance, Chatbot, NLP, LLM**



# ملخّص

هذا المشروع النهائي، المنجز في إطار ماجستير مهني في علوم البيانات، يتمحور حول تطوير مساعد ذكاء اصطناعي يهدف إلى أتمتة عملية التوظيف. يقوم المشروع على ثلاثة وحدات : تتولى الوحدتان الأولى والثانية تحليل السير الذاتية ومطابقتها بذكاء مع عروض العمل، بينما توفر الوحدة الثالثة روبوت دردشة تفاعلي.



الكلمات المفاتيح **إالتوظيف، التحليل، المطابقة، الدردشة الآلية، نظام تتبع المتقدمين، النماذج اللغوية الكبيرة، معالجة اللغة الطبيعية**

# *Dédicace*

*La réussite de ce projet de fin d'études n'aurait pas été possible sans le soutien et l'implication d'un certain nombre de personnes.*
*Dire "merci" semble bien insuffisant pour exprimer toute la reconnaissance que je ressens à leur égard.*

*Je dédie ce travail à mes chers parents, pour leur amour inconditionnel, leur soutien sans faille, et les sacrifices qu'ils ont consentis tout au long de mon parcours. Vous êtes ma plus grande source de force et d'inspiration.*
*À mon cher frère, malgré la distance qui nous sépare, ta complicité et tes encouragements constants ont été une véritable force pour moi.*
*À mon fiancé, je te remercie du fond du cœur ta confiance indéfectible. Tu as su trouver les mots justes et être là quand j'en avais le plus besoin.*
*Enfin, à toute ma famille et à mes amis, merci pour votre soutien, vos conseils et votre affection. Vous avez tous contribué à ce succès d'une manière ou d'une autre.*

# Remerciements

À l'issue de ce projet de fin d'études, il m'est particulièrement agréable d'exprimer ma profonde gratitude envers toutes les personnes qui ont, de près ou de loin, contribué à sa réalisation.

Je tiens à remercier tout particulièrement **Madame Leila GHORBEL**, mon encadrante académique, pour sa disponibilité, son expertise et ses précieux conseils. Sa guidance, son suivi rigoureux et ses encouragements tout au long de la période de stage ont été essentiels à la réussite de ce projet. Elle a su m'orienter avec bienveillance et m'offrir les clés nécessaires pour progresser, tant sur le plan technique que personnel.

Je souhaite également exprimer mes remerciements à **Madame Mariem ZRIDA**, mon encadrante professionnelle et à toute l'équipe de **AstroLab Group** pour m'avoir accueilli au sein de l'entreprise dans un environnement aussi stimulant que formateur. Cette expérience a été pour moi une opportunité enrichissante, non seulement sur le plan professionnel mais aussi humain. J'ai beaucoup appris de chaque membre de l'équipe, et leur soutien ainsi que leur collaboration ont grandement facilité le bon déroulement de ce stage.

J'adresse aussi mes sincères remerciements à **Madame Ghada FEKI** et à **Monsieur Mohamed Neji MAATOUK**, membres de mon jury, pour l'honneur qu'ils m'ont fait en acceptant d'évaluer mon travail. Leur regard critique et leurs retours seront, j'en suis certaine, sources d'amélioration pour mon futur parcours professionnel.

Je tiens également à remercier mes professeurs, pour l'ensemble des connaissances transmises au cours de ces années de formation.

Je n'oublie pas non plus toutes les personnes, collègues, collaborateurs et amis, qui ont contribué, de près ou de loin, à l'élaboration de ce travail, par leurs échanges constructifs, leur assistance ou même leurs encouragements. Chaque geste, chaque conseil a compté et m'a permis d'aller de l'avant.

Enfin, à toutes celles et ceux qui, de près ou de loin, ont participé à cette aventure, je vous adresse mes plus sincères remerciements.

# Table des matières

**Intoduction Générale** **1**

**1 Contexte général de projet** **3**

1.1 Présentation de l'organisme d'accueil . . . . . 3

1.1.1 Aperçu général de l'entreprise . . . . . 3

1.1.2 Produits et Services . . . . . 4

1.1.3 Mission, Vision et Valeurs . . . . . 4

1.2 Description du Stage . . . . . 6

1.3 Présentation de la problématique . . . . . 6

1.4 Etude de l'existant et solution proposée . . . . . 8

1.4.1 Solutions existantes . . . . . 8

1.4.2 Critique des solutions existantes . . . . . 12

1.4.3 Solution proposée . . . . . 13

1.5 Environnements de développement . . . . . 14

1.5.1 Environnements matériels . . . . . 14

1.5.2 Environnements logiciels . . . . . 14

1.6 Méthodologie de travail . . . . . 17

1.6.1 Choix de Kanban et avantages . . . . . 18

1.6.2 Représentation visuelle du tableau Kanban . . . . . 19

1.6.3 Backlog du Produit . . . . . 21

**2 Etat de l'Art : Méthodes et Techniques de Recrutement 24**

2.1 Évolution des méthodes de recrutement . . . . . 24

2.1.1 Le recrutement traditionnel : un processus manuel et subjectif . . . . . 24

2.1.2 La digitalisation et les premiers outils numériques . . . . . 25

2.1.3 L'essor de l'Intelligence Artificielle dans le recrutement . . . . . 25

2.1.4 Vers un modèle hybride homme-machine . . . . . 26

2.1.5 Domaines clés d'application de l'IA dans le recrutement . . . . . 26

2.2 Le traitement du langage naturel (NLP) . . . . . 27

2.2.1 Les categories de NLP . . . . . 27

2.2.2 Les étapes du NLP . . . . . 28

2.3 Modèles de langage avancés et Transformers . . . . . 31

2.3.1 L'architecture Transformer . . . . . 31

2.3.2 Modèles d'embeddings . . . . . 33

2.3.3 Bases vectorielles . . . . . 33

2.3.4 Analyse comparative des modèles LLM modernes . . . . . 35

2.3.5 Prompt Engineering . . . . . 36

2.4 Analyse des données de recrutement : Parsing . . . . . 37

2.4.1 Définition du Parsing . . . . . 37

2.4.2 Techniques classiques du Parsing . . . . . 38

2.4.3 Approches modernes basées sur le NLP et l'apprentissage automatique . 38

2.5 Mise en correspondance CV–offre d'emploi : Matching . . . . . 39

2.5.1 Définition du Matching . . . . . 39

2.5.2 Techniques classiques de Matching . . . . . 40

2.5.3 Approches modernes avec Transformers et LLM . . . . . 41

2.6 Chatbots et systèmes conversationnels . . . . . 42
2.6.1 Définition et rôle . . . . . 42
2.6.2 Composants clés d'un chatbot . . . . . 42
2.6.3 Limites et défis . . . . . 43

**3 Analyse des Données des CV et des Offres d'emploi 45**

3.1 Approche d'analyse basée sur LLM et LangChain . . . . . 46
3.1.1 Objectif et motivation . . . . . 46
3.1.2 Pipeline de traitement . . . . . 47
3.2 Données Initiales . . . . . 49
3.2.1 Curriculum Vitae (CV) : . . . . . 49
3.2.2 Offre d'emploi . . . . . 51
3.3 Construction du prompt dynamique et Integration de LangChain . . . . . 53
3.3.1 Construction des prompts dynamiques . . . . . 53
3.3.2 Intégration dans LangChain . . . . . 55
3.4 Evaluation de l'approche proposée . . . . . 56
3.4.1 Métriques d'évaluation . . . . . 56
3.4.2 Résultats comparatifs & choix final du modèle . . . . . 57
3.5 Résultats du parsing . . . . . 59
3.5.1 Exemple d'un CV aprés analyse . . . . . 59
3.5.2 Profil candidat basé sur les données analysées . . . . . 60

**4 Mise en Correspondance : Le Matching 62**

4.1 Apperçu général des approches proposées . . . . . 62
4.2 Premiére Approche de matching basée sur NLP classique . . . . . 64
4.2.1 Présentation du pipeline . . . . . 64

4.2.2 Comparaison des modèles d'embeddings sur le cas d'usage CV - Offre de travail . . . 67
4.3 Deuxième approche de matching basée sur les modèles LLM . . . 68
4.3.1 Pipeline de traitement . . . 68
4.3.2 Construction du prompt de matching . . . 69
4.4 Evaluation . . . 71
4.4.1 Modèles testés . . . 71
4.4.2 Métriques adaptées . . . 71
4.4.3 Évaluation des modèles testés . . . 72
4.5 Intégration du module de matching dans Talinty . . . 73
4.5.1 Architecture de l'API . . . 73
4.5.2 Fonctionnalités principales exposées . . . 73
4.5.3 Résultats du matching dans Talinty . . . 74

**5 Chatbot de la plateforme Talinty 76**

5.1 Objectif et cas d'usage du Chatbot Talinty . . . 76
5.2 Pipeline du Chatbot Talinty . . . 78
5.3 Base de données utilisée . . . 80
5.3.1 Organisation des collections principales . . . 81
5.3.2 Avantages de MongoDB dans ce contexte . . . 81
5.3.3 Rôle central de MongoDB dans le chatbot Talinty . . . 83
5.4 Embeddings et Recherche Vectorielle . . . 83
5.4.1 Génération des Embeddings . . . 83
5.4.2 Stockage Vectoriel dans MongoDB . . . 84
5.4.3 Collection dédiée aux embeddings . . . 84
5.4.4 Recherche par Similarité Cosinus . . . 84

5.5 Résultats et cas d'utilisation . . . . . 85

5.5.1 Exemples de requêtes valides . . . . . 85

5.5.2 Qualité des réponses observées . . . . . 85

5.5.3 Limites rencontrées . . . . . 86

5.5.4 Résultat Final . . . . . 87

**Conclusion Générale et Perspectives** **91**

**Annexe** **93**

**Bibliographie** **101**

# Table des figures


1.1 Logo de l'organisme d'accueil . . . . . 4

1.2 Logo de l'API Affinda . . . . . 10

1.3 Logo Python . . . . . 17

1.4 Capture du tableau Kanban sur ClickUp . . . . . 20

2.1 Différences entre NLU et NLP [19] . . . . . 28

2.2 Différences entre stemming et lemmatisation . . . . . 29

2.3 Architectures CBOW et Skip-Gram de Word2Vec . . . . . 31

2.4 Schéma général de l'architecture Transformer [68] . . . . . 32

2.5 Le mécanisme de l'attention [5] . . . . . 32

3.1 Pipeline du Parsing basé sur LLM et LangChain . . . . . 48

3.2 Exemple d'un CV . . . . . 50

3.3 Exemple d'une offre d'emploi . . . . . 52

3.4 Résultat du parsing d'un CV- Partie 1 . . . . . 59

3.5 Résultat du parsing d'un CV- Partie 2 . . . . . 59

3.6 Résultat du parsing d'un CV - Partie 4 . . . . . 60

3.7 Résultat du parsing d'un CV -Partie 5 . . . . . 60

3.8 Profil candidat alimenté automatiquement grâce au parsing du CV . . . . . 61

4.1 Pipeline avec LLM du Matching . . . . . 64

4.2 Résultat du matching d'un CV- offre d'emploi . . . . . 74

5.1 Pipeline du chatbot Talinty . . . . . 79

5.2 Diagramme de classe - BD Talinty . . . . . 82

5.3 Conversation Chatbot- Partie 1 . . . . . 87

5.4 Conversation Chatbot- Partie 2 . . . . . 88

5.5 Conversation Chatbot- Partie 3 . . . . . 88

5.6 Conversation Chatbot - Partie 4 . . . . . 89

5.7 Conversation Chatbot -Partie 5 . . . . . 89

5.8 Conversation Chatbot -Partie 6 . . . . . 90

5.9 Pipeline du Parsing . . . . . 93

# Liste des tableaux

1.1 Matériel de base . . . . . 14

1.2 Outils et logiciels utilisés dans le projet . . . . . 17

1.4 Environnements d'exécution et frameworks . . . . . 18

2.1 Comparatif des modèles d'embeddings . . . . . 34

2.2 Tableau comparatif des modèles de langage récents . . . . . 35

3.1 Résultats comparatifs des modèles évalués sur le corpus de test. . . . . 58

4.1 Comparaison des modèles d'embedding . . . . . 67

4.2 Résultats détaillés des modèles Gemini et LLaMA sur 10 CV . . . . . 72

4.3 Comparaison des performances moyennes des modèles LLM testés . . . . . 72

5.1 Exemples de requêtes valides et résultats attendus . . . . . 86

5.2 Résultats des performances du parsing classique par type d'entité . . . . . 99

# Liste des abréviations

— AI - Intelligence Artificielle
— ATS - Système de Suivi des Candidatures
— NLU - Natural Language Understanding
— NLP - Natural Language Processing
— GPT - Generative Pre-trained Transformer
— NER - Named Entity Recognition
— LLM - Large Language Model
— SIRH - Système d'Information de Gestion des Ressources Humaines
— API - Application Programming Interface
— PDF - Portable Document Format

# Introduction Générale

Le recrutement en ligne a changé en profondeur la façon dont les entreprises trouvent leurs futurs collaborateurs. Avec Internet et les plateformes spécialisées, les recruteurs ont désormais sous la main une immense réserve de talents, où des millions de profils circulent chaque jour. Cette diversité ouvre beaucoup d'opportunités afin de choisir le bon profil , mais elle complique aussi la tâche : plus de choix signifie aussi plus de tri des candidatures à faire.
C'est dans ce contexte que les entreprises investissent massivement dans des solutions technologiques innovantes pour faciliter et optimiser leurs processus de recrutement. Parmi ces innovations indispensables pour naviguer efficacement dans cet environnement de recrutement digitalisé, nous pouvons citer l'automatisation des tâches répétitives, l'intelligence artificielle appliquée à l'analyse des profils, et les interfaces conversationnelles. Ces derniers permettent aux entreprises de centraliser la gestion des candidatures, de structurer leurs processus de sélection et d'exploiter intelligemment les données des candidats pour identifier les profils les plus pertinents.

C'est dans cette dynamique d'innovation que s'inscrit notre projet de fin d'études, réalisé au sein de l'entreprise AstroLab disposant déjà d'une plateforme de suivi des candidatures (Applicant Tracking System : ATS) opérationnelle "Talinty". Notre mission consiste à enrichir cette solution existante par le développement de trois modules complémentaires conçus pour répondre aux défis contemporains du recrutement en ligne.
Le premier module développé est un moteur d'analyse intelligent des données des CVs et offres d'emploi, capable d'extraire automatiquement et de structurer toutes les informations pertinentes. Ce module répond directement au besoin de traiter efficacement de gros volumes de candidatures.
Le second module constitue un système intelligent de mise en correspondance qui évalue et quantifie la compatibilité entre les profils des candidats (CV) et les exigences des offres d'emploi. Ce module facilite la recherche du candidat idéal en l'identifiant rapidement par un score precis.

Le troisième module consiste à integrer un chatbot conversationnel dans la plateforme. Il permet de moderniser l'expérience utilisateur en offrant une assistance automatisée et instantanée aux recruteurs.

Ce rapport présente l'ensemble des travaux réalisés au cours de notre projet et se structure en cinq chapitres principaux.
Le premier chapitre introduit le contexte général du projet.Il présente l'organisme d'accueil, décrit le stage ainsi que la problématique rencontrée, puis expose l'étude de l'existant, la solution proposée, les environnements de développement et la méthodologie adoptée.
Le deuxième chapitre fait le point sur les notions théoriques essentielles : l'intelligence artificielle appliquée au recrutement, le traitement du langage naturel, les modèles de langage avancés, le prompt engineering, ainsi que les techniques de parsing, de matching et les systèmes conversationnels.
Le troisième chapitre détaille le pipeline d'analyse des données des CV et des offres d'emploi. Il expose les différentes approches testées et leurs résultats.
Le quatrième chapitre se concentre sur les approchess de mise en correspondance entre les CV et les offres d'emploi tout en analysant leurs performances.
Le cinquième chapitre décrit la conception et l'intégration du chatbot, ses cas d'usage, l'architecture adoptée et les résultats obtenus.
Le rapport se conclut par une synthèse générale accompagnée de perspectives d'amélioration.

# Chapitre 1

# Contexte général de projet

### Introduction

Dans ce chapitre, nous présentons l'organisme d'accueil et le contexte de notre stage, avant de situer le projet dans son cadre général. Nous abordons ensuite l'étude de l'existant, en mettant en évidence les solutions disponibles ainsi que leurs limites, de flexibilité et d'interactivité. Cette analyse permet de mieux comprendre les enjeux liés à la gestion des candidatures et de justifier la nécessité d'une approche améliorée. Nous allons expliquer la solution proposée, qui vise à dépasser ces contraintes et à offrir un outil plus adapté aux besoins actuels du recrutement. Enfin, nous présentons l'environnement ainsi que la méthodologie de travail adoptés.

## 1.1 Présentation de l'organisme d'accueil

Le stage est réalisé au sein de l'entreprise AstroLab Group Tunisie, spécialisée dans le développement et l'intégration de solutions digitales. Dans cette section, nous présentons l'entreprise , ses produits, sa mission, sa vision et ses valeurs

### 1.1.1 Aperçu général de l'entreprise

Astrolab Agency est une entreprise innovante de conseil numérique et de services informatiques, dédiée à l'accompagnement des entreprises dans leur transformation digitale. Depuis sa création, Astrolab Agency s'est forgée une solide réputation en proposant des solutions digitales à fort impact, adaptées aux besoins évolutifs de divers secteurs d'activité. L'agence réunit une équipe d'experts passionnés, alliant excellence technique, créativité et approche centrée sur le

client pour favoriser la transformation et la croissance durable. Astrolab Agency se distingue par son engagement envers l'innovation, l'agilité et l'excellence opérationnelle, se positionnant ainsi comme un partenaire stratégique pour les organisations souhaitant réussir leur transition numérique [3].




FIGURE 1.1 – Logo de l'organisme d'accueil

### 1.1.2 Produits et Services

Astrolab Agency propose une large gamme de produits et services, notamment :

- **Développement de logiciels sur mesure :** conception et création d'applications évolutives, sécurisées et intuitives adaptées aux besoins spécifiques des entreprises.
- **Solutions Cloud :** services de migration, de gestion et d'optimisation d'infrastructures cloud pour améliorer l'agilité et l'efficacité opérationnelle.
- **Analyse de données et Business Intelligence :** solutions avancées de traitement et d'analyse de données pour soutenir la prise de décision stratégique.
- **Services de marketing digital :** élaboration de stratégies digitales, SEO, création de contenus et gestion de campagnes pour accroître la visibilité en ligne.
- **Conseil et stratégie IT :** services d'accompagnement et de conseil pour aligner les initiatives technologiques avec les objectifs business.

### 1.1.3 Mission, Vision et Valeurs

**Mission**

La mission d'Astrolab Agency est d'accompagner les entreprises dans leur innovation, leur évolution et leur succès à l'ère du numérique. Pour ce faire, l'agence propose des solutions technologiques sur mesure, conçues pour répondre précisément aux besoins de chaque client. L'objectif est de générer des résultats tangibles, mesurables et durables, en plaçant la performance et la valeur ajoutée au cœur de chaque projet. Cette mission traduit une volonté claire :

devenir un partenaire stratégique des organisations dans leur transformation digitale et non un simple prestataire de services.

**Vision**

Astrolab Agency vise à devenir un acteur de référence International dans le domaine de la transformation digitale. Sa vision s'articule autour de trois piliers majeurs :

— Être reconnu pour son engagement total envers la réussite de ses clients.
— Exploiter les technologies les plus avancées afin d'offrir des solutions innovantes et pertinentes.
— Créer des expériences digitales enrichissantes, favorisant non seulement l'efficacité des entreprises, mais aussi une meilleure interaction avec leurs collaborateurs, partenaires et clients finaux.

À travers cette vision, Astrolab Agency ne cherche pas uniquement à accompagner les entreprises dans leur croissance, mais aussi à anticiper les évolutions du marché et à contribuer activement à façonner l'avenir numérique.

**Valeurs**

Les valeurs fondamentales d'Astrolab Agency représentent les principes directeurs qui guident ses actions, ses décisions et ses relations avec l'ensemble de ses parties prenantes :

— **Innovation :** Explorer en permanence de nouvelles idées, méthodologies et technologies pour concevoir des solutions créatives, pertinentes et pérennes.
— **Excellence :** Viser la plus haute qualité dans chaque projet et garantir une valeur exceptionnelle, quel que soit le niveau de complexité ou l'enjeu.
— **Orientation client :** Placer le client au centre de toutes les préoccupations, en bâtissant des relations solides, transparentes et fondées sur la confiance.
— **Agilité :** S'adapter rapidement aux évolutions constantes du marché et aux avancées technologiques, afin de rester toujours en avance.
— **Intégrité :** Agir avec honnêteté, éthique et responsabilité dans toutes les interactions professionnelles et dans toutes les décisions stratégiques.
— **Collaboration :** Encourager le travail d'équipe, le partage de connaissances et l'intelligence collective, afin d'atteindre des résultats supérieurs et durables.

Ces valeurs constituent le socle identitaire d'Astrolab Agency. Elles assurent la cohérence

de ses actions au quotidien et orientent son développement vers un futur aligné avec sa mission et sa vision.

## 1.2 Description du Stage

Afin d'assurer une transition efficace entre la formation académique et l'insertion professionnelle, le stage de fin d'études constitue une étape clé. Il permet à l'étudiant de mettre en pratique les acquis théoriques, de développer de nouvelles compétences et de s'initier aux exigences du monde du travail.

Ce stage pratique s'inscrit dans le cadre du projet de fin d'études obligatoire pour l'obtention du diplôme de Master en Data Science à la Faculté des Sciences de Monastir pour l'année universitaire 2024-2025. Réalisé au sein d'Astrolab Agency, il offre un environnement stimulant où l'innovation et les technologies numériques occupent une place centrale. Ce contexte m'a permis de contribuer à un projet réel en lien direct avec les besoins actuels des entreprises en matière de digitalisation et d'intelligence artificielle.

Ce stage constitue une étape clé pour préparer l'intégration dans le monde professionnel, en offrant l'opportunité de mettre en pratique les concepts et méthodologies acquis tout au long du cursus. Il permet d'appliquer, dans un contexte réel, les compétences en science des données, en intelligence artificielle et en traitement du langage naturel.

Au cours de cette expérience, j'ai pu :

— Approfondir mes connaissances en IA et NLP ;
— Manipuler et intégrer des LLM dans un contexte applicatif ;
— Développer mes compétences en analyse de données et en ingénierie logicielle ;
— Collaborer au sein d'une équipe multidisciplinaire ;
— Interagir avec des experts du domaine pour affiner mes approches techniques.

## 1.3 Présentation de la problématique

Dans un contexte où le volume de candidatures reçues par les entreprises ne cesse de croître, la gestion manuelle des CV devient rapidement inefficace et chronophage. Les recruteurs passent

une grande partie de leur temps à analyser des documents variés, souvent hétérogènes en termes de format, de structure et de contenu. Cette tâche répétitive, à faible valeur ajoutée, peut non seulement allonger les délais de recrutement, mais aussi accroître le risque d'erreurs humaines et de biais inconscients.

Les systèmes de gestion de candidatures (ATS) se sont alors imposés comme une solution incontournable. Ces outils technologiques remplissent une double mission :

— **Côté candidats :** ils offrent une plateforme centralisée pour déposer leurs CV, suivre l'avancement de leurs candidatures et bénéficier d'une expérience plus structurée.
— **Côté recruteurs :** ils permettent de collecter, d'organiser et de gérer efficacement un volume important de candidatures, en centralisant les CV et en assurant le suivi des différentes étapes du processus de recrutement.

Dans notre cas, l'accent est mis sur l'usage de l'ATS au service des recruteurs, qui ont besoin de trouver rapidement les profils les plus pertinents et les plus adaptés aux offres d'emploi publiées. Cependant, les ATS traditionnels restent limités dans leur fonctionnement : ils se basent principalement sur des filtres simples, comme les mots-clés, les critères fixes ou les champs obligatoires. Cette logique reste superficielle, car elle ne permet pas de comprendre véritablement le contenu d'un CV, d'évaluer les expériences dans leur contexte, ni de saisir les nuances entre différents profils.

Dans un marché de l'emploi compétitif et en constante évolution, cette approche se révèle insuffisante. Les recruteurs ont désormais besoin d'outils capables :

— de comprendre le langage naturel,
— de détecter les compétences implicites et transférables,
— d'analyser les expériences professionnelles dans leur contexte,
— de réaliser une mise en correspondance intelligente entre les CV et les offres d'emploi.

L'absence de ces fonctionnalités avancées entraîne plusieurs limites parmi lesquelles nous pouvons citer la perte de temps considérable dans le tri des candidatures, le risque d'écarter des profils pertinents faute d'une analyse approfondie, le manque de fluidité dans l'expérience proposée, etl'impossibilité d'instaurer une interaction naturelle entre l'utilisateur et le système.

Certes, certaines solutions concurrentes commencent à intégrer des modules d'intelligence artificielle, mais souvent de manière partielle. Par exemple, elles proposent une analyse limitée aux informations basiques d'un CV (coordonnées, diplômes, expériences principales), ou une

mise en correspondance simpliste qui repose uniquement sur la similarité de mots-clés entre CV et offres. De plus, la dimension interactive reste encore marginale : la plupart des ATS ne permettent pas aux recruteurs d'échanger avec le système, de poser des questions ou d'obtenir des recommandations personnalisées en temps réel.

Ainsi, la problématique centrale est la suivante : *comment intégrer efficacement l'intelligence artificielle dans un ATS afin de transformer la gestion des candidatures en un processus intelligent, précis et interactif ?*

L'enjeu est double :

1. Améliorer la pertinence et la rapidité du tri et de la mise en correspondance des CV, en dépassant les limites des approches purement basées sur des filtres ou des mots-clés.
2. Offrir au recruteur une assistance conversationnelle via un chatbot, capable de répondre à ses questions, de l'aider à naviguer dans les données et de proposer des insights pertinents.

## 1.4 Etude de l'existant et solution proposée

Cette section présente une analyse critique de l'existant et expose la solution proposée pour répondre à la problématique identifiée.

### 1.4.1 Solutions existantes

Dans le cadre de cette étude, nous avons analysé différentes solutions déjà mises en place afin d'identifier leurs forces et leurs limites. Parmi elles, la plateforme Talinty dans sa version initiale constitue une référence centrale, puisqu'elle représente la base sur laquelle nous avons construit notre travail.

#### Présentation de Talinty

Talinty est une plateforme de ressources humaines dédiée au recrutement, conçue pour simplifier et optimiser le processus de sélection des candidats. Elle propose un ensemble d'outils pour publier des offres d'emploi, gérer les candidatures et suivre tout le parcours de recrutement.

Talinty est un *Applicant Tracking System (ATS)* qui permet aux recruteurs de gérer leurs offres d'emploi, leurs candidatures et leur pipeline de recrutement. Son objectif est d'aider les recruteurs à gagner du temps, à mieux organiser les informations et à prendre des décisions plus éclairées [64].

Dans sa version initiale (sans intégration d'IA), Talinty propose :

— La publication d'offres d'emploi sur plusieurs canaux.
— La centralisation des candidatures dans une interface unique.
— Des filtres de recherche basés sur des mots-clés, l'expérience ou la localisation.
— La possibilité de collaborer entre recruteurs via un espace partagé.

Cependant, l'analyse des CV reste manuelle. Le recruteur devait :

— Ouvrir chaque CV,
— Lire et interpréter les informations,
— Comparer manuellement les profils à la fiche de poste.

Cette méthode impliquait que chaque responsable de recrutement devait passer du temps à parcourir et analyser chaque document, identifier les informations pertinentes et décider de la suite à donner à la candidature. Cette phase était non seulement chronophage, mais aussi sujette à des biais humains et à des erreurs d'interprétation, notamment lorsque le volume de candidatures était élevé.

Pour pallier partiellement cette contrainte, Talinty utilisait l'API commerciale **Affinda** pour réaliser le *parsing automatique* des CV soumis par les candidats. Ce parsing permettait d'extraire et de structurer des informations clés (coordonnées, expériences professionnelles, diplômes, compétences, etc.) afin d'alimenter dynamiquement la section « Profil candidat » sur Talinty.

### Présentation d'Affinda

Dans la première version de Talinty, avant la mise en place d'une API interne, la plateforme s'appuyait sur une solution externe : **Affinda**. Affinda est une solution spécialisée dans l'extraction automatique de données à partir de CV et de documents RH. Elle utilise des technologies d'intelligence artificielle et de traitement automatique du langage naturel pour analyser des fichiers non structurés et restituer les informations sous forme de données structurées et exploitables [2].

**Fonctionnement :**

— Le recruteur télécharge un CV sur Talinty.

— Ce document est envoyé via l'API Affinda (`CreateDocument`).
— Affinda traite le CV et renvoie un JSON structuré contenant : informations personnelles, expériences professionnelles, formations, compétences, langues, certifications, liens vers des profils en ligne, etc.
— Ces données sont ensuite utilisées pour enrichir la section *Applicant* de Talinty, offrant ainsi au recruteur une vue claire et homogène du profil candidat.

**État initial :** Dans la première version de Talinty, la plateforme avait besoin d'afficher automatiquement les informations clés d'un CV (expériences, formations, compétences...) dans la section *Applicant*, c'est-à-dire la fiche candidat consultée par les recruteurs.

À ce stade, le module **Resume Matcher** n'existait pas encore ; le parsing servait uniquement à enrichir l'affichage dans la section Applicant.

Cette automatisation représentait déjà un gain de temps pour les recruteurs, en évitant une saisie manuelle de base. Toutefois, cette solution restait limitée : elle ne prenait pas en compte la pertinence des compétences par rapport à un poste donné, ni la correspondance précise entre les besoins d'une offre et le profil d'un candidat. De plus, étant payante, elle ne correspondait pas à l'objectif d'Astrolab de disposer d'une solution efficace mais peu coûteuse.




FIGURE 1.2 – Logo de l'API Affinda

### Les concurrents de Talinty

Sur le marché, nous avons distingués qu'il existe plusieurs plateformes qui proposent des fonctionnalités similaires ou complémentaires à celles offertes par Talinty. Ces concurrents se distinguent par leur approche, leurs fonctionnalités et leur intégration de l'IA dans les processus de recrutement. Une analyse comparative est essentielle pour situer Talinty dans son environnement concurrentiel et identifier ses atouts et axes d'amélioration.

#### 1. Workable [73]

C'est une solution SaaS tout-en-un pour la gestion du processus de recrutement.

Workable offre les fonctionnalités suivantes :

— Gestion des candidatures centralisée.
— Parsing automatisé des CV avec extraction d'informations clés.
— Publication d'offres sur plus de 200 job boards.
— Collaboration entre recruteurs via un espace partagé.
— Intégration avec des outils de visioconférence et d'évaluation en ligne.

Cependant, la solution Workable présente certaines limites,

— Personnalisation limitée des workflows pour les besoins spécifiques d'entreprises complexes.
— Coût relativement élevé pour les petites structures.

**2. Lever [34]**

Lever est une solution axée sur la relation et l'engagement candidat (ATS + CRM) et parmi ses fonctionnalités clés, on trouve :

— Gestion proactive des viviers de talents.
— Analyse des données pour optimiser les recrutements.
— Intégration avec des outils de sourcing, d'évaluation et de signature électronique.
— Interface moderne et collaborative.

Mais, Lever presente des limites :

— Peu adapté aux très petites entreprises.
— Certaines fonctionnalités avancées réservées aux plans premium.

**3. SmartRecruiters [59]**

C'est une plateforme de recrutement ouverte et modulable qui offre diverses fonctionnalités clés :

— Parsing des CV et scoring automatique.
— Place de marché d'applications partenaires.
— Gestion multilingue et multi-pays.
— Analyse avancée des KPIs de recrutement.

Cependant, elle présente certaines limites :

— Courbe d'apprentissage pour exploiter pleinement toutes les intégrations.
— Dépendance à des modules externes pour certaines fonctions spécialisées.

**4. Greenhouse [60]**

**Positionnement :** Plateforme centrée sur l'optimisation du processus de sélection.

**Fonctionnalités clés :**

— Guides d'entretien et scorecards pour évaluer les candidats.
— Automatisation de certaines tâches administratives.
— Outils de diversité et inclusion intégrés.
— Reporting détaillé.

**Limites :**

— Complexité de configuration initiale.
— Coût élevé pour les petites structures.

**5. Recruitee [54]**

Recruitee se positionne comme un ATS collaboratif et simple à déployer (ciblant PME et startups) et présente des fonctionnalités clés telles que :

— Interface intuitive.
— Outils de cooptation.
— Publication multi-sites et suivi centralisé des candidatures.
— API ouverte pour intégrations tierces.

Néanmoins, elle comporte des contraintes

— Moins de fonctionnalités avancées d'IA.
— Analyses moins poussées que les solutions premium.

### 1.4.2 Critique des solutions existantes

La première version de Talinty présentait plusieurs limites importantes qui affectent l'efficacité et la qualité du processus de recrutement :

— **Hétérogénéité des CV reçus :** formats variés (PDF, Word, CV graphiques), rendant leur lecture et leur traitement complexes.
— **Analyse manuelle lourde :** nécessité de comparer et d'évaluer chaque candidature de manière hétérogène, augmentant la charge cognitive et le temps de sélection.

— **Temps consacré à l'exploitation des données :** extraction d'informations utiles restait chronophage.

Pour pallier ces contraintes, Talinty avait intégré l'API **Affinda** pour réaliser le parsing automatique des CV. Malgré cet apport, plusieurs limites subsistaient :

— **Coût élevé et faible scalabilité :** dépendance au volume de CV traités.
— **Dépendance externe :** toute évolution de l'API impactait directement Talinty.
— **Personnalisation limitée :** impossibilité d'ajouter des champs ou une logique métier interne.
— **Problèmes de confidentialité :** transfert de données sensibles vers un service tiers.

Ces contraintes ont mis en évidence le besoin de repousser les limites de l'automatisation et d'offrir plus d'autonomie aux recruteurs.

Sur le marché, la majorité des plateformes concurrentes à Talinty, telles que Workable, Lever, SmartRecruiters, Greenhouse ou Recruitee, proposent des fonctionnalités complètes pour la gestion des candidatures, le parsing automatisé des CV et la collaboration entre recruteurs. Cependant, elles présentent plusieurs limites communes : la plupart sont payantes et coûteuses pour les petites structures, certaines offrent un niveau limité d'intégration de l'IA, et plusieurs nécessitent une courbe d'apprentissage importante ou dépendent de modules externes pour exploiter toutes leurs fonctionnalités. Ces contraintes rendent leur adoption moins flexible et adaptée aux entreprises recherchant une solution performante mais économique.

### 1.4.3 Solution proposée

À la lumière des limites identifiées, il est apparu essentiel de repenser le traitement des candidatures. La nouvelle solution repose sur l'intégration d'une **couche d'intelligence artificielle** au cœur de la plateforme, avec trois innovations majeures :

1. **Un Parser interne :** extraction automatique des informations essentielles des CV, réduction des coûts et renforcement de la confidentialité.
2. **Un Matcher intelligent :** évaluation de la pertinence des candidats par rapport aux offres, en tenant compte du contexte et de la proximité sémantique.
3. **Un chatbot conversationnel intégré :** assistance en langage naturel, résumés de profils, recommandations en temps réel.

Ainsi, la nouvelle version de Talinty devient un ATS de nouvelle génération :

— Plus **autonome**, en réduisant sa dépendance à des prestataires externes.
— Plus **efficace**, grâce à l'automatisation et l'analyse intelligente.
— Plus **sûr**, en maîtrisant les données pertinants.

## 1.5 Environnements de développement

Dans cette section, nous exposons les différents environnements utilisés pour la réalisation de notre solution. Il est clair que ces environnements peuvent être classés en deux catégories : matériels et logiciels.

### 1.5.1 Environnements matériels

Le développement de la solution a été réalisé via un ordinateur portable ayant les caractéristiques suivantes :

| Caractéristique | Valeur |
|---|---|
| Modèle PC | Asus |
| Système d'exploitation | Windows 10 pro |
| Processeur | AMD Ryzen 5 3500U Radeon Vega Gfx |
| Mémoire RAM | 8 Go (DDR4, 2666 MHz) |
| Disque dur | 465 Go (SSD M.2 PCIe Gen.3 NVMe) |

TABLE 1.1 – Matériel de base

### 1.5.2 Environnements logiciels

Les environnements logiciels nécessaires à la mise en œuvre de notre projet. Cela inclut une description des différents outils logiciels, frameworks et langages de programmation qui seront utilisés pour développer, tester et déployer notre application.

### Outils logiciels

| Outil | Description | Usage dans le projet | Logo |
|---|---|---|---|
| Visual Studio Code [40] | Éditeur de code source extensible, gratuit et open-source, prenant en charge le débogage, la coloration syntaxique, la complétion intelligente, les snippets et l'intégration Git. | Développement et maintenance de la plateforme |  |
| Git [17] | Système de contrôle de version distribué et open-source, permettant de gérer efficacement l'évolution du code source et de coordonner le travail collaboratif des développeurs. | Gestion de versions et suivi du code source |  |
| GitHub [18] | Plateforme collaborative basée sur Git, utilisée pour héberger, partager et gérer le code source tout en facilitant la collaboration via issues, pull requests et gestion des branches. | Hébergement et collaboration sur le code source |  |
| Postman [51] | Application permettant de concevoir, tester et documenter des API à travers des requêtes HTTP (GET, POST, PUT, PATCH, DELETE), avec interface graphique conviviale. | Tests et validation des API REST |  |
| Swagger [63] | Suite d'outils open-source basée sur la spécification OpenAPI, permettant de concevoir, générer et documenter des API RESTful de manière interactive. | Documentation et simulation des API |  |

Suite à la page suivante...

**– Suite du Tableau 1.2–**

| Outil | Description | Usage dans le projet | Logo |
|---|---|---|---|
| MongoDB [43] | MongoDB2025 Base de données NoSQL orientée documents, flexible et scalable, stockant les informations sous format JSON/BSON et offrant réplication, sharding et indexation avancée. | Stockage et gestion des données non structurées |  |
| MongoDB Compass [42] | Interface graphique officielle de MongoDB, permettant d'explorer, visualiser et administrer les bases de données, d'exécuter des requêtes et d'analyser les performances. | Visualisation et gestion des bases de données |  |
| Docker [15] | Plateforme de conteneurisation open-source permettant d'isoler les applications, d'assurer leur portabilité et de simplifier leur déploiement sur différents environnements. | Conteneurisation et déploiement des services |  |
| Draw.io [14] | Outil libre et en ligne pour créer des diagrammes variés : organigrammes, modèles UML, schémas de réseau et flux de processus. | Conception de diagrammes UML et schémas techniques |  |
| Overleaf [48] | Éditeur en ligne collaboratif dédié à LaTeX, intégrant un contrôle de version, des modèles variés et une compilation en temps réel. | Rédaction et mise en forme du rapport technique |  |
| Zoho Cliq [76] | Outil de communication et de collaboration en temps réel offrant messagerie instantanée, appels audio/vidéo, partage de fichiers et intégrations d'applications. | Communication interne et collaboration d'équipe |  |

Suite à la page suivante...

**– Suite du Tableau 1.2–**

| Outil | Description | Usage dans le projet | Logo |
|---|---|---|---|
| ClickUp [11] | Plateforme de gestion de projets et de productivité centralisant tâches, documents et communications d'équipe, avec support de la méthode agile. | Planification et suivi des tâches du projet | |

TABLE 1.2 – Outils et logiciels utilisés dans le projet

### Langages de programmation

**Python** [52]
Python est un langage de programmation open-source polyvalent, apprécié pour sa simplicité et sa lisibilité. Il est conçu pour être facile à apprendre tout en étant puissant, avec une syntaxe claire qui permet un développement rapide et efficace. Python est largement utilisé dans divers domaines tels que le développement web, l'analyse de données, l'intelligence artificielle, l'automatisation et la science des données. Il bénéficie d'une vaste communauté et d'un écosystème riche de bibliothèques et de frameworks, ce qui en fait un choix de prédilection pour les développeurs souhaitant créer des applications robustes et évolutives.

FIGURE 1.3 – Logo Python

### Environnements d'exécution et frameworks

## 1.6 Méthodologie de travail

La réalisation de ce projet a nécessité une organisation rigoureuse et une gestion efficace des différentes étapes de développement. En collaboration avec notre encadrant en entreprise, nous avons adopté une méthodologie agile adaptée à notre contexte et aux ressources disponibles.

| Framework | Description | Usage dans le projet | Logo |
|---|---|---|---|
| **Flask** [49] | Framework web Python léger et open-source, conçu pour être simple et flexible. Il permet de créer des applications web rapidement et reste très extensible. | Création d'API RESTful et de services web légers. |  |
| **FastAPI** [16] | Framework Python moderne et asynchrone pour la création d'API performantes, avec une documentation automatique et interactive. | Structuration de l'API de matching et de parsing, en assurant une faible latence. |  |
| **LangChain** [32] | Framework spécialisé pour l'orchestration de modèles de langage (LLM), fournissant des abstractions pour la gestion des prompts et l'intégration de données. | Encapsulation de la logique de parsing et de matching en chaînes modulaires. |  |
| **LangGraph**[33] | Extension de LangChain permettant de représenter les interactions avec un LLM sous forme de graphes pour visualiser et contrôler le pipeline IA. | Organisation et monitoring des étapes du processus de matching. |  |

TABLE 1.4 – Environnements d'exécution et frameworks

L'objectif était de maintenir un rythme de progression constant tout en restant flexibles face aux imprévus et aux évolutions des besoins fonctionnels.

### 1.6.1 Choix de Kanban et avantages

Compte tenu de la taille et de la nature de notre projet, nous avons choisi la méthode Kanban, une approche agile légère, flexible et particulièrement adaptée au travail en solo ou en petite équipe. Pour sa mise en œuvre, nous avons utilisé ClickUp, un outil qui permet de créer un tableau Kanban interactif où chaque fonctionnalité ou tâche est représentée par une carte, ce qui facilite leur priorisation et leur suivi.

Nous avons structuré notre tableau autour de colonnes correspondant aux différentes étapes du flux de travail : Backlog, À faire, En cours, En révision/test et Terminées. Afin d'éviter

la surcharge de travail, nous avons défini une limite WIP (Work In Progress) restreignant le nombre de tâches pouvant être menées en parallèle.

L'utilisation de Kanban avec ClickUp nous a permis de bénéficier de plusieurs avantages concrets : une visualisation claire et continue de l'avancement, une flexibilité accrue dans l'ajustement des priorités, une gestion efficace du temps et des urgences grâce aux étiquettes, ainsi qu'un suivi rigoureux de la progression, garantissant la traçabilité et la fluidité des échanges avec notre encadrant.

### 1.6.2 Représentation visuelle du tableau Kanban

Afin de mieux illustrer l'organisation et le suivi des tâches, la figure 1.4 présente une capture réelle du tableau Kanban mis en place sur ClickUp. Chaque carte correspond à une tâche ou fonctionnalité du projet, positionnée dans la colonne appropriée selon son état d'avancement (À faire : TO DO, En cours : IN PROGRESS, En révision/test : IN REVIEW, Terminé : COMPLETE).

Cette visualisation a permis d'avoir une vue d'ensemble claire et dynamique sur l'évolution du projet, tout en facilitant la gestion des priorités et le respect des délais.

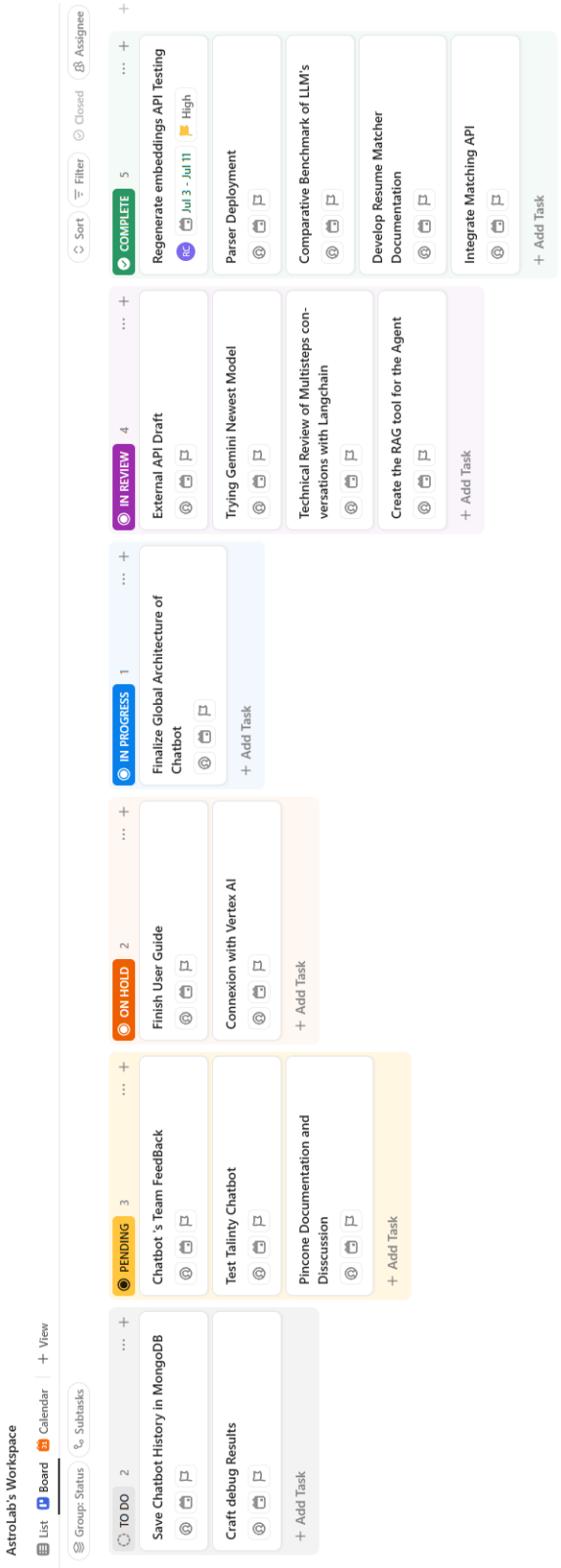



FIGURE 1.4 – Capture du tableau Kanban sur ClickUp

### 1.6.3 Backlog du Produit

le Backlog du produit tourne autour de 3 modules :

**Module 1 : Analyse de CV et Offre d'emploi (Parsing)**

**Épic :** Construire un système fiable qui lit et comprend automatiquement les CV et les offres d'emploi.

**Récit utilisateur 1 :** En tant que recruteur, je veux importer un CV PDF et que le système en extraie automatiquement les informations clés (formation, expériences, compétences) afin de ne plus avoir à le faire manuellement et de pouvoir filtrer les candidats.

**Tâches :**

— Implémenter l'extraction de texte brut depuis les fichiers PDF.
— Appliquer le prétraitement et le nettoyage des données.
— Mettre en œuvre la reconnaissance des sections du CV (formation, expérience, etc.) via une approche NLP.

**Récit utilisateur 2 :** En tant que recruteur, je veux que l'analyse des CV soit la plus précise possible afin de minimiser les erreurs et de ne pas passer à côté d'informations importantes.

**Tâches :**

— Développer un premier modèle d'analyse (baseline) pour mesurer la précision.
— Étudier et tester des technologies d'analyse plus avancées (basées sur les LLM) avec LangChain.
— Comparer les performances et sélectionner la meilleure approche.

## Module 2 : Mise en correspondance CV – Offre d'emploi (Matching)

**Épic :** Mettre en place un système qui évalue les candidats par rapport à une offre d'emploi.

**Récit utilisateur 1 :** En tant que recruteur, pour une offre donnée, je veux obtenir un score qui traduit la compatibilité du CV à l'offre d'emploi.

**Tâches :**

— Développer un score de compatibilité initial basé sur la similarité sémantique (sentence transformers, similarité cosinus).
— Intégrer une base de données vectorielle pour stocker les CV et les offres.

**Récit utilisateur 2 :** En tant que recruteur, je veux que le score de pertinence soit fiable et aille au-delà de la simple correspondance de mots-clés afin de découvrir des candidats qualifiés que j'aurais pu manquer.

**Tâches :**

— Expérimenter avec différents modèles d'*embedding* pour améliorer la pertinence.
— Étudier la possibilité d'utiliser les LLM pour une analyse de compatibilité plus fine (raisonnement avancé).
— Mener des tests comparatifs pour valider et choisir le meilleur modèle de scoring.
— Intégrer le modèle final dans la plateforme.

## Module 3 : Chatbot de Recrutement

**Épic :** Créer un assistant conversationnel intelligent pour aider les recruteurs à explorer leur base de candidats et des offres de travail.

**Récit utilisateur 1 :** En tant que recruteur, je veux pouvoir poser des questions simples au chatbot (ex : "Trouve-moi les candidats qui ont appliquer pour l'offre Java Developper") afin d'obtenir une liste de candidats instantanément.

**Tâches :**

— Analyser les types de questions que les recruteurs pourraient poser.
— Concevoir l'architecture du chatbot.
— Connecter le chatbot à la base de données des candidats pour récupérer les informations.

**Récit utilisateur 2 :** En tant que recruteur, je veux pouvoir affiner ma recherche en discutant avec le chatbot (ex : "Parmi ceux-là, lequel a plus d'experince ?") afin de trouver le profil parfait de manière interactive.

**Tâches :**

— Mettre en place des flux de conversation logiques (ex : avec LangGraph).
— Mener des tests internes et du débogage.

— Organiser des sessions de test avec des recruteurs.
— Recueillir les retours et améliorer le chatbot en continu.

# Conclusion

L'analyse menée dans ce chapitre a permis de mettre en évidence les atouts, mais aussi les limites de la version actuelle de Talinty et des solutions concurrentes. Malgré une base fonctionnelle solide, la plateforme souffrait de contraintes liées au traitement manuel, à l'hétérogénéité des CV et à la dépendance à des outils externes.

Face à ces constats, la nécessité d'une évolution vers une solution plus performante et intelligente s'impose. L'intégration d'un analyseur de CV et d'offre d'emploi interne, d'un moteur de mise en correspondance avancé et d'un chatbot conversationnel constitue une réponse concrète, ouvrant la voie vers un ATS de nouvelle génération, plus précis, interactif et adapté aux recruteurs.

# Chapitre 2

# Etat de l'Art : Méthodes et Techniques de Recrutement

## Introduction

Dans ce chapitre, nous passons en revue les concepts, méthodes et approches existantes liés à notre problématique. Cette étude permet de dégager les limites actuelles et de justifier la pertinence de notre contribution.

## 2.1 Évolution des méthodes de recrutement

Cette section présente l'evolution du recrutement traditionnel vers un recrutement assisté par l'IA

### 2.1.1 Le recrutement traditionnel : un processus manuel et subjectif

Avant la révolution numérique, le recrutement s'appuyait principalement sur la presse écrite, le bouche-à-oreille et les réseaux de proximité. Les candidats soumettaient leur dossier sous format papier (CV et lettre manuscrite) envoyé par courrier ou déposé en main propre. Ce processus mettait l'humain au centre, chaque interaction étant une occasion pour le candidat

de marquer une première impression. Toutefois, cette approche présentait plusieurs limites : elle était lente, lourde administrativement et fortement influencée par la subjectivité et les biais du recruteur. De plus, la gestion d'un volume croissant de candidatures devenait rapidement difficile pour les entreprises attractives [7].

### 2.1.2 La digitalisation et les premiers outils numériques

L'essor d'Internet à la fin des années 1990 a profondément transformé le recrutement. Les plateformes en ligne, telles queles ATS Monster [44], puis LinkedIn [35] ou Indeed [25], ont facilité la diffusion des offres et accéléré la réception de candidatures dématérialisées, permettant aux entreprises d'atteindre un public beaucoup plus large.

Dans le même temps, les premiers Applicant Tracking Systems (ATS) sont apparus pour automatiser la gestion des candidatures. Ces logiciels offraient trois fonctions principales[9] :

— **Centralisation** : toutes les candidatures étaient regroupées dans une base unique.
— **Filtrage automatique** : recherche par mots-clés ou critères prédéfinis (diplôme, expérience, compétences techniques).
— **Suivi des candidats** : visualisation des étapes du processus (candidature reçue, entretien planifié, rejeté, embauché).

Ces innovations représentent un gain de temps considérable. Toutefois, elles restent limitées par leur approche purement syntaxique, elles excluent parfois des profils pertinents faute de correspondance exacte des termes. Cette rigidité a mis en évidence la nécessité d'outils plus intelligents et contextuels.

### 2.1.3 L'essor de l'Intelligence Artificielle dans le recrutement

L'IA regroupe les techniques permettant aux machines d'imiter certaines fonctions cognitives humaines, comme l'apprentissage, le raisonnement ou le traitement du langage. Dans les ressources humaines, elle s'est particulièrement développée dans le recrutement, où le volume important de données nécessite une sélection rapide et efficace des candidats.

L'introduction de l'IA dans les ressources humaines a marqué une véritable rupture par rapport aux ATS, qui se limitent à un tri basique des candidatures. Les solutions basées sur l'IA apportent uneintelligence contextuelle, capable d'analyser et d'interpréter le contenu d'un dossier au-delà des simples mots-clés.

Elles reposent sur plusieurs avancées : l'**analyse sémantique** et le **NLP** (Natural Language Processing), qui comprennent le sens global des phrases ; le **scoring de compatibilité**, qui mesure l'adéquation entre profil et offre ; la **détection de compétences implicites**, inférées à partir du parcours ; et la **prédiction de performance**, estimée grâce aux données historiques. De plus, l'intégration des modèles de grande taille (**LLM**) permet d'améliorer la compréhension fine du langage naturel, de générer des représentations plus riches des profils et des offres.

Ainsi, l'IA permet par exemple de rapprocher un candidat qui maitrise la «reconnaissance d'images» d'un poste en «computer vision», même si les termes diffèrent. Elle ne se limite plus à réduire la charge de travail des recruteurs, mais améliore considérablement la pertinence et la qualité des résultats obtenus [6].

### 2.1.4 Vers un modèle hybride homme-machine

Cette évolution ne signifie pas la disparition du recruteur. Au contraire, elle redéfinit son rôle. L'IA prend en charge les opérations répétitives et quantitatives (tri massif de CV, analyse sémantique, correspondances automatiques), tandis que l'humain conserve la responsabilité des dimensions qualitatives [39]. En effet, il reste encore des aspects difficilement mesurables par une machine tels que Les soft skills, l'adéquation culturelle ou encore la motivation d'un candidat.

Le recrutement assisté par l'IA s'inscrit donc dans un modèle hybride, où la technologie et l'expertise humaine se complètent. Le recruteur devient moins un gestionnaire de candidatures et davantage un décideur éclairé, capable de se concentrer sur ce qui fait la valeur ajoutée du métier : la relation humaine, l'évaluation des potentiels et la construction de la confiance.

### 2.1.5 Domaines clés d'application de l'IA dans le recrutement

L'intelligence artificielle s'intègre à l'ensemble du cycle de recrutement, depuis la recherche de talents jusqu'à l'évaluation finale des candidats, redéfinissant progressivement la fonction RH [6].

#### Le tri et l'analyse des candidatures

L'IA optimise le*parsing* et le *matching* des CV, extrayant rapidement les informations essentielles (diplômes, expériences, compétences). Grâce à l'analyse sémantique, elle comprend le contexte et rapproche des termes équivalents, améliorant ainsi la pertinence du tri des candidats.

#### Le sourcing et l'attraction de talents

Les systèmes d'IA identifient également des profils passifs sur des bases de données professionnelles et proposent un score de compatibilité, facilitant la sélection et la personnalisation des approches de recrutement à l'échelle mondiale.

**Les chatbots et l'assistance aux candidats**

Les assistants conversationnels disponibles 24h/24 répondent aux questions, collectent des informations préliminaires et peuvent automatiser des pré-entretiens, améliorant l'expérience candidat et déchargeant les recruteurs des tâches répétitives.

**L'évaluation et la prédiction des performances**

L'IA contribue à l'évaluation des compétences et à la prédiction de réussite via des tests et simulations couplés à des modèles prédictifs, offrant un aperçu des performances et de la fidélité future des candidats, tout en soulevant des questions éthiques.

## 2.2 Le traitement du langage naturel (NLP)

Le NLP est une branche de l'informatique qui permet aux machines de comprendre et d'interpréter le langage humain. Contrairement au traitement de texte classique, il ne se limite pas aux symboles ou aux mots-clés, mais prend en compte la structure, le contexte et les nuances du langage pour extraire le sens et l'intention des phrases

### 2.2.1 Les categories de NLP

Le langage naturel présente de nombreux défis : ambiguïté, polysémie, métaphores, ironie ou sarcasme compliquent l'interprétation même pour les humains [29]. Au debut, le NLP est basé sur des règles manuelles complexes, le NLP a évolué vers des méthodes d'apprentissage automatique et statistiques, plus flexibles, évolutives et capables de gérer la complexité du langage moderne.

Le NLP est divisé en deux catégories clés :

— **NLU (Natural Language Understanding)** : compréhension du langage naturel ;
— **NLG (Natural Language Generation)** : génération de langage naturel.

Dans notre projet, nous nous intéressons à la NLU.

**NLU (Compréhension du langage naturel)**

La NLU est un sous-ensemble du NLP. Elle exprime la capacité d'une machine à comprendre le langage naturel sous une forme fournie par les humains. La NLU prend du texte en entrée, comprend le contexte et l'intention.

La figure 2.1 met en évidence les différences entre NLU et NLP.

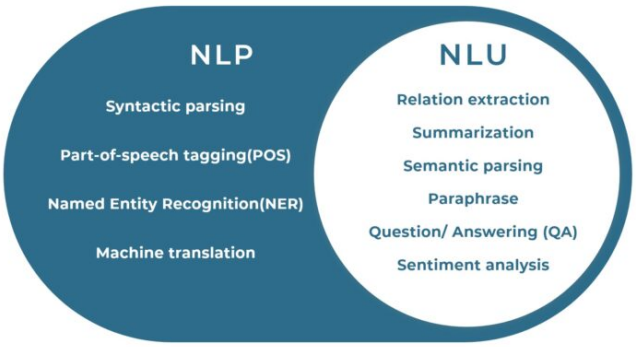



FIGURE 2.1 – Différences entre NLU et NLP [19]

La classification d'intention est un élément important de tout système de compréhension du langage naturel (NLU) et dans toute plateforme de chatbot. Elle consiste à associer les énoncés de l'utilisateur à des significations prédéfinies appelées *intentions*.

Pour qu'un chatbot fonctionne bien, il doit reconnaître correctement l'intention de l'utilisateur à partir de ses entrées afin de déclencher l'action ou la réponse adéquate.

Exemples :

— "Bonjour", "Bonsoir", "Salut" → *Salutation*.

— "Pourquoi la connexion est toujours suspendue ?", "Ma connexion ne fonctionne plus ?" → *Connexion suspendue*.

### 2.2.2 Les étapes du NLP

Dans le domaine du NLP, il existe plusieurs étapes qui consistent à prétraiter et transformer les informations en entrée en un jeu de données exploitables [38].

### Tokenisation

La **tokenisation** est la première étape de prétraitement. C'est un moyen de séparer un texte ou une phrase en une liste d'unités plus petites appelées *jetons* (tokens). Les jetons peuvent être des mots, des caractères ou des sous-mots.

Cette opération peut se faire avec des règles simples (séparation par espaces, ponctuation...), mais aussi à l'aide de méthodes statistiques comme *WordPiece, Tokenization* ou *SentencePiece Tokenization*.

### Morphologie

La morphologie est le domaine de la linguistique qui analyse la structure interne des mots. Il existe deux approches principales : le *stemming* et la *lemmatisation*.

#### a- Stemming

Le stemming est l'approche classique en linguistique. Les mots sont réduits à leurs racines, qui ne sont pas forcément les mêmes que celles définies dans un dictionnaire. Cette approche est simple mais parfois approximative.

#### b- Lemmatisation

La lemmatisation consiste à ramener un mot à sa forme canonique (lemma) en se basant sur un dictionnaire. Elle est plus précise que le stemming mais plus coûteuse en calcul.

La figure 2.2 présente les différences entre le stemming et la lemmatisation.

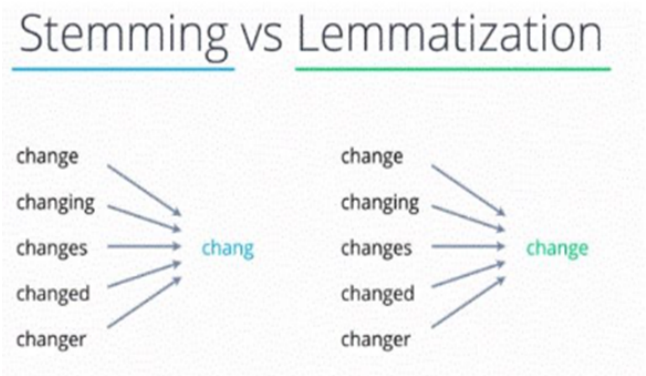



FIGURE 2.2 – Différences entre stemming et lemmatisation

### Vectorisation du texte

Pour que le traitement du langage naturel fonctionne, il faut transformer le langage naturel en une représentation numérique [41].Il existe plusieurs modeles :

### Bag of words (BoW)

Le modèle BoW représente un texte comme un vecteur de taille égale au vocabulaire. Chaque mot est codé par une valeur binaire (présence/absence) ou une fréquence.

Exemple pour la phrase : "J'adore le domaine de NLP" :

$$Vocabulaire = \{J, adore, le, domaine, de, NLP\}$$

$$J = [1,0,0,0,0,0], \quad adore = [0,1,0,0,0,0], \quad NLP = [0,0,0,0,0,1]$$

### TF-IDF (Term Frequency – Inverse Document Frequency)

Le TF-IDF pondère l'importance d'un mot dans un document relativement à sa fréquence dans l'ensemble du corpus.

La formule est :

$$q(w) = f_d(w) \cdot \log \frac{D}{f_D(w)}$$

où :

- $f_d(w)$ est la fréquence du mot $w$ dans le document $d$ ;
- $f_D(w)$ est le nombre de documents contenant $w$ ;
- $D$ est le nombre total de documents.

### Word2Vec

Word2Vec permet de représenter les mots par des vecteurs continus, capables de capturer leurs relations sémantiques et syntaxiques. Deux architectures existent :

- **CBOW (Continuous Bag of Words)** : prédit un mot à partir de son contexte ;
- **Skip-Gram** : prédit le contexte à partir d'un mot central.

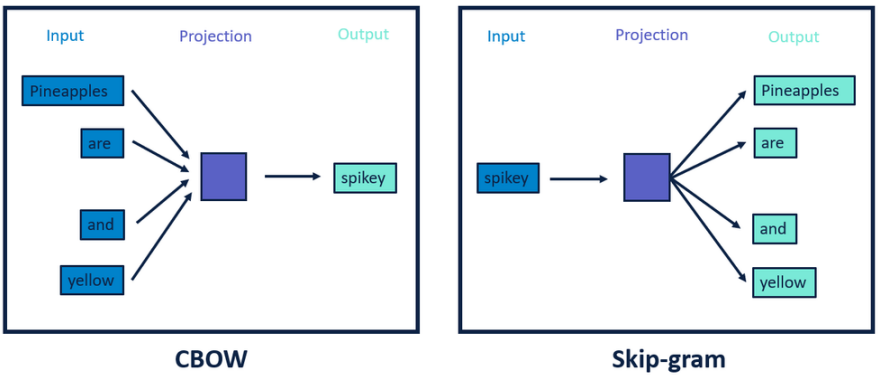



FIGURE 2.3 – Architectures CBOW et Skip-Gram de Word2Vec

## 2.3 Modèles de langage avancés et Transformers

Le NLP a connu une avancée majeure avec les modèles de langage avancés, capables de saisir les nuances contextuelles et de générer du texte proche du langage humain. Ces modèles reposent sur les architectures Transformers, qui améliorent la compréhension sémantique, la généralisation et les performances sur des tâches complexes telles que l'analyse de CV ou la génération de réponses personnalisées. Ils représentent l'évolution des approches statistiques vers les grands modèles de langage (LLM) actuels.

### 2.3.1 L'architecture Transformer

L'introduction du Transformer par Vaswani et al. (2017) a marqué une véritable rupture dans le domaine du traitement automatique du langage [68]. Le Transformer repose sur un mécanisme central : l'attention, et plus précisément l'attention multi-têtes (multi-head attention).

Le principe de l'attention consiste à attribuer un poids relatif à chaque mot d'une séquence en fonction de son importance contextuelle par rapport aux autres. Ainsi, le modèle peut établir des liens même lorsque les mots sont éloignés dans le texte, comme relier un pronom à son antécédent plusieurs phrases auparavant [5]. Dans l'attention multi-têtes, plusieurs mécanismes d'attention fonctionnent en parallèle, chacun capturant des relations différentes (syntaxe, sémantique, dépendances longues, etc.), ce qui permet une meilleure richesse de représentation.

L'architecture Transformer est composée de deux blocs principaux :

— **L'encodeur :** il reçoit une séquence en entrée (par exemple, un CV ou une description

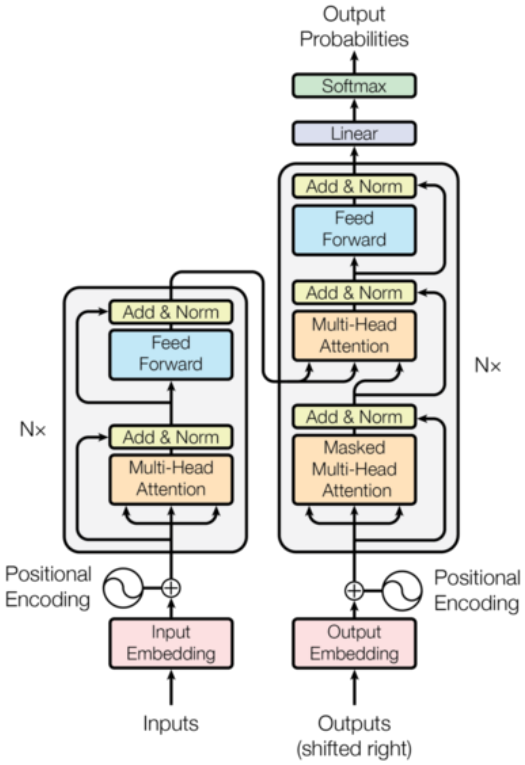



FIGURE 2.4 – Schéma général de l'architecture Transformer [68]

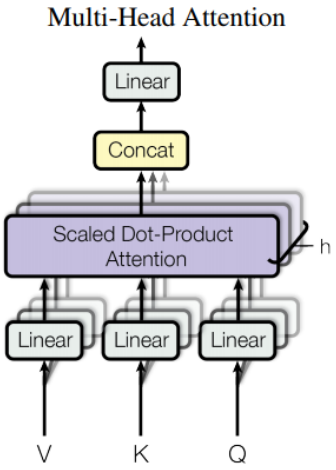



FIGURE 2.5 – Le mécanisme de l'attention [5]

de poste) et génère une représentation vectorielle enrichie de ses relations contextuelles.

— **Le décodeur :** il exploite cette représentation pour produire une séquence en sortie, comme une traduction, une reformulation ou un résumé.

L'empilement de plusieurs couches d'encodeurs et de décodeurs favorise la compréhension fine et la génération cohérente du langage. Dans certaines variantes (par ex. BERT), seule la partie encodeur est utilisée, tandis que dans d'autres (par ex. GPT), seule la partie décodeur est

conservée.

Les Transformers permettent de traiter les séquences en parallèle, contrairement aux RNN et LSTM qui analysaient les données mot par mot. Cette parallélisation accélère l'apprentissage et améliore la capture des dépendances longues, rendant possible l'entraînement de modèles sur des corpus massifs.

Cette architecture a donné naissance à des modèles spécialisés : :

— **BERT (Bidirectional Encoder Representations from Transformers)** et ses dérivés (RoBERTa, CamemBERT) sont principalement utilisés pour les tâches de compréhension (classification de textes, recherche d'information, appariement CV-offre d'emploi) [13].

— **GPT (Generative Pre-trained Transformer)** et ses variantes (GPT-2, GPT-3, GPT-4) sont davantage orientés vers la génération de texte (dialogues, rédaction automatique, résumés, réponses aux candidats) [53].

Ainsi, Le Transformer a ouvert la voie aux grands modèles de langage (LLM) et aux applications d'IA générative. Il est important de comprendre comment ces modèles représentent et exploitent le texte. Au cœur de cette représentation se trouvent les embeddings, c'est-à-dire des vecteurs numériques qui capturent les relations sémantiques entre les mots ou documents. Ces embeddings permettent de transformer le langage naturel en données exploitables par des modèles de machine learning ou pour la recherche sémantique.

### 2.3.2 Modèles d'embeddings

Les modèles d'embeddings jouent un rôle clé dans le NLP moderne, car ils transforment les phrases ou documents en vecteurs numériques qui préservent les relations sémantiques. Ces représentations vectorielles sont ensuite utilisées pour le recherche sémantique, le matching CV-offres d'emploi ou encore la recommandation de profils.

Un résumé des principaux modèles d'embeddings est présenté dans le tableau 2.1.

Pour que ces représentations vectorielles soient réellement exploitables à grande échelle, elles doivent être stockées et recherchées efficacement. C'est le rôle des bases vectorielles, qui permettent de retrouver rapidement les textes les plus proches dans l'espace vectoriel.

### 2.3.3 Bases vectorielles

Pour exploiter pleinement la puissance des embeddings, il est nécessaire de stocker et rechercher efficacement ces représentations vectorielles. Les bases vectorielles (*vector databases*) sont

| Modèle | Points forts | Limites | Cas d'usage |
|---|---|---|---|
| all-MiniLM-L6-v2[70] | Léger, rapide, latence faible | Moins précis sur nuances sémantiques complexes | Scénarios temps réel, matching rapide |
| all-mpnet-base-v2[61] | Bon compromis qualité/rapidité | Légèrement plus lent que MiniLM | Recherches sémantiques, appariement CV-offres |
| BAAI/bge-large-en[75] | Excellente précision, très performant en recherche sémantique | Latence et ressources élevées | Cas où la qualité prime sur la rapidité |
| Sentence-T5[46] | Très performant sur tâches multilingues | Plus lourd que MiniLM | Matching dans contextes multilingues |
| E5-Large[69] | Optimisé pour retrieval & QA | Gourmand en ressources | Requêtes complexes, analyse documentaire |

TABLE 2.1 – Comparatif des modèles d'embeddings

spécialement conçues pour gérer ces tâches en assurant des recherches rapides par similarité. Elles sont largement utilisées dans les systèmes de recommandation, la recherche sémantique, et plus récemment dans les applications d'IA générative (via le *retrieval-augmented generation*, RAG).

Les principales bases vectorilles incluent :

**Qdrant [65]** Une base open source, performante pour la recherche sémantique. Elle supporte la recherche hybride (mots-clés + vecteurs) et s'intègre facilement avec les modèles d'embeddings.

**Pinecone [24]** Une solution managée dans le cloud, orientée vers la scalabilité. Elle fournit une API prête à l'emploi avec gestion simplifiée. Pinecone est payante, mais permet de réduire les coûts d'infrastructure interne.

**Weaviate[67]** Une base open source et extensible. Elle supporte la recherche multimodale (texte, image, audio) et dispose de connecteurs avec divers LLM.

**FAISS (Facebook AI Similarity Search)[28]** Une bibliothèque optimisée par Meta, très performante sur de gros volumes de données. Elle nécessite cependant une gestion manuelle, car ce n'est pas une base de données prête à l'emploi.

Les embeddings servent ensuite de fondation aux modèles de langage avancés (LLM). Ces modèles utilisent ces représentations vectorielles pour comprendre le contexte, générer du texte cohérent et effectuer des tâches complexes comme le résumé, la classification ..

### 2.3.4 Analyse comparative des modèles LLM modernes

Un résumé des principales caractéristiques des modèles LLM modernes, est présenté dans le tableau 2.2.

| **Modèle** | **Ouverture** | **Contexte max** | **Points forts** | **Limites** |
|---|---|---|---|---|
| GPT-4o [26] | Fermé | ∼128k tokens | Qualité, robustesse | Coût, dépendance API |
| Gemini 1.5 [12] | Fermé | 1M tokens | Multimodalité, intégration Google | Moins accessible |
| LLaMA 3.3 [20] | Ouvert | ∼128k tokens | Open source, flexible | Déploiement technique |
| Mistral [23] | Ouvert | 32–64k tokens | Léger, rapide | Communauté jeune |
| Claude 3 [4] | Fermé | 100k–200k tokens | Contexte long, sécurité | API payante |

TABLE 2.2 – Tableau comparatif des modèles de langage récents

Plusieurs modèles de langage se distinguent. **GPT** (OpenAI) offre robustesse, polyvalence et un large écosystème, mais reste coûteux et dépendant de l'API ; il est utilisé pour la génération de descriptions d'emploi, le matching CV–offres et les chatbots RH. **Gemini** (Google DeepMind), successeur de PaLM 2, apporte une multimodalité native et une intégration à l'écosystème Google, mais son accès demeure limité. Il est adapté à la recherche sémantique et à l'analyse de CV complexes. **LLaMA** (Meta), open source et soutenu par une large communauté, assure flexibilité et contrôle local, bien qu'il nécessite une expertise technique. Il convient aux entreprises voulant réduire leur dépendance aux solutions propriétaires. Enfin, **Claude** (Anthropic) met l'accent sur la sécurité, l'explicabilité et le traitement de longs contextes, mais son usage est encore restreint. Il est pertinent pour l'analyse de bases massives de CV.

Ces modèles reposent largement sur le *prompting*, qui consiste à formuler des instructions ou requêtes adaptées afin d'orienter et d'optimiser leurs réponses. La qualité des résultats dépend fortement de la précision et de la pertinence des prompts, faisant de cette compétence un élément central pour tirer pleinement parti des modèles de langage dans les processus de recrutement et de gestion des talents.

### 2.3.5 Prompt Engineering

Le *Prompt Engineering* regroupe les techniques de formulation et de structuration des requêtes adressées aux grands modèles de langage (LLM) pour en optimiser les réponses. Contrairement aux méthodes classiques nécessitant un entraînement complexe, il permet de spécialiser un modèle généraliste simplement en reformulant la consigne d'entrée.

#### Principes fondamentaux du Prompt Engineering

Le *Prompt Engineering* consiste à formuler des requêtes de manière précise pour influencer directement les réponses des LLM. Une consigne structurée produit des résultats plus ciblés et exploitables [36]. Les principaux leviers sont :

— **La contextualisation** : fournir au modèle un contexte riche (extraits de CV, description du poste, statut de la candidature) pour ancrer la réponse dans les données réelles.
— **La précision de la consigne** : indiquer clairement le format de sortie attendu (liste, tableau, résumé, explication argumentée).
— **La contrainte de style ou de ton** : orienter la génération vers un langage formel, concis, ou au contraire plus explicatif.
— **La gestion des rôles** : formuler le prompt en assignant un rôle explicite au modèle (ex. : « Tu es un recruteur qui analyse des candidatures... »).

#### Techniques de Prompting

Plusieurs techniques de prompting sont utilisées pour optimiser les performances des LLM :

— **Zero-shot prompting** : poser une tâche directement sans exemples [72].
— **Few-shot prompting** : inclure dans le prompt quelques exemples d'entrées et de sorties attendues pour guider le modèle [8].
— **Chain-of-Thought prompting** : demander explicitement au modèle de raisonner étape par étape avant de donner une réponse finale, afin d'améliorer la fiabilité du raisonnement [71].
— **Instruction tuning et fine-tuning léger** : combiner prompts et modèles adaptés à des tâches spécifiques, en s'appuyant sur des corpus spécialisés.
— **RAG (Retrieval-Augmented Generation)** : enrichir dynamiquement le prompt avec des informations pertinentes récupérées depuis une base de données (par ex. un ensemble de CV ou d'offres d'emploi) [47].

# 2.4 Analyse des données de recrutement : Parsing

Cette section introduit d'abord le concept général de parsing, avant d'expliquer son application spécifique aux CV, puis d'exposer les principales techniques utilisées, depuis les approches classiques basées sur des règles jusqu'aux méthodes modernes reposant sur le NLP et l'apprentissage automatique.

## 2.4.1 Définition du Parsing

Le terme **parsing** provient du domaine de la linguistique et de l'informatique. Il désigne le processus consistant à analyser une séquence d'éléments (texte, code, données brutes) afin d'en extraire une structure hiérarchique et compréhensible par une machine. En informatique, le parsing est largement utilisé pour :

— Compiler du code source (parsing syntaxique en langage de programmation).
— Analyser des documents textuels pour en extraire des entités.
— Transformer des données brutes (par exemple, fichiers XML, JSON ou HTML) en formats structurés exploitables.

Dans le NLP, le parsing analyse la structure grammaticale et syntaxique des phrases. Il identifie les éléments clés (sujet, verbe, objet, etc.) et les relations entre mots. Cette étape est essentielle pour permettre à une machine de comprendre le langage humain..

Le parsing est appliqué au recrutement en analysant automatiquement des CV et les offres d'emploi aux formats variés (PDF, Word, texte ou images) pour en extraire des informations structurées.

Il transforme des documents non structurés en formats exploitables par les systèmes de gestion des candidatures (ATS), incluant :

— **Données personnelles :** nom, prénom, adresse e-mail, numéro de téléphone.
— **Parcours académique :** diplômes, établissements, dates.
— **Expériences professionnelles :** postes occupés, entreprises, durée, missions.
— **Formations complémentaires et certifications.**
— **Compétences requises**.

Cette approche permet un traitement rapide et standardisé des candidatures, facilitant le travail des recruteurs [21].

### 2.4.2 Techniques classiques du Parsing

Historiquement, les premières solutions de *parsing* reposent sur des approches basées sur des règles et des mots-clés :

— Expressions régulières (Regex) pour détecter numéros, e-mails ou dates
— Dictionnaires et ontologies pour repérer des noms d'écoles, de diplômes ou de compétences.
— Heuristiques pour segmenter un CV en sections (formation, expériences, compétences).

Ces méthodes, simples, restent limitées face à la diversité des formats, la variabilité du langage et les différences culturelles ou linguistiques [27].

### 2.4.3 Approches modernes basées sur le NLP et l'apprentissage automatique

Avec l'essor du Machine Learning et du NLP moderne, de nouvelles approches se sont développées pour rendre le parsing plus robuste et précis :

— Modèles supervisés (SVM, CRF, Random Forests) entraînés sur des corpus annotés de CV pour reconnaître les entités (nom, poste, diplôme) [30].
— Techniques de Named Entity Recognition (NER) qui permettent d'extraire automatiquement des entités nommées à partir du texte [31].
— Représentations vectorielles (Word2Vec, GloVe, FastText) qui permettent de capturer les relations sémantiques entre mots [50].
— Transformers (BERT, RoBERTa) pour un parsing contextuel, capable de distinguer, par exemple, « Python » le langage de programmation de « Python » le reptile [37].

# 2.5 Mise en correspondance CV–offre d'emploi : Matching

Le Matching est une étape essentielle du recrutement automatisé. Contrairement au parsing qui extrait les informations liés au recrutement, il compare ces informations pour évaluer la compatibilité entre candidat et offre d'emploi. Appuyées par le NLP, le Machine Learning et les modèles de langage, les méthodes modernes dépassent la simple comparaison de mots-clés pour fournir un score contextuel. Cette section présente ses fondements, les données exploitées ainsi que les approches classiques et avancées utilisées.

## 2.5.1 Définition du Matching

Le **Matching** est le processus par lequel un système informatique évalue dans quelle mesure un CV correspond à une offre d'emploi donnée. L'objectif principal est de faciliter la présélection des candidats en fournissant un score ou une mesure de compatibilité basée sur l'analyse des informations extraites des documents.

Concrètement, le Matching repose sur l'identification et la mise en relation de différentes catégories d'informations, telles que :

- **Compétences techniques et fonctionnelles :** langages de programmation, outils métiers, certifications.
- **Expériences professionnelles :** postes occupés, missions réalisées, durée des expériences.
- **Formation et diplômes :** niveau d'études, spécialisation, établissement fréquenté.
- **Soft skills et traits comportementaux :** leadership, gestion de projet, esprit d'équipe, lorsque ces informations sont disponibles dans le CV ou l'offre d'emploi.

Le Matching utilise le NLP pour aller au-delà de la simple correspondance de mots, en comprenant le contexte et la sémantique des CV et offres d'emploi. Il identifie des correspondances implicites, comme «gestion de projet agile» pour un poste de «Scrum Master». Cette technique optimise la sélection des candidats et fournit aux recruteurs des recommandations objectives, tout en permettant l'intervention humaine pour la décision finale.

### 2.5.2 Techniques classiques de Matching

Avant l'avènement du NLP avancé et des réseaux neuronaux, la mise en correspondance entre CV et offre d'emnpoi, reposait sur des méthodes simples basées sur des règles, des mots-clés et des statistiques textuelles.

Les premiers systèmes de matching fonctionnaient de manière relativement simple : ils recherchaient la présence ou l'absence de certains mots-clés dans les CV par rapport à l'offre d'emploi.

- **Exemple :** si une offre mentionne "Python", "machine learning" et "SQL", l'algorithme scanne les CV pour repérer ces termes exacts.
- Des pondérations pouvaient être attribuées (ex. Python = 2 points, SQL = 1 point) afin de hiérarchiser les candidatures.
- Cette logique "si-alors" pouvait également intégrer des règles manuelles définies par les recruteurs, comme exiger une certaine certification ou un minimum d'années d'expérience.

Ces approches sont simples à mettre en œuvre et interprétables par les recruteurs.Par contre elles restent rigides, peu flexibles face aux variations de langage

Pour dépasser le simple repérage de mots-clés exacts, des outils plus structurés ont été introduits :

- **Regex (expressions régulières) :** pour extraire motifs spécifiques (dates, numéros, années d'expérience).
- **Dictionnaires et ontologies :** pour gérer synonymes et relations hiérarchiques (ex. «Java» et «Python» dans «langages de programmation»).

Ces techniques améliorent la précision en introduisant un niveau de compréhension sémantique limité.

#### Métriques statistiques : TF-IDF et Cosine Similarity

- **TF-IDF (Term Frequency – Inverse Document Frequency) :**pour pondérer l'importance des termes dans un CV ou une offre. Un mot courant comme "projet" aura peu de poids, tandis qu'un mot plus spécifique comme "Spark" ou "TensorFlow" sera fortement valorisé'(cf. Chapitre 2, Section 2).
- **Cosine Similarity :** mesure la similarité entre deux vecteurs de texte (CV et offre d'emploi) en calculant l'angle entre eux. Plus l'angle est petit (valeur proche de 1), plus les deux textes sont similaires.

☞ **Exemple :** un CV mentionnant fréquemment "analyse de données", "Python" et "SQL" sera considéré comme proche d'une offre contenant ces termes.

Les méthodes classiques de **matching**, bien que simples et rapides, présentent plusieurs limites :

— Difficulté à gérer **synonymes**, **reformulations** et **variations linguistiques**.
— **Manque de contexte** : un mot présent ne garantit pas la pertinence.
— **Rigidité** : toute mise à jour nécessite une intervention manuelle.

Ces contraintes ont motivé le passage à des approches plus avancées basées sur le **NLP** et le **machine learning**, capables de mieux saisir la complexité du **langage naturel**.

### 2.5.3 Approches modernes avec Transformers et LLM

Les approches contemporaines du Matching s'appuient sur les modèles Transformers et les Large Language Models (cf. Chapitre 2, section 2.3) , qui offrent une compréhension sémantique profonde et contextuelle des textes. Elles surpassent ainsi les méthodes classiques fondées sur des règles ou sur des représentations statistiques limitées, telles que le TF-IDF ou le Bag-of-Words.

Les modèles Transformers, tels que BERT , génèrent des embeddings contextuels : chaque mot est représenté en tenant compte de son contexte dans la phrase.

— **Exemple :** le mot "Java" dans "Java Developer" est compris comme un langage de programmation, et non comme l'île indonésienne.

Cela améliore considérablement la qualité de la comparaison entre un CV et une offre d'emploi, car les nuances sémantiques sont prises en compte.

Les Transformers permettent de transformer les CV et les descriptions de poste en vecteurs dans un espace sémantique. La similarité est ensuite mesurée à l'aide de métriques comme la cosine similarity appliquée sur ces embeddings. Par exemple le Sentence-BERT (cf. Chapitre 2, Section 3.1) et ses variantes, spécialement conçues pour mesurer la similarité entre phrases et documents. Elle a l'avantage d'offrir une meilleure gestion des synonymes, paraphrases et formulations diverses.

Les LLM enrichissent le matching par une analyse contextuelle approfondie, la normalisation

des textes, l'extraction de compétences implicites et la génération d'explications interprétables pour les recruteurs.

Ces approches exigent d'importantes ressources de calcul, restent vulnérables aux biais des données, présentent un coût élevé à grande échelle et souffrent d'un manque de transparence lié à leur fonctionnement de "boîte noire".

## 2.6 Chatbots et systèmes conversationnels

Cette section aborde les systèmes conversationnels : leur définition, leur rôle, leurs composants, leurs limites et leurs défis.

### 2.6.1 Définition et rôle

Les chatbots, ou agents conversationnels, sont des systèmes capables de dialoguer en langage naturel afin de répondre aux questions, assister les utilisateurs ou automatiser des tâches répétitives [1]. Dans le domaine RH, ils facilitent les échanges entre candidats et recruteurs, optimisent l'expérience utilisateur et renforcent l'efficacité opérationnelle. Reposant sur des technologies avancées comme le NLP et les LLM, ils intègrent la gestion du contexte et l'accès à des bases de connaissances pour offrir des réponses cohérentes et adaptées. Au-delà de simples scripts, ils s'imposent désormais comme de véritables assistants intelligents capables de s'ajuster aux besoins spécifiques des processus de recrutement.

### 2.6.2 Composants clés d'un chatbot

Le fonctionnement d'un chatbot repose sur un ensemble de compasnats clés interconnectés, chacun jouant un rôle spécifique pour assurer une interaction fluide et pertinente avec l'utilisateur [58].

**NLP/NLU** : Le traitement automatique du langage naturel((cf. Chapitre 2, Section 2)) permet d'analyser les requêtes, d'identifier les intentions et d'extraire les entités importantes, assurant une compréhension contextuelle au-delà des simples mots-clés.

**Moteur de dialogue** : Le moteur de dialogue gère la logique conversationnelle, détermine les

réponses appropriées et maintient la cohérence des échanges dans les conversations multi-turn.

**Base de connaissances et connecteurs externes** : Le chatbot accède à des bases de données internes ou externes via des API pour fournir des réponses précises. Des outils avancés, tels que les bases vectorielles ou moteurs de recherche sémantique, permettent de retrouver des informations même si elles ne correspondent pas exactement aux mots utilisés.r.

**Interface utilisateur** : L'interface constitue le point de contact avec l'utilisateur : messagerie, site web, mobile ou assistant vocal. Son ergonomie et sa fluidité sont déterminantes pour l'expérience utilisateur.

L'intégration de ces composants permet aux chatbots modernes de dépasser les simples réponses automatisées et de devenir de véritables assistants intelligents, capables d'interagir contextuellement et de s'adapter aux besoins des utilisateurs et des recruteurs.

### 2.6.3 Limites et défis

Les chatbots sont des outils stratégiques pour automatiser la communication en recrutement, mais ils présentent encore plusieurs limites [10] :

- **Fiabilité et cohérence des réponses**
  Même les modèles avancés peuvent fournir des réponses inexactes ou incohérentes, surtout face à des questions complexes ou inédites, nécessitant une supervision humaine pour garantir la précision.

- **Biais et discrimination**
  Les données d'entraînement peuvent contenir des biais implicites (genre, origine, expérience) que le chatbot risque de reproduire, impactant l'équité des processus de sélection. Le dé-biaisage et la gestion éthique des données sont essentiels.

- **Qualité, disponibilité et intégrité des données**
  Le fonctionnement du chatbot dépend de données fiables, à jour et structurées. Des informations obsolètes ou incohérentes réduisent la pertinence des réponses et affectent l'expérience utilisateur.

En conclusion, les chatbots offrent des avantages significatifs : automatisation des tâches répétitives, amélioration de l'expérience candidat, soutien aux recruteurs dans la pré-sélection et le suivi des candidatures. Cependant, leur efficacité repose sur la qualité des données, la robustesse technique, la gestion des biais et la capacité à instaurer la confiance auprès des

utilisateurs. Ces limites constituent autant de pistes d'amélioration pour les générations futures de systèmes conversationnels, et elles doivent être soigneusement prises en compte lors de la conception, du déploiement et de l'évaluation de ces outils dans le domaine du recrutement.

# Conclusion

Ce chapitre a présenté l'état de l'art de l'intelligence artificielle appliquée au recrutement, en soulignant son rôle croissant dans la transformation du processus de sélection. Nous avons exploré les techniques de traitement du langage naturel, les modèles de langage avancés ainsi que les chatbots et systèmes conversationnels. Les méthodes de resume parsing et de resume matching, qu'elles soient classiques ou basées sur les Transformers et LLM, ont également été abordées. Le chapitre suivant présentera les modèles et méthodes que nous avons choisis et implémentés pour ce projet.

# Chapitre 3

# Analyse des Données des CV et des Offres d'emploi

## Introduction

Après avoir défini le cadre général de notre projet et présenté l'état de l'art, nous entamons dans ce chapitre la phase d'analyse des données des CV et des offres d'emploi . Nous commençons par présenter les données sources utilisées, à savoir les caractéristiques des CV et des offres d'emploi. Nous exposerons ensuite en détail les deux approches techniques développées pour l'extraction des données : une première basée sur le NLP classique et une seconde exploitant les grands modèles de langage (LLM). Enfin, nous passerons à l'analyse comparative des performances, en présentant les résultats obtenus par chaque méthode afin de valider la solution finale retenue.

### Approche avec NLP Classique

Le parsing a été conçu pour répondre à deux objectifs principaux :

— Structurer les informations clés des candidats et les informations des offres afin de faciliter leur correspondance ultérieure à partir de documents bruts.
— Alimenter automatiquement l'interface profil candidat de la plateforme Talinty, afin de faciliter la gestion et la visualisation des données par les recruteurs.

Le pipeline du parsing a été conçu autour de plusieurs étapes illustrées dans la figure 5.9.

Une partie des travaux présentés dans l'état de l'art pour le module de parsing reposent sur le NLP. Nous avons d'abord opté pour l'implémentation de cette approche. Toutefois, après évaluation, les résultats se sont révélés insuffisants et ont montré plusieurs limites ( cf. Annexe). Pour tous les détails relatifs à cette approche classique, nous nous référer à l'annexe. Afin de surmonter ces insuffisances, nous proposons une approche plus moderne.

# 3.1 Approche d'analyse basée sur LLM et LangChain

Cette section présente une alternative à notre première approche, en exploitant la puissance des grands modèles de langage (LLM) et le framework LangChain. Elle détaille l'intégration de ces technologies pour surmonter les limitations initiales et améliorer la robustesse du système d'extraction

## 3.1.1 Objectif et motivation

Afin d'analyser les données de CV et des offres d'emploi, les travaux precedents reposent principalement sur des techniques classiques telles que les expressions régulières (Regex) et les modèles de Reconnaissance d'Entités Nommées (NER), a rapidement montré ses limites. Ces approches, bien qu'efficaces pour des informations simples comme les adresses e-mail ou les numéros de téléphone, se révèlent insuffisantes lorsqu'il s'agit de traiter la richesse et la complexité d'un CV ou d'une offre d'emploi.

En effet, un CV n'est pas un simple réservoir de mots-clés : il s'agit d'un document narratif, contextuel et hiérarchisé, où chaque information prend son sens dans une logique temporelle (dates d'expériences, parcours académique), organisationnelle (entreprises, écoles) et sémantique (missions, compétences acquises, résultats obtenus). Cependant, les méthodes traditionnelles échouent dès qu'elles doivent :

- distinguer une compétence technique noyée dans la description d'une mission,
- reconstruire la structure complète d'une expérience professionnelle (poste, entreprise, période, description),
- ou séparer correctement une formation académique d'un projet réalisé en entreprise.

En effet , nous avons appliqué (cf. chapitre 2, section 2) et nous avons trouvé des resultats

d'evaluation non satisfaisantes (cf. Annexe)

Pour franchir ce cap, nous avons proposé une approche basée sur les modèles de langage de grande taille (**LLM**). Ces modèles offrent une capacité bien supérieure de compréhension sémantique et de raisonnement contextuel, rendant possible une extraction beaucoup plus fine et fidèle à la réalité du profil candidat ou de l'exigence du recruteur.

L'objectif n'était plus simplement de repérer des mots-clés, mais d'interpréter leur rôle dans le contexte du document :

- comprendre qu'un mot comme *Python* désigne une compétence technique,
- identifier qu'*École Polytechnique* correspond à un établissement d'enseignement,
- ou encore reconnaître que *stage de fin d'études* doit être classé comme une expérience professionnelle particulière.

Pour répondre à ces enjeux, le choix s'est porté sur **LangChain**, une infrastructure permettant d'orchestrer l'utilisation des LLM au sein d'un pipeline structuré. Cette technologie apporte plusieurs avantages déterminants [74] :

- **Connexion multi-modèles** : LangChain permet d'expérimenter et de basculer entre différents LLM (Gemini, LLaMA, etc.) sans réécrire l'architecture sous-jacente.
- **Structuration des sorties** : grâce à ses parseurs JSON et ses intégrations avec Pydantic, il est possible de forcer le LLM à produire des réponses normalisées et directement exploitables dans nos bases de données.
- **Interopérabilité** : l'approche s'intègre naturellement dans un environnement applicatif, avec exposition des résultats sous forme d'API consommables par la plateforme Talinty.

Par conséquent, cette deuxième approche se fixe comme objectif de transformer le parsing de CV et d'offres d'emploi en un processus intelligent, capable non seulement d'extraire des données, mais aussi de les interpréter et de les structurer de manière exploitable pour le moteur de matching.

### 3.1.2 Pipeline de traitement

La mise en œuvre de la deuxième version du système a nécessité la conception d'un pipeline de traitement modulaire et robuste, permettant d'assurer la transition fluide entre l'extraction brute de données et leur mise à disposition sous une forme exploitable par la plateforme Talinty.

Le pipeline du parsing avec les LLMs et LangChain a été conçu autour de plusieurs étapes illustrées dans la figure 3.1.




FIGURE 3.1 – Pipeline du Parsing basé sur LLM et LangChain

**1. Lecture du PDF**

— La première étape consiste à extraire le contenu textuel des documents PDF (CV ou offres d'emploi).

— La meme etape conçue durant la pemiere approche mais avec une nouvelle bibliothèque **fitz** qui offre une bonne couverture en termes de performance et de compatibilité avec différents formats de CV.

**2. Construction du Prompt Dynamique**

— Une fois le texte extrait, il est préparé pour le modèle de langage. Le texte est d'abord prétraité et nettoyé pour supprimer les caractères spéciaux, uniformiser les formats (dates, numéros de téléphone, etc.), et segmenter le document en sections logiques. Ce texte nettoyé est ensuite intégré dans un prompt dynamique qui sera soumis au modèle pour l'extraction des données.

**3. Appel au LLM via LangChain**

— Une fois le texte normalisé, il est transmis au modèle de langage de grande taille sélectionné.

— L'intégration se fait à travers LangChain, qui agit comme une couche d'orchestration. Cette architecture présente plusieurs avantages :

  — encapsulation de l'appel au modèle,

  — gestion des prompts complexes et contextualisés,

  — compatibilité future avec d'autres LLM (Gemini, GPT-4.1, Mistral, etc.).

— L'objectif est que le modèle ne se limite pas à identifier des mots-clés, mais qu'il comprenne la sémantique et la structure des entités présentes dans le document.

**4. Parsing structuré en JSON**

— Les sorties générées par le modèle sont immédiatement converties en JSON valide, grâce au `JsonOutputParser` de LangChain, combiné avec des schémas Pydantic prédéfinis.

— Deux structures principales sont utilisées :

  — `ResumeData` : qui encapsule toutes les informations extraites d'un CV (expériences, compétences, formations, certifications, langues, etc.),

  — `JobDescription` : qui organise les informations extraites d'une offre d'emploi (compétences requises, expériences attendues, soft skills, diplômes).

— Cette étape assure que les données ne soient pas simplement des blocs textuels, mais des objets structurés et exploitables dans la suite du pipeline.

**5. Génération d'un identifiant unique**

— Pour chaque document traité, un UUID (Universally Unique Identifier) est généré.

— Ce mécanisme garantit la traçabilité et facilite le suivi des documents dans la base de données, tout en permettant d'associer facilement un CV structuré à un profil candidat unique.

## 3.2 Données Initiales

Le moteur de parsing s'appuie sur deux types de données principales : les CV des candidats et les offres d'emploi. Ces documents contiennent les informations essentielles pour extraire les compétences, expériences et exigences de poste. Comprendre leur diversité est crucial pour concevoir un parsing efficace et préparer le traitement automatisé des informations.

### 3.2.1 Curriculum Vitae (CV) :

Le Curriculum Vitae, ou résumé, constitue la principale source de données pour le parsing, car il rassemble toutes les informations essentielles sur le parcours d'un candidat : expériences professionnelles, formations, compétences et certifications. Dans ce chapitre, le terme "Resume" sera utilisé de manière interchangeable avec CV

Les CV sont toujours soumis au format PDF, choix imposé par l'entreprise pour assurer une stabilité de mise en page et une compatibilité universelle. Cela garantit une lisibilité uniforme et préserve les éléments graphiques (logos, icônes, couleurs), ce qui facilite le travail des recruteurs.

**Exemples de CV**

**Example 1 :** la figure 3.2 represente l'exemple d'un CV

**Format :** PDF avec mise en page sobre, une seule colonne, titres de sections mis en évidence.

**Caractéristiques :**

**Langue unique** (anglais), ce qui réduit la complexité de détection linguistique.
**Sections bien délimitées** Professional Summary, Skills, Experience, Education, Projects.
**Organisation chronologique claire** dans la partie Experience, avec dates alignées à gauche.



Alfred Pennyworth

*Product Manager*

*Silicon Valley, CA, USA*
*(123) 456-7890*
*alfred.pennyworth@email.com*
*apennyworth*
*apennyworth*

**Professional Summary**

Seasoned Product Manager with over 20 years of experience in software development and product management, having worked at all FAANG companies. Exceptional leadership skills, strategic thinking, and a track record of managing products from conception to market success.

**Skills**

Product management, Agile methodologies, Leadership, Communication, Project management, User Experience Design, Market Research, Data Analysis, Java, Python, JavaScript, HTML/CSS, SQL, AWS

**Experience**

2017 – Present **Product Manager**, *Google*, Mountain View, CA, USA
Leading cross-functional teams to design, develop, and launch innovative products. Developing product strategies and making data-driven decisions to improve user experience and meet business goals.

2012 – 2017 **Software Development Engineer III**, *Amazon*, Seattle, WA, USA
Led a team of developers in building scalable and high-performing e-commerce applications. Successfully delivered multiple projects within the stipulated time and budget.

2007 – 2012 **Software Development Engineer II**, *Apple*, Cupertino, CA, USA
Designed and implemented software components for various Apple services. Optimized the performance of applications and improved code quality through thorough testing.

2002 – 2007 **Software Development Engineer I**, *Netflix*, Los Gatos, CA, USA
Developed and maintained the user interface for the Netflix web application. Worked closely with product managers and designers to create an optimal user experience.

1999 – 2002 **Software Development Engineer I**, *Facebook*, Menlo Park, CA, USA
Played a key role in the development of early Facebook features. Implemented scalable back-end services using Java and SQL.

**Education**

2016 – 2018 **Master of Business Administration**, *Stanford University*, Stanford, CA, USA

1997 – 1999 **Master of Science in Computer Science**, *Massachusetts Institute of Technology*, Cambridge, MA, USA

1994 – 1997 **Bachelor of Science in Computer Science**, *University of California, Berkeley*, Berkeley, CA, USA

FIGURE 3.2 – Exemple d'un CV

**Pas de bruit visuel** très peu d'icônes, pas de colonnes multiples, peu de couleurs → facilite l'extraction automatique.

**Intitulés homogènes** les rubriques correspondent à des standards très fréquents dans les CV anglo-saxons.

### Caractéristiques principales des CV

Bien que tous les CV de ce projet soient fournis au format PDF et rédigés en anglais, leur forte hétérogénéité constitue un défi majeur pour le parsing. Cette variabilité se manifeste à plusieurs niveaux :

— **Longueur** : certains CV sont très concis (1 page), tandis que d'autres s'étendent sur 2 à 3 pages. Plus un CV est long, plus les informations sont dispersées et difficiles à relier.

— **Organisation** : les structures varient entre CV chronologiques, thématiques ou hybrides, obligeant le parser à identifier la logique de présentation.

— **Rubriques** : les intitulés diffèrent selon les candidats (*Work Experience, Professional Experience, Employment History* ; *Skills, Core Competencies, Technical Expertise*), ce

qui nécessite des dictionnaires de synonymes ou des règles souples.

— **Éléments spécifiques** : certains incluent une photo, des références ou des projets personnels, et les coordonnées ne sont pas toujours placées au même endroit.

— **Aspect visuel** : l'usage de colonnes, tableaux, logos ou couleurs facilite la lecture humaine, mais complique l'analyse automatique.

— **Variabilité linguistique et terminologique** : au-delà de la structure, les candidats emploient des formulations très diverses pour exprimer une même compétence (*Python Programming* vs *Proficient in Python*), parfois de manière implicite (*Led a team of 5 developers* → leadership). De plus, les CV contiennent souvent des abréviations ou acronymes (ex. *NLP, CI/CD*), qui nécessitent des techniques de normalisation lexicale et des bases de connaissances spécifiques.

Ainsi, la diversité de structures, de terminologies et de représentations graphiques rend l'extraction automatique complexe. Le parser doit donc être capable de gérer cette variabilité et de distinguer les informations essentielles du bruit visuel.

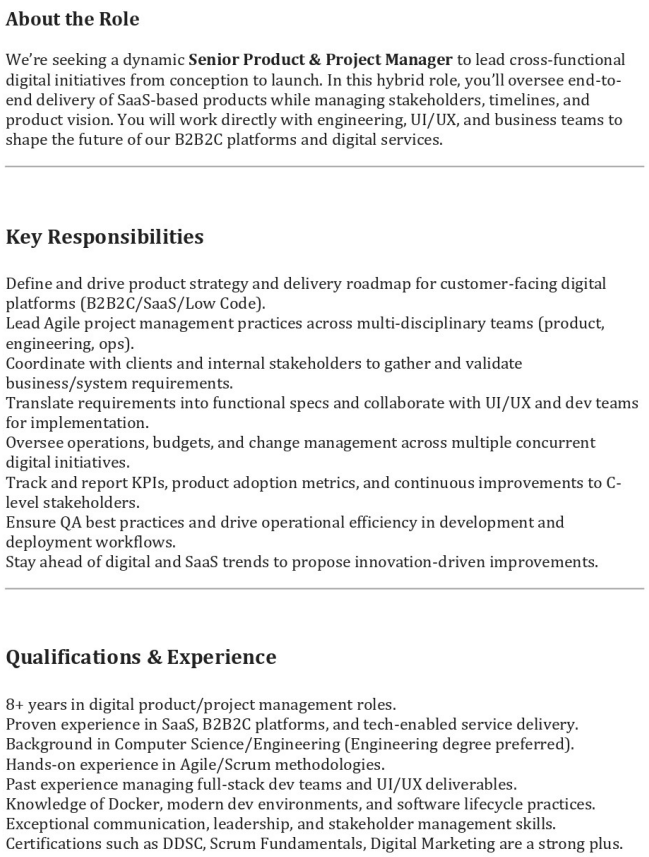
### 3.2.2 Offre d'emploi

L'offre d'emploi, ou Job Description (JD), est le second document clé du processus de parsing. Elle constitue la cible à laquelle le CV du candidat est comparé, car elle détaille les besoins, les exigences et les responsabilités d'un poste spécifique. Si le CV représente "l'offre" de compétences d'un candidat, l'offre d'emploi représente la "demande" de l'entreprise.

Dans cette sous section, nous présentons le format de JD et ses caracteristiques.la figure 3.3 represente un exemple d'un Job Description

Tout comme les CVs, les offres de travail traitées dans ce projet sont principalement au format PDF. L'utilisation de templates d'entreprise garantit une présentation homogène de leurs offres, intégrant les logos, les couleurs etles mises en page spécifiques.

Cependant, cette standardisation visuelle propre à chaque entreprise crée une hétérogénéité globale qui est un défi pour le parsing. Contrairement à un CV où la structure est centrée sur le candidat, la structure d'une offre est souvent dictée par des impératifs de marketing et de communication. Le parser doit donc être capable de naviguer à travers des mises en page variées (multi-colonnes, encadrés, bannières) pour distinguer les informations essentielles du contenu promotionnel.



**About the Role**

We're seeking a dynamic **Senior Product & Project Manager** to lead cross-functional digital initiatives from conception to launch. In this hybrid role, you'll oversee end-to-end delivery of SaaS-based products while managing stakeholders, timelines, and product vision. You will work directly with engineering, UI/UX, and business teams to shape the future of our B2B2C platforms and digital services.

**Key Responsibilities**

Define and drive product strategy and delivery roadmap for customer-facing digital platforms (B2B2C/SaaS/Low Code).
Lead Agile project management practices across multi-disciplinary teams (product, engineering, ops).
Coordinate with clients and internal stakeholders to gather and validate business/system requirements.
Translate requirements into functional specs and collaborate with UI/UX and dev teams for implementation.
Oversee operations, budgets, and change management across multiple concurrent digital initiatives.
Track and report KPIs, product adoption metrics, and continuous improvements to C-level stakeholders.
Ensure QA best practices and drive operational efficiency in development and deployment workflows.
Stay ahead of digital and SaaS trends to propose innovation-driven improvements.

**Qualifications & Experience**

8+ years in digital product/project management roles.
Proven experience in SaaS, B2B2C platforms, and tech-enabled service delivery.
Background in Computer Science/Engineering (Engineering degree preferred).
Hands-on experience in Agile/Scrum methodologies.
Past experience managing full-stack dev teams and UI/UX deliverables.
Knowledge of Docker, modern dev environments, and software lifecycle practices.
Exceptional communication, leadership, and stakeholder management skills.
Certifications such as DDSC, Scrum Fundamentals, Digital Marketing are a strong plus.

FIGURE 3.3 – Exemple d'une offre d'emploi

### Caractéristiques principales des Offres d'emploi

Bien que leur objectif soit clair, les offres de travail sont hétérogenes.

La mise en page est fortement liée à l'identité visuelle de l'entreprise.

— Mises en page multi-colonnes, avec des informations clés (lieu, type de contrat) dans une barre latérale.
— Utilisation d'icônes et d'éléments graphiques pour lister les avantages ou les compétences requises.
— Présence de bannières en en-tête ou de pieds de page contenant des informations non pertinentes pour le matching.

Cette complexité structurelle exige un parser capable de réaliser une analyse de la mise en page pour délimiter sémantiquement les différentes zones du document.

Similairement aux CVs, les intitulés des sections varient considérablement d'une offre à l'autre, même pour décrire le même type d'information.

— Responsabilités : “Your Role”, “What You'll Do”, “Missions”.
— Compétences requises : “Requirements”, “Qualifications”, “What You Bring”, “Ideal Profile”.
— Compétences souhaitées : “Nice to Have”, “Bonus Points”, “Preferred Qualifications”.

Le système doit s'appuyer sur des dictionnaires de synonymes étendus pour identifier et catégoriser correctement ces sections.

Les offres de travail contiennent une quantité significative de texte non pertinent pour l'analyse des compétences, souvent appelé "bruit".

— Longues descriptions de l'entreprise, de sa culture et de ses valeurs.
— Détails sur les avantages sociaux (assurance, tickets restaurant, etc.).
— Mentions légales et déclarations sur l'égalité des chances.

L'un des principaux défis du parser est de filtrer ce bruit pour se concentrer exclusivement sur les éléments décrivant le poste et le profil recherché.

Ainsi, le parsing d'offres de travail requiert des mécanismes capables de gérer une grande diversité de mises en page, de normaliser une terminologie non standardisée, de comprendre la hiérarchie des exigences et de filtrer un volume important d'informations non pertinentes.

## 3.3 Construction du prompt dynamique et Integration de LangChain

Cette section présente la conception des prompts dynamiques et leur intégration au sein de LangChain.

### 3.3.1 Construction des prompts dynamiques

La qualité d'un système basé sur des LLM ne dépend pas uniquement du choix du modèle, mais aussi de la façon dont on guide le modèle via des *prompts*. Dans notre projet, la **prompt engineering** a joué un rôle central : elle a permis de transformer des instructions génériques en tâches précises, orientées vers l'extraction structurée des données.

Pour assurer la fiabilité du parsing, les prompts doivent être conçus de manière à réduire toute ambiguïté, en imposant une structuration stricte (par exemple au format JSON conforme

aux schémas Pydantic), en contextualisant clairement la tâche autour du traitement de CV ou d'offres d'emploi, et en anticipant les erreurs possibles par des consignes précises afin d'éviter les dérives comme les hallucinations ou les paraphrases inutiles.

**Méthodologie adoptée**

- **Définition des entités clés :** avant de rédiger les prompts, nous avons identifié les informations indispensables (ex. : expériences professionnelles, diplômes, certifications, compétences techniques/soft skills).
- **Élaboration de schémas `Pydantic` :** chaque prompt fait explicitement référence à une structure JSON cible (`ResumeData`, `JobDescription`) qui sert de contrat entre le LLM et notre système.
- **Itérations successives :** plusieurs versions des prompts ont été testées, en modifiant le niveau de détail, les instructions sur les formats de date, ou encore la manière d'exiger des tableaux pour certaines sections (par ex. expériences).
- **Optimisation pour Groq :** comme les modèles sont exécutés via Groq, nous avons ajusté la concision des prompts afin d'améliorer le temps de réponse sans sacrifier la précision.

*Prompt pour parsing de CV (`FULL_RESUME_PROMPT`) :*

```
" You are an expert resume parser. First, clean the resume text by removing
unnecessary whitespace, special characters (e.g., •, -, multiple spaces),
and formatting artifacts (e.g., page numbers, headers/footers, PDF metadata,
watermarks), while preserving the content's meaning, such as names, dates,
and job descriptions.
Then, extract all relevant sections and return a JSON object matching the
following structure:
{{
    "parsed_resume": {{
        "education_section": [{{
            "date": "",
            "degree": "",
            "institution": "",
            "location": ""
        }}],
        "email_section": "",
        "experience_section": [{{
            "date": "",
            "job_info": "",
            "responsibilities": []
        }}],
```

```
        "extracted_keywords": "",
        "location_section": "",
        "summary": "",
        "name_section": {{
            "First Name": "",
            "Last Name": ""
        }},
        "phone_section": "",
        "skills_section": [],
        "social_media_section": {{
            "github": "",
            "linkedin": ""
        }},
        "unique_id": ""
    }}
}}

Ensure extracted_keywords are lowercase and space-separated.
Do not generate the unique_id; it will be added programmatically.
If a section is not found, return empty values (e.g., "", [], or {{}}).
Use the following resume text:

{content}

Return only the JSON output.
"
```

### 3.3.2 Intégration dans LangChain

L'intégration finale de notre pipeline repose sur l'utilisation de **LangChain** comme framework d'orchestration. Celui-ci agit comme une couche intermédiaire entre le modèle de langage et notre logique métier, apportant à la fois une structuration claire et une modularité précieuse.

Concrètement, l'intégration se fait via :

- **Des prompts spécialisés** (`FULL_RESUME_PROMPT`, `FULL_JOB_DESC_PROMPT`) permettant de guider le modèle de manière précise selon le type de document à traiter.
- **Un LLM :** exécuté via l'API Groq pour bénéficier à la fois de la puissance computationnelle et d'une latence réduite par rapport aux serveurs classiques.
- **Un JsonOutputParser** assurant que les réponses du modèle respectent systématiquement les schémas Pydantic prédéfinis (`ResumeData`, `JobDescription`).
- **Deux classes de parsing dédiées :**

- `ParseResume` → conversion d'un CV brut en JSON enrichi, avec structuration hiérarchique des expériences, diplômes et compétences.
- `ParseJobDesc` → conversion d'une offre d'emploi en JSON enrichi, intégrant exigences, missions et soft skills.

— Enfin, l'ensemble est exposé via une **API REST**, ce qui permet une intégration fluide dans la plateforme *Talinty* et prépare déjà le terrain pour un moteur de matching IA plus évolué.

## 3.4 Evaluation de l'approche proposée

Dans cette section, nous présentons la méthodologie d'évaluation adoptée et montrons, à travers des métriques objectives et des résultats comparatifs, la pertinence de l'approche proposée.

### 3.4.1 Métriques d'évaluation

L'évaluation des performances du système ne peut pas se limiter à un simple jugement qualitatif ; elle doit reposer sur des indicateurs objectifs et reproductibles. Deux métriques principales ont été retenues afin de mesurer la qualité des extractions réalisées par les LLM :

**Word Error Rate (WER)**

Le *WER* (taux d'erreur mot-à-mot) mesure les erreurs au niveau des mots (substitutions, suppressions, insertions) par rapport à la vérité de référence. Un WER de 0 indique une transcription parfaite. Cette métrique évalue la fidélité lexicale du modèle (intitulés de postes, établissements, dates) [45].

$$WER = \frac{S + D + I}{N} \tag{3.1}$$

où $S$ = substitutions, $D$ = suppressions, $I$ = insertions, et $N$ = nombre total de mots dans la référence.

*Interprétation :* Un WER de 0 signifie une transcription parfaite. Plus le WER est élevé, plus le texte extrait est éloigné de la vérité. Dans notre contexte, cette métrique permet de mesurer la fidélité lexicale du modèle : le système a-t-il extrait correctement les intitulés de postes, les noms d'établissements, ou les dates, sans erreurs de transcription ou d'orthographe ?

**Character Error Rate (CER)**

Le *cER* (taux d'erreurs par caractere, ou *slot*) calcule les erreurs caractère par caractère. Plus précis que le WER, il permet de détecter les fautes d'orthographe, de ponctuation ou de typographie [62]..

$$CER = \frac{S + D + I}{N_c} \tag{3.2}$$

où $N_c$ est le nombre total de caractères dans la référence, $S$ le nombre de substitutions, $D$ le nombre de suppressions, et $I$ le nombre d'insertions.

L'utilisation conjointe du WER et du CER permet d'obtenir une évaluation équilibrée :

— Le WER juge la proximité textuelle avec la référence.

— Le CER juge la précision au niveau des caractères.

### 3.4.2 Résultats comparatifs & choix final du modèle

Nous avons commencé par l'approche classique basée sur NLP , mais d'aprés les valeurs de F1 score obtenues, elle échoue largement sur les informations déterminantes (compétences, expérience, éducation) Afin de surmonter ces insuffisances, nous avons proposé une approche basé sur les LLM. Pour identifier le modèle le plus adapté aux besoins de la plateforme *Talinty*, trois grands modèles de langage (LLM) ont été évalués sur un même corpus de CV et d'offres d'emploi. L'objectif était de comparer leurs performances selon trois critères principaux :

1. **La précision de l'extraction sémantique** (capacité à comprendre et restituer correctement les entités complexes),
2. **La cohérence des structures produites** (validité JSON, respect du schéma Pydantic),
3. **Le temps de traitement** (indispensable pour un usage temps réel sur la plateforme).

Trois grands modèles de langage (LLM) ont été évalués afin d'identifier le plus adapté aux besoins du projet, en tenant compte de la précision d'extraction, de la cohérence des données structurées et du temps de traitement. Les modèles analysés sont **LLaMA 3.1 8B Instant**, reconnu pour sa rapidité, **Gemini 2.0 Flash**, offrant un bon équilibre entre performance et efficacité, et **LLaMA 3.3 70B Versatile**, plus puissant mais également plus exigeant en ressources.

Afin de mesurer objectivement l'efficacité des différents modèles testés, les métriques WER et CER définies précédemment ont été calculées sur un même corpus de CV et d'offres d'emploi. Les résultats sont présentés dans le tableau 3.1 :

| **Modèle** | **WER ↓** | **CER ↓** | **F1 ↑** | **Observations** |
|---|---|---|---|---|
| LLaMA 3.1 8B Instant | 0.29 | 0.34 | 0.78 | Extraction partielle, pertes fréquentes sur les expériences et diplômes. |
| Gemini 2.0 Flash | 0.18 | 0.22 | 0.86 | Bon compromis, mais encore des confusions entre expériences et projets académiques. |
| LLaMA 3.3 70B Versatile | 0.11 | 0.14 | 0.92 | Meilleure précision, structure très fidèle, capture fine des compétences implicites et des contextes. |

TABLE 3.1 – Résultats comparatifs des modèles évalués sur le corpus de test.

**LLaMA 3.1 8B Instant** : modèle très rapide ( 30 s) mais limité en compréhension contextuelle. Avec un WER de 0.29, un CER de 0.34 et un F1 de 0.78, il présente des erreurs récurrentes sur les expériences et diplômes, ce qui limite fortement son usage en production.

**Gemini 2.0 Flash** : compromis intéressant entre rapidité et précision ( 35 s). Ses scores de WER (0.18), CER (0.22) et F1 (0.86) témoignent d'une performance équilibrée, bien qu'il reste instable sur la distinction entre expériences professionnelles et projets académiques.

**LLaMA 3.3 70B Versatile** : plus lent ( 40 s) mais nettement supérieur. Avec un WER de 0.11, un CER de 0.14 et un F1 de 0.92, il offre une structuration très fidèle des expériences et une détection fine des compétences implicites, en faisant le modèle le plus robuste pour une utilisation en production.

Malgré ses limites (notamment une latence élevée pouvant atteindre 20 secondes par document et des besoins matériels conséquents), **LLaMA 3.3 70B Versatile** s'est imposé comme le meilleur choix. Ce modèle open source, à la différence de solutions propriétaires comme Gemini Pro ou GPT, offre une transparence et une maîtrise totale de son implémentation. Il se distingue par la pertinence de ses résultats, avec une fidélité d'extraction supérieure et une meilleure capture des nuances contextuelles. Sa robustesse garantit des données plus cohérentes et directement exploitables dans les pipelines de matching.

Par conséquent, LLaMA 3.3 70B Versatile constitue un compromis stratégique : c'est le seul modèle capable de garantir une qualité de parsing suffisante pour un usage réel, tout en conservant l'avantage d'un écosystème open source, flexible et économiquement viable.

# 3.5 Résultats du parsing

Afin d'illustrer concrètement le fonctionnement du module de parsing, nous présentons dans cette section quelques captures d'écran.

## 3.5.1 Exemple d'un CV aprés analyse

Les figures suivantes montrent la représentation structurée générée à partir d'un CV. On observe que les informations clés (formations, expériences, compétences, etc.) ont été extraites et converties en un format JSON exploitable.


```
{
    "parsed_resume": {
            "education_section": [
                {
                    "date": "2009 - 2012",
                    "degree": "PhD in Computer Science",
                    "institution": "CalTech",
                    "location": "Pasadena, USA"
                },
                {
                    "date": "2007 - 2009",
                    "degree": "Master of Science in Computer Science",
                    "institution": "MIT",
                    "location": "Cambridge, USA"
                },
                {
                    "date": "2003 - 2007",
                    "degree": "Bachelor of Science in Computer Science",
                    "institution": "UC San Diego",
                    "location": "San Diego, USA"
                }
```


FIGURE 3.4 – Résultat du parsing d'un CV- Partie 1


```
            ],
            "email_section": "bwayne@email.com",
            "experience_section": [
                {
                    "date": "June 2020 - Present",
                    "job_info": "MERN Stack Developer, OpenAI, San Francisco, USA",
                    "responsibilities": [
                        "Working on the development of the ChatGPT project, using Node.js, Express.js, and
                            React.",
                        "Implementing RESTful services for communication between frontend and backend.",
                        "Utilizing Docker and Kubernetes for deployment and management of applications.",
                        "Working in an Agile environment, delivering high-quality software every sprint.",
                        "Contributing to the design and implementation of machine learning algorithms for
                            natural language processing tasks."
                    ]
                },
                {
                    "date": "July 2015 - May 2020",
                    "job_info": "Full Stack Developer, Uber, San Francisco, USA",
                    "responsibilities": [
```


FIGURE 3.5 – Résultat du parsing d'un CV- Partie 2



```
"extracted_keywords": "javascript typescript mongodb express.js react node.js restful apis git
    github docker kubernetes agile scrum python machine learning",
"location_section": "Gotham, USA",
"name_section": {
    "First Name": "Bruce",
    "Last Name": "Wayne"
},
"phone_section": "I (123) 456-7890",
"skills_section": [
    "JavaScript",
    "TypeScript",
    "MongoDB",
    "Express.js",
    "React",
    "Node.js",
    "RESTful APIs",
    "Git",
    "GitHub",
    "Docker",
    "Kubernetes",
    "Agile",
```


FIGURE 3.6 – Résultat du parsing d'un CV - Partie 4



```
    "Git",
    "GitHub",
    "Docker",
    "Kubernetes",
    "Agile",
    "Scrum",
    "Python",
    "Machine Learning"
],
"social_media_section": {
    "github": "",
    "linkedin": ""
},
"summary": "Highly skilled MERN Stack Developer with over 10 years of experience, specializing in
    designing, building, and maintaining complex web applications. Proficient in MongoDB, Express.
    js, React, and Node.js. Currently contributing to the development of AI technologies at OpenAI,
    with a primary focus on the ChatGPT project.",
"unique_id": "f80186fe-7c57-48b9-b474-26d36cad1740"
}

}
```


FIGURE 3.7 – Résultat du parsing d'un CV -Partie 5

### 3.5.2 Profil candidat basé sur les données analysées

Comme indiqué précédemment, le parsing n'est pas seulement utilisé pour le matching, mais aussi pour alimenter directement l'interface *profil candidat* dans la plateforme Talinty. La figure 3.8 illustre la génération automatique d'un profil à partir des données analysées.

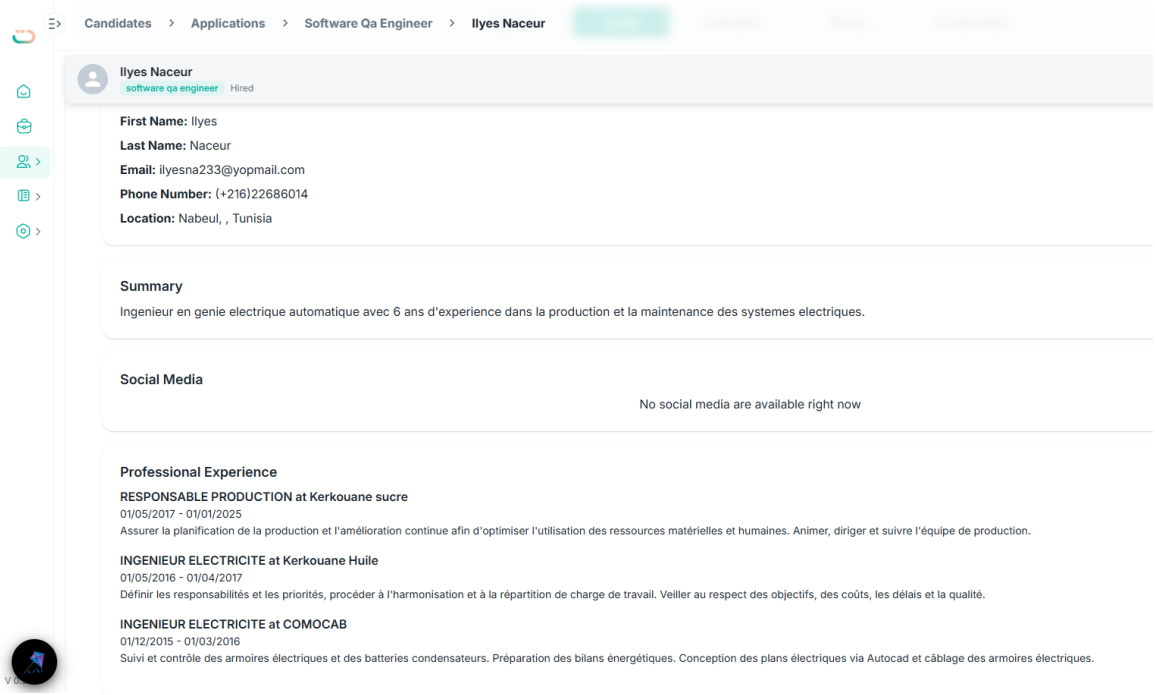



FIGURE 3.8 – Profil candidat alimenté automatiquement grâce au parsing du CV

# Conclusion

Ce chapitre a permis de retracer l'évolution progressive du pipeline de parsing et de matching au sein de la plateforme Talinty. La première version a posé les bases fonctionnelles d'un système automatisé, capable d'extraire et de structurer des informations essentielles à partir des CV et des offres d'emploi. La deuxième version est venue enrichir ce socle en intégrant les Large Language Models, apportant une finesse d'analyse et une compréhension sémantique inédite, au prix toutefois d'une latence plus élevée.

Ainsi, l'expérience menée confirme que l'usage des LLM représente aujourd'hui une avancée incontournable pour améliorer la qualité du matching entre candidats et recruteurs. Elle met également en lumière les arbitrages nécessaires entre performance technique, précision des résultats et scalabilité de la solution. Ces constats ouvrent la voie à de nouvelles optimisations et à l'exploration de modèles encore plus spécialisés dans la version suivante.

# Chapitre 4

# Mise en Correspondance : Le Matching

# Introduction

Après avoir présenté dans le chapitre précédent le Parsing des CV et des offres d'emploi, nous abordons à présent le second module du processus : la mise en correspondance entre CV et offre d'emploi.

Ce chapitre se focalise sur la mise en correspondance entre les informations extraites (compétences, expériences, diplômes, etc.) et les exigences des offres. L'objectif est de mesurer la compatibilité CV - Offre d'emploi , afin d'alimenter la plateforme Talinty avec des candidatures fiables et directement exploitables par les recruteurs.Pour cela, Nous avons conçu deux approches , Une approche basée sur NLP classique et l'autre basée sur les LLM.

Ainsi, le Matching s'inscrit comme la continuité naturelle du parsing : il ne s'agit plus seulement d'organiser les données, mais de leur donner du sens en établissant des correspondances précises et pertinentes.

## 4.1 Apperçu général des approches proposées

Les approches du *Matching* se déclinent principalement en deux catégories. D'un côté, les approches classiques qui reposent sur des règles, des mots-clés et des mesures statistiques telles que le TF-IDF et la similarité cosinus. Elles offrent une mise en correspondance simple mais

restent limitées face aux variations de langage et au manque de contexte. De l'autre coté, les approches modernes fondées sur le NLP avancé, les modèles Transformers et les LLM (ex. BERT, RoBERTa, GPT). Ces modèles génèrent des représentations contextuelles et permettent un appariement plus précis et sémantique entre les CV et les offres d'emploi.

Cette section présente le principe de l'intégration du *Matching* dans *Talinty*, qui vise à accélérer la sélection des candidats en réduisant le temps de lecture manuelle des CV et à améliorer la pertinence des recommandations, en proposant aux entreprises les profils les plus adéquats.

## Principe

Le principe de cette première approche repose sur la représentation vectorielle des entités extraites à l'étape de parsing (compétences, expériences professionnelles, diplômes, missions, etc.). Chaque entité textuelle est convertie en un *embedding*, c'est-à-dire un vecteur numérique de dimension fixe qui encode sa signification sémantique.

— un CV n'est plus vu comme un texte brut, mais comme un ensemble de vecteurs décrivant ses différentes composantes,
— une offre d'emploi est encodée de la même manière, sous la forme d'un profil vectoriel de poste.

L'idée de cette approche est simple mais puissante : au lieu de comparer les documents avec des mots exacts (*matching lexical*), on les projette dans un espace vectoriel où des phrases proches en sens sont représentées par des vecteurs voisins.

Par exemple, un candidat qui écrit :

« software engineer with expertise in Python and cloud computing »

sera rapproché d'une offre mentionnant :

« backend developer experienced in Python and AWS »

même si les termes employés ne sont pas identiques.

# 4.2 Premiére Approche de matching basée sur NLP classique

Dans cette section , nous allons discuter la première approche mise en place en s'appuyant sur des méthodes classiques de NLP

## 4.2.1 Présentation du pipeline

La mise en place du moteur de Resume Matching basé sur les embeddings s'est traduite par la construction d'un pipeline clair et structuré. Celui-ci repose sur quatre étapes principales illustrées dans la figure 4.1 :

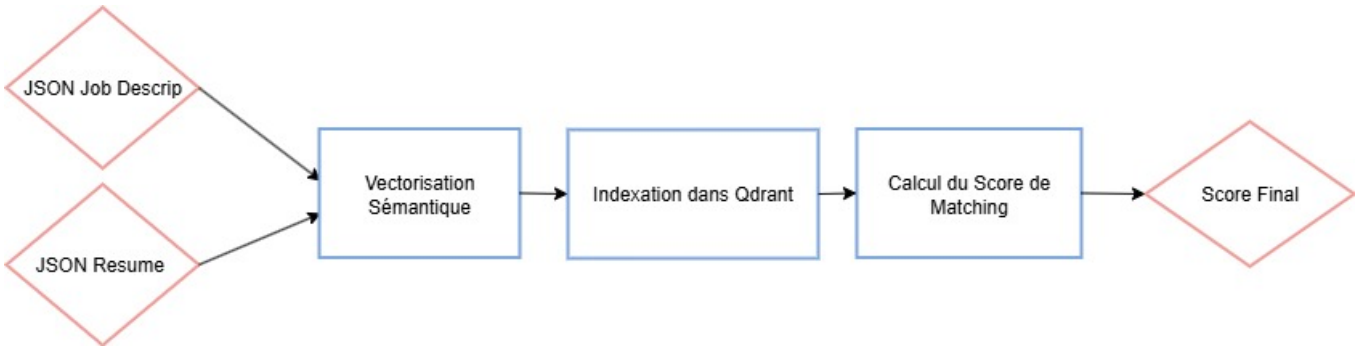



FIGURE 4.1 – Pipeline avec LLM du Matching

Ce pipeline prend comme inputs les deux documents Json du CV et de l'offre d'emploi comme présentés dans le chiptre précedent

**1. Vectorisation Semantique**

— Chaque élément textuel (par exemple une compétence comme Python, ou une mission comme développement d'API REST) est converti en un vecteur numérique grâce à un modèle d'embeddings.

— Plusieurs modèles ont été testés (MiniLM, MPNet, BGE) afin de trouver le meilleur compromis entre rapidité et précision pour notre cas d'usage [55].

— Le résultat est une représentation dense qui capture le sens sémantique des informations plutôt que leur simple forme lexicale.

**2. Indexation dans Qdrant**

— Les vecteurs obtenus sont stockés dans une base Qdrant, qui joue le rôle d'index spécialisé.
— Cette indexation permet de retrouver efficacement les vecteurs les plus proches, même dans un volume important de données.
— Qdrant offre également des options d'organisation (collections, filtres) qui nous ont permis de séparer les CV et les offres et de faciliter les recherches ciblées.

**3. Calcul du score de matching**

— Lorsqu'un CV doit être évalué par rapport à une offre, Qdrant renvoie les vecteurs les plus proches de la requête [55].
— Une similarité cosinus est ensuite calculée entre les embeddings du CV et ceux de l'offre, ce qui donne un score numérique de compatibilité.
— Ce score constitue la base du moteur de recommandation : plus il est élevé, plus le CV est jugé pertinent par rapport à l'offre.

⇒ Ce pipeline met en place une chaîne complète allant de l'extraction des informations textuelles à la génération d'un score de correspondance, en exploitant les capacités des modèles d'embeddings et d'une base vectorielle spécialisée.

Pour encoder les textes en vecteurs, plusieurs modèles pré-entraînés issus de la famille des *Sentence Transformers* et des modèles d'embeddings avancés ont été testés et comparés.

Nous avons evalué les modeles suivnats :

— **all-MiniLM-L6-v2**
— **all-mpnet-base-v2**
— **BAAI/bge-large-en**

Le choix de ces modèles s'est appuyé sur deux critères principaux :

1. Une adoption large dans la communauté NLP, ce qui garantit leur robustesse et leur fiabilité.
2. Une compatibilité directe avec la recherche sémantique et les bases vectorielles, facilitant l'intégration dans notre pipeline.

En pratique, ces différents modèles ont servi de base comparative pour déterminer lequel offrait le meilleur équilibre entre précision des résultats et efficacité opérationnelle dans le contexte de la plateforme Talinty.

### Indexation dans Qdrant

Dans notre projet, une fois les CV et les offres convertis en embeddings, nous devions disposer d'un mécanisme capable de stocker ces vecteurs et surtout de retrouver rapidement les plus proches lors d'une comparaison.

Dans Talinty, le volume de documents est significatif : chaque CV contient plusieurs dizaines d'éléments (compétences, expériences, diplômes) et chaque offre d'emploi est décrite par plusieurs critères. Si l'on devait comparer chaque nouvel embedding à l'ensemble des autres de manière brute, les temps de calcul deviendraient rapidement trop élevés.

Nous avons donc eu besoin d'un outil spécialisé qui puisse :

— accélérer la recherche de correspondances entre un CV et une offre,
— assurer la pertinence des résultats en exploitant la similarité vectorielle plutôt qu'une recherche par mots-clés,
— s'adapter à la montée en charge prévue de la plateforme, où des milliers de CV et d'offres devront être gérés simultanément.

En d'autres termes, l'introduction d'une base vectorielle n'était pas un choix théorique, mais une nécessité pratique pour que le moteur de matching de Talinty reste utilisable, fluide et pertinent.

C'est dans ce contexte que s'est posé le choix concret de la technologie, et que nous avons opté pour **Qdrant**.

**Qdrant**, une base vectorielle open-source conçue spécifiquement pour la recherche sémantique [66] . Plusieurs facteurs ont motivé ce choix :

— **Efficacité et scalabilité** : grâce à son indexation optimisée (notamment l'algorithme HNSW), Qdrant assure des recherches rapides et performantes, même sur de larges volumes de CV, répondant ainsi à l'exigence de Talinty de fournir des recommandations quasi en temps réel.
— **Recherche sémantique native** : l'intégration directe de métriques adaptées, comme la *cosine similarity*, permet un matching précis entre CV et offres, sans développements supplémentaires complexes.
— **Flexibilité et intégration** : son API REST, ses bibliothèques clientes (Python, JavaScript, etc.) et sa compatibilité avec les frameworks NLP modernes facilitent son adoption et son intégration fluide dans le microservice de matching.
— **Prototypage rapide** : la simplicité de déploiement (Docker, cloud ou local) a permis de valider rapidement l'approche par embeddings avant d'explorer des solutions plus avancées

avec les LLM.

— **Robustesse et communauté active** : bénéficiant d'une documentation claire et d'un fort soutien communautaire, Qdrant offre une solution fiable et pérenne pour accompagner l'évolution de la plateforme Talinty.

Le choix de Qdrant s'est imposé comme le meilleur compromis entre performance, flexibilité et rapidité de mise en œuvre. Il nous a permis d'assurer un premier niveau de matching robuste et opérationnel, tout en laissant la porte ouverte à une évolution future vers des approches plus avancées basées sur des LLM.

### 4.2.2 Comparaison des modèles d'embeddings sur le cas d'usage CV - Offre de travail

Afin de déterminer quel modèle était le plus adapté à notre premier approche, nous avons évalué trois modèles d'embeddings sur un corpus interne de CV annotés et d'offres d'emploi. Les métriques retenues mesuraient à la fois la qualité du *matching* et la faisabilité opérationnelle comme le montre le tableau :

— **Cosine Similarity moyenne** (plus élevée = meilleur rapprochement entre CV et offres pertinentes)

— **MRR (Mean Reciprocal Rank) sur top-5** (plus élevé = meilleur classement des offres pertinentes)

— **Temps d'exécution(s/doc)** : temps moyen d'encodage et recherche dans Qdrant

— **F1 Score**

| Modèle | Similarité | MRR@5 | F1-score | Temps (s/doc) |
|---|---|---|---|---|
| MiniLM-L6-v2 | 0.62 | 0.48 | 0.44 | 0.12 |
| MPNet-base-v2 | 0.71 | 0.61 | 0.67 | 0.28 |
| BGE-large-en | 0.79 | 0.69 | 0.74 | 0.30 |

TABLE 4.1 – Comparaison des modèles d'embedding

Nous remarquons que le modéle **MiniLM-L6-v2** offre une faible consommation mémoire et une exécution rapide, mais ses performances en similarité et F1-score restent limitées, le rendant peu fiable pour un moteur de matching précis.
Le modèle **BGE-large-en** atteint les meilleurs scores de similarité et F1, mais sa consommation mémoire élevée le rend moins adapté à un service en temps réel.
Le modèle **MPNet-base-v2** constitue le meilleur compromis, combinant des performances satisfaisantes en similarité et F1-score avec une utilisation mémoire modérée, ce qui en fait le

modèle retenu comme référence pour cette première approche embeddings indexés dans Qdrant, garantissant à la fois qualité d'appariement et efficacité opérationnelle .
Pour cette raison, nous avons retenu **all-mpnet-base-v2** comme modèle de référence pour cette première approche qui repose pricipalement sur les modéles d'embeddings et sur Qdrant comme base de données.

Malgré sa simplicité, cette première approche de matching présente plusieurs limites dans Talinty :

— **Perte d'information contextuelle :** la vectorisation réduit la richesse des CV et offres, en négligeant des détails comme la durée, le niveau de responsabilité ou la progression de carrière.
— **Compétences implicites ignorées :** certaines compétences non explicitées (ex. Python pour un projet de machine learning) ne sont pas détectées, ce qui peut sous-estimer un profil.
— **Dépendance aux mots-clés explicites :** le matching basé sur les termes exacts ne prend pas en compte synonymes et variations, ce qui peut biaiser l'évaluation d'un candidat.

## 4.3 Deuxième approche de matching basée sur les modèles LLM

Face aux limites de l'approche par embeddings et recherche vectorielle, une nouvelle stratégie a été mise en place : l'utilisation de modèles de langage de grande taille (*Large Language Models – LLM*). Cette approche vise à dépasser la simple similarité numérique et à introduire une compréhension contextuelle et sémantique plus riche dans le processus de mise en correspondance **CV - Offre de travail**.

### 4.3.1 Pipeline de traitement

Les outputs de ce pipeline sont les CV et les offres d'emploi , après parsing, sont convertis en `JSON` structuré.

#### 1-Construction du prompt de matching

— Un prompt dynamique est généré en intégrant le contenu `JSON` du CV et de l'offre de travail.
— Ce prompt guide le modèle pour comparer les deux documents sur des dimensions précises (*compétences, expériences, diplômes*).
— Le recours au *prompt engineering* permet de cadrer l'analyse et d'obtenir des résultats exploitables.

**2-Appel au LLM**

— Le modèle est sollicité pour analyser le CV et l'offre d'emploi dans leur ensemble.
— Contrairement aux embeddings, il est capable de prendre en compte des nuances contextuelles, des relations implicites et des formulations variées.
— Cette étape apporte une valeur ajoutée en permettant une comparaison plus fine et plus proche d'une analyse humaine.

**3-Sortie enrichie**

— Le modèle retourne une réponse au format JSON contenant un score global de matching représentant le degré d'adéquation entre le CV et l'offre

### 4.3.2 Construction du prompt de matching

L'efficacité d'un moteur de *Matching* basé sur un LLM dépend en grande partie de la qualité du prompt utilisé. Dans notre cas, l'objectif était de guider le modèle vers une évaluation claire et mesurable de l'adéquation entre un CV et une offre d'emploi, en évitant toute ambiguïté dans la sortie.

La conception du prompt prend en considération les principes suivnats :

**- Mise en contexte du rôle attendu :** le modèle est invité à agir comme un *Applicant Tracking System (ATS)* expert dans le domaine des technologies et de l'informatique. Ce cadrage contextuel permet d'orienter ses réponses vers une logique de recrutement et d'évaluation de profils techniques.

**- Précision sur les critères d'évaluation :** le prompt explicite les éléments à prendre en compte :

— compétences techniques et comportementales,

— maîtrise des outils et technologies,
— expériences professionnelles,
— pertinence des mots-clés par rapport à l'offre.

Cette étape assure que le modèle adopte une analyse multi-dimensionnelle sans se limiter à une simple correspondance lexicale.

**- Format de sortie strictement défini :** pour garantir l'intégrabilité dans le pipeline, le modèle est contraint à retourner uniquement un score numérique (0–100), sans texte additionnel ni explication. Ce choix assure la robustesse et la simplicité d'exploitation du résultat par l'API.

### Exemple de prompt utilisé

```
Hey, act like a highly experienced and skilled Applicant Tracking System (ATS)
with deep knowledge of the tech and IT fields, including software engineering,
data science, data analysis, and big data engineering roles.

Your task is to analyze how well this resume json matches the given job
description json. The job market is extremely competitive, so be precise and
thorough. Use your expertise in evaluating both technical and soft skills,
tool proficiency, experience, and keyword relevance.

Return ONLY a numerical percentage (0-100) with no additional text or symbols.

Resume content:
{resume_json}

Job Description:
{job_desc_json}
```

L'utilisation d'un LLM pour le matching a présenté plusieurs défis. La taille importante de certains CV ou offres entraînait parfois un dépassement de la limite de contexte du modèle, nécessitant de tronquer ou de résumer certaines parties, au risque de perdre des informations pertinentes. Des phénomènes d'hallucinations ont également été observés, le modèle ajoutant ou interprétant des compétences absentes du texte, ce qui pouvait biaiser le score final. Par ailleurs, le respect du format attendu posait problème, le modèle générant parfois du texte explicatif en plus du score numérique requis, perturbant ainsi le pipeline automatisé. Pour pallier ces difficultés, il a été nécessaire d'ajuster la conception des prompts et de mettre en place des garde-fous techniques afin de garantir la fiabilité des résultats.

# 4.4 Evaluation

Dans cette section, nous présentons l'évaluation de notre approche de matching en détaillant les modèles testés, les métriques retenues et les résultats obtenus.

## 4.4.1 Modèles testés

Lors de l'intégration des modèles de langage pour le matching, plusieurs alternatives ont été explorées.

Nous avons d'abord testé **Gemini 2.5 Flash**, qui offrait de bonnes performances de raisonnement et de compréhension contextuelle, mais dont l'intégration technique et la gestion des coûts ne correspondaient pas entièrement aux contraintes de notre projet.Nous avons ensuite évalué différents modèles de la famille **LLaMA 3.3 70B Versatile**, connus pour leur pertinence en tâches de NLP avancées et leur accessibilité en open source.

## 4.4.2 Métriques adaptées

Dans le cadre de l'évaluation de notre approche de Matching, nous disposons de notre jeu de données annoté indiquant le degré de compatibilité entre un CV et une offre d'emploi. L'objectif de l'évaluation est donc de mesurer dans quelle mesure notre approche est capable de reproduire fidèlement ces scores réels. Pour cela, nous avons retenu trois métriques principales :

**1. MAE (Mean Absolute Error)** Cette métrique mesure l'erreur absolue moyenne entre le score réel et le score prédit [22].

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

Plus la MAE est faible, meilleure est la précision globale du modèle.

*Exemple* : score réel = 80, score prédit = 72 $\Rightarrow$ erreur = 8.

**2. RMSE (Root Mean Squared Error)** Comme la MAE, mais cette fois les grandes erreurs sont davantage pénalisées [22].

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

Cette métrique est utile lorsque les erreurs importantes doivent être fortement sanctionnées.

### 4.4.3 Évaluation des modèles testés

Afin d'évaluer la performance des modèles, nous avons appliqué Gemini et LLaMA sur un jeu de dix CV. Les résultats détaillés pour chaque CV sont présentés dans le tableau 4.2, tandis que le tableau 4.3 en résume les moyennes globales.

Prenons par exemple le **CV1** : le modèle **Gemini** obtient un MAE de 6.4 et un RMSE de 8.3, tandis que **LLaMA** obtient respectivement 4.9 et 6.7. Ces valeurs correspondent à l'écart moyen et à l'erreur quadratique moyenne entre les prédictions du modèle et les valeurs de référence (scores attendus pour ce CV).

| **CV** | **Gemini** | | **LLaMA** | |
|---|---|---|---|---|
| | **MAE** | **RMSE** | **MAE** | **RMSE** |
| CV1 | 6.4 | 8.3 | 4.9 | 6.7 |
| CV2 | 6.1 | 8.0 | 4.7 | 6.5 |
| CV3 | 6.3 | 8.2 | 4.8 | 6.6 |
| CV4 | 6.0 | 7.9 | 4.6 | 6.4 |
| CV5 | 6.2 | 8.1 | 4.7 | 6.5 |
| CV6 | 6.5 | 8.4 | 4.9 | 6.7 |
| CV7 | 6.1 | 8.0 | 4.8 | 6.6 |
| CV8 | 6.3 | 8.2 | 4.7 | 6.5 |
| CV9 | 6.4 | 8.3 | 4.8 | 6.6 |
| CV10 | 6.2 | 8.1 | 4.7 | 6.5 |

TABLE 4.2 – Résultats détaillés des modèles Gemini et LLaMA sur 10 CV

| **Modèle LLM testé** | **MAE moyen** | **RMSE moyen** |
|---|---|---|
| Gemini 2.0 Flash | 6.3 | 8.1 |
| LLaMA 3.3 70B | 4.8 | 6.6 |

TABLE 4.3 – Comparaison des performances moyennes des modèles LLM testés

**Analyse :** Gemini affiche des résultats honorables, mais LLaMA est systématiquement meilleur,

avec un MAE et un RMSE plus faibles. Le modèle **LLaMA** est donc retenu comme le plus performant.

## 4.5 Intégration du module de matching dans Talinty

Nous avons opté pour la deuxieme approche Une fois le moteur de matching opérationnel, l'étape cruciale consistait à le rendre exploitable dans un contexte production-ready, directement au sein de la plateforme **Talinty**. L'objectif était de transformer un module expérimental en un service interopérable, accessible par les autres briques logicielles (front-end, moteur de recherche, chatbot de recrutement, etc.).

### 4.5.1 Architecture de l'API

Pour cette intégration, nous avons choisi de mettre en place une **API REST** basée sur Flask. Cette API constitue une couche intermédiaire entre le moteur IA (basé sur le LLM et le pipeline de prompts) et la plateforme Talinty.

Elle permet de :

— recevoir des requêtes externes contenant un CV et une offre d'emploi,
— orchestrer le pipeline de matching (préparation des données, appel au modèle, génération du score),
— retourner un résultat JSON clair et normalisé, facilement exploitable par les différents services de la plateforme.

### 4.5.2 Fonctionnalités principales exposées

Les endpoints ont été implémentés pour répondre aux besoins identifiés :

**/get_compatibility_score_with_resume**

— **Entrées** : un fichier CV + une offre d'emploi.
— **Traitement** : combine parsing et matching.
— **Sortie** : un JSON contenant à la fois :

  - le score de compatibilité,
  - une version structurée du CV.
- **Usage typique** : utile pour l'interface utilisateur de Talinty, permettant d'afficher le score tout en affichant le contenu extrait ou analysé du CV. apres la phase du parsing

L'intégration de ces APIs a été pensée pour être fiable et sécurisée :

- **Authentification par clé API** : chaque requête doit inclure un jeton d'autorisation, garantissant que seuls les services internes de Talinty peuvent interagir avec l'API.
- **Validation des entrées** : la présence du CV et de l'offre est systématiquement vérifiée, évitant les erreurs dues à des requêtes incomplètes.
- **Gestion des exceptions** : chaque appel est encapsulé dans un bloc de gestion des erreurs afin de fournir des messages clairs en cas d'échec (ex. fichier vide, mauvais format, erreur côté LLM).

### 4.5.3 Résultats du matching dans Talinty

Afin d'illustrer concrètement le fonctionnement du module de matching, nous présentons dans cette section l'interface Evaluation ou le score du matching par AI est affiché comme le montre la figure 4.2.

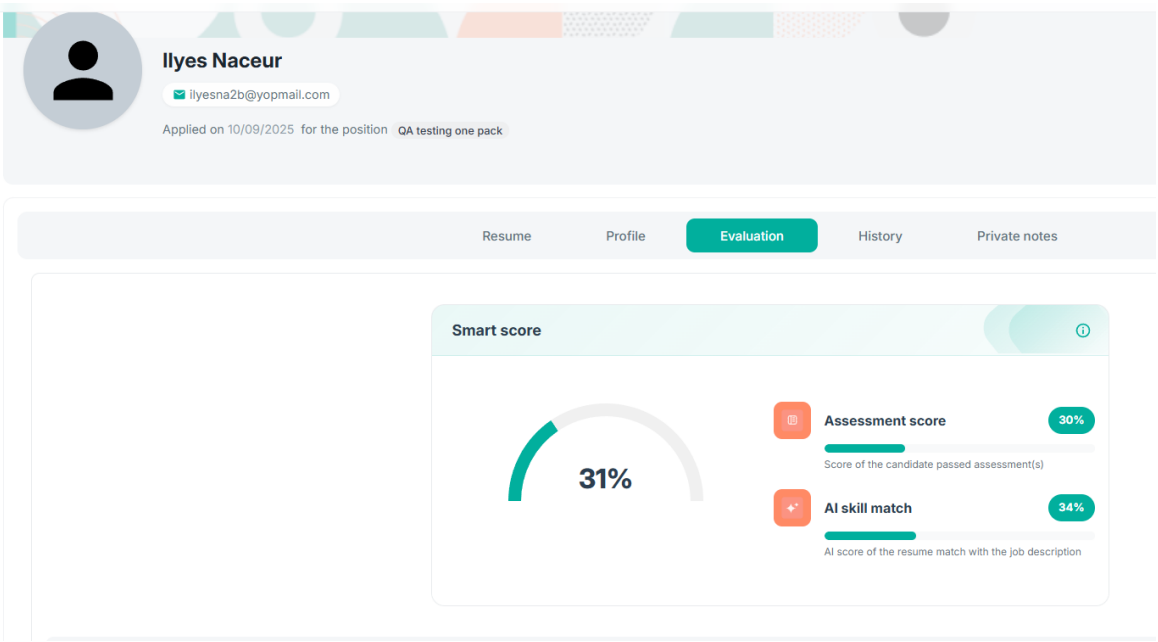



FIGURE 4.2 – Résultat du matching d'un CV- offre d'emploi

# Conclusion

Dans ce chapitre, nous avons présenté le processus de mise en correspondance entre CV et offres d'emploi au sein de la plateforme Talinty. Nous avons décrit les deux approches explorées : d'abord les modèles d'embeddings associés à une base vectorielle, puis l'intégration des LLM pour une compréhension plus fine et un scoring global du matching. Le chapitre suivant portera sur le chatbot développé pour la plateforme Talinty, un module interactif visant à assister les utilisateurs et à exploiter les résultats de parsing et de matching de manière plus intuitive.

# Chapitre 5

# Chatbot de la plateforme Talinty

# Introduction

Dans les chapitres précédents, nous avons présenté le moteurde parsing puis de matching, dont les résultats permettent d'exploiter des informations essentielles à partir des CV et des offres d'emploi. Ces données, combinées à d'autres sources présentes dans la plateforme, peuvent être exploitées dans un cadre plus interactif grâce à un chatbot intelligent. Celui-ci offre aux utilisateurs la possibilité d'interroger le système en langage naturel, d'obtenir des réponses contextuelles et enrichies, et de naviguer plus facilement dans les informations disponibles. Le chapitre suivant est ainsi consacré à la conception et à la mise en œuvre du Chatbot Talinty, basé sur les LLM et la recherche vectorielle sémantique.

Dans ce chapitre, nous commençons par définir les objectifs poursuivis par le chatbot ainsi que ses principaux cas d'usage. Nous décrivons ensuite son pipeline de traitement, la base de données sous-jacente et le mécanisme d'embeddings utilisé, avant de présenter les résultats expérimentaux et les limites rencontrées.

## 5.1 Objectif et cas d'usage du Chatbot Talinty

L'objectif principal du **Chatbot Talinty** est de permettre une interaction fluide en langage naturel avec la plateforme de recrutement. Plutôt que de naviguer manuellement dans les dif-

férentes sections (offres, candidats, candidatures, rapports, etc.), l'utilisateur peut simplement poser une question ou formuler une requête en langage courant.

Le chatbot se charge alors de comprendre l'intention, de rechercher les informations pertinentes dans la base `MongoDB` (et ses embeddings vectoriels), puis de fournir une réponse claire, personnalisée et contextualisée.

Cette approche vise à :

— **Simplifier l'expérience utilisateur** en réduisant le nombre de clics et de recherches manuelles.

— **Accélérer l'accès à l'information** en exploitant la puissance de la recherche vectorielle et du LLM.

— **Améliorer l'efficacité** des recruteurs et candidats dans leurs interactions quotidiennes avec Talinty.

Les cas pricipaux d'usage du chatbot sont :

### La recherche d'offres

L'utilisateur peut interroger le chatbot pour trouver rapidement les offres correspondant à un domaine, un poste ou un ensemble de compétences.

**Exemple :** *« Montre-moi les offres de data scientist disponibles à Paris. »*

Le chatbot interroge la base, filtre selon les critères demandés et retourne une liste pertinente.

### La recherche de candidats

Les recruteurs peuvent demander directement au chatbot de leur fournir une sélection de candidats adaptés à un poste ou à un profil recherché.

**Exemple :** *« Trouve-moi des candidats avec 5 ans d'expérience en développement Python. »*

Le chatbot exploite les embeddings des CV stockés dans `MongoDB` pour retrouver les profils les plus proches.

**Le suivi de candidatures**

Les utilisateurs peuvent consulter l'état d'avancement de leurs candidatures ou celles qu'ils gèrent (recruteurs).

**Exemple :** *« Quelle est la progression de la candidature de Sarah Dupont pour le poste de Project Manager ? »*

Le chatbot restitue les informations issues du suivi de candidature dans la plateforme.

## 5.2 Pipeline du Chatbot Talinty

Le pipeline du Chatbot Talinty suit une chaîne de traitement séquentielle qui transforme une requête utilisateur exprimée en langage naturel en une réponse claire, pertinente et enrichie. Chaque étape joue un rôle bien précis dans l'acheminement de l'information, depuis l'expression du besoin initial jusqu'à l'affichage de la réponse finale.

La figure 5.1 presente le pipeline complet du chatbot

### 1. Entrée : Requête de l'utilisateur

Le processus commence lorsque l'utilisateur soumet une demande à travers l'interface conversationnelle. Cette interface, intégrée à la plateforme **Talinty**, permet une interaction fluide et intuitive. Les requêtes peuvent concerner divers cas d'usage (cf section 5.1) .

### 2. Traitement initial de la requête

Une fois envoyée, la requête est transmise au serveur `FastAPI [16]`, qui sert de passerelle entre l'interface utilisateur et les composants internes du chatbot. La requête est ensuite analysée par l'agent conversationnel orchestré par `LangGraph [33]`. Cet agent joue un rôle de décision :

— soit il estime que la question peut être résolue directement par le modèle de langage (LLM) sans autre traitement,

— soit il choisit de consulter la base de données vectorielle afin de récupérer des informations complémentaires.

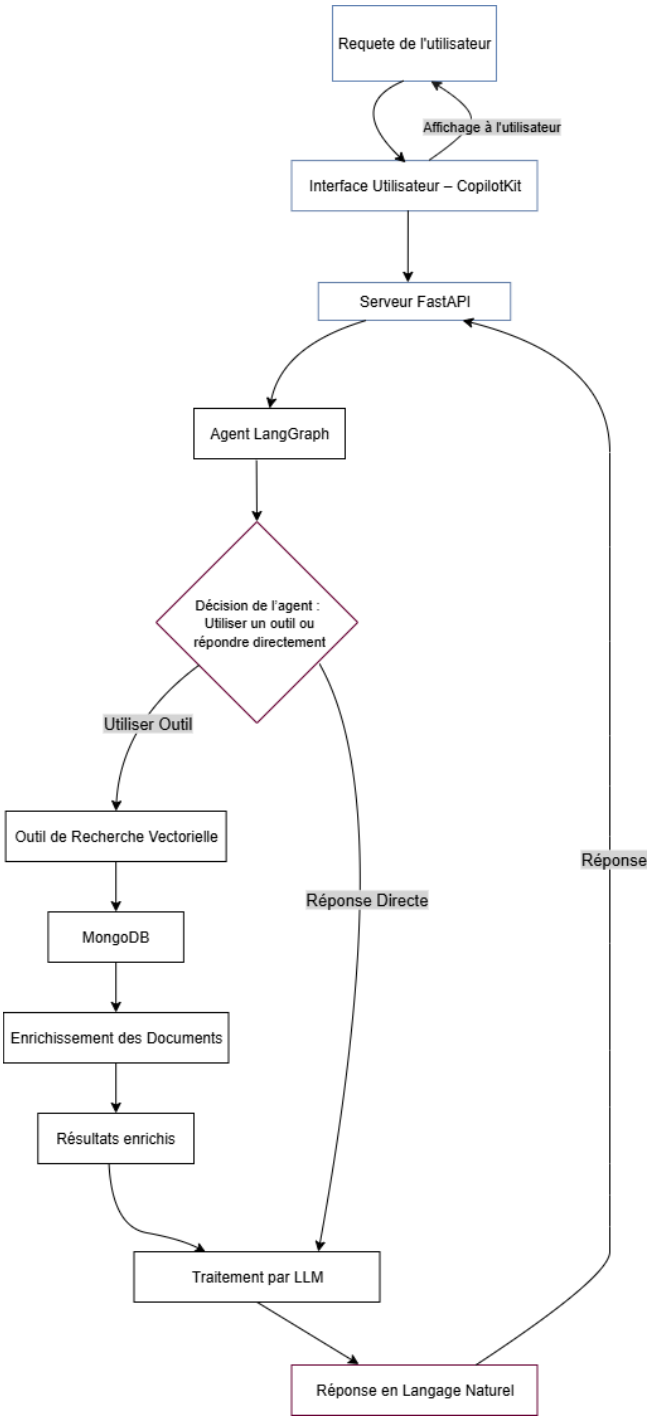



FIGURE 5.1 – Pipeline du chatbot Talinty

## 3. Génération des embeddings

Si une recherche documentaire est nécessaire, la requête est convertie en une représentation vectorielle (*embedding*) à l'aide d'un modèle spécialisé d'encodage sémantique. Cette étape est essentielle car elle permet de représenter le sens de la requête dans un espace vectoriel

multidimensionnel. Ainsi, deux textes proches en signification produisent des vecteurs proches dans cet espace, ce qui ouvre la voie à une comparaison efficace avec les données stockées.

### 4. Recherche vectorielle dans la base MongoDB

L'embedding généré est ensuite utilisé pour interroger la base vectorielle, constituée de documents enrichis tels que les offres d'emploi, les CV et les candidatures. Dans le cadre de **Talinty**, cette base repose sur `MongoDB`, couplée à un module de recherche vectorielle. Grâce à cette technologie, la plateforme peut effectuer une recherche sémantique, c'est-à-dire retrouver les documents les plus proches conceptuellement de la requête, même si les mots exacts ne correspondent pas.

### 5. Enrichissement des résultats

Les documents retrouvés sont ensuite enrichis. Cet enrichissement inclut l'ajout de métadonnées contextuelles par exemple les scores de pertinence, les correspondances extraites lors du parsing et du matching, ou encore les informations clés identifiées dans le CV ou la description de poste. Cette étape permet de fournir au LLM un contexte plus riche et plus structuré, afin d'améliorer la qualité et la précision de la réponse générée.

### 6. Réponse finale générée par le LLM

Enfin, le modèle de langage (LLM), choisi par l'entreprise — en l'occurrence **Gemini** de Google prend en entrée les résultats enrichis et génère une réponse en langage naturel. Celle-ci est formulée de manière claire, concise et contextualisée afin de répondre précisément au besoin exprimé par l'utilisateur. La réponse est ensuite transmise à l'interface utilisateur, bouclant ainsi le cycle de l'interaction.

## 5.3 Base de données utilisée

La plateforme Talinty utilise MongoDB Atlas comme base centrale, un choix motivé à la fois par la continuité avec l'infrastructure existante de l'entreprise et par la volonté d'explorer ses capacités avancées, notamment l'intégration d'embeddings et de la recherche vectorielle, malgré la présence d'outils plus spécialisés

### 5.3.1 Organisation des collections principales

Les données métier de **Talinty** sont structurées autour de collections interconnectées, chacune représentant une dimension clé du processus de recrutement :

— **jobs** : regroupe les offres publiées avec leurs métadonnées (titre, description, compétences requises, localisation, statut de publication, etc.).
— **candidates** : contient les données relatives aux candidats, incluant CV bruts, expériences, compétences, profils sociaux, résumé professionnel et informations issues du parsing.
— **evaluations** : relie un candidat à une offre spécifique, en enregistrant le suivi du processus de recrutement (étapes franchies, scores attribués, commentaires des recruteurs, statut).
— **talentpools** : réunit des profils intéressants mais non rattachés à une offre active, afin de constituer un vivier pour des opportunités futures.
— **users** : représente les différents acteurs de la plateforme (recruteurs, managers, administrateurs), avec gestion des rôles et permissions.

Ces collections constituent le socle opérationnel du système **Talinty**, permettant une navigation fluide entre les offres, les candidats et leurs candidatures.

la figure 5.2 presente le diagramme de classe déja développé par l'équipe backend

### 5.3.2 Avantages de MongoDB dans ce contexte

L'utilisation de `MongoDB` présente plusieurs atouts pour **Talinty** :

— **Continuité technologique** : pas besoin d'introduire un nouveau SGBD, ce qui simplifie la maintenance et réduit la dette technique.
— **Scalabilité native** : `MongoDB Atlas` supporte la montée en charge, garantissant la performance même avec des millions de CV et d'offres stockés.
— **Modèle document flexible** : un CV enrichi peut contenir des expériences, compétences, résultats de parsing et métadonnées hétérogènes dans une même structure JSON.
— **Vector Search intégré** : depuis les dernières versions, `MongoDB Atlas` propose un module de recherche vectorielle natif, évitant de recourir à un outil externe.
— **Requêtes hybrides** : possibilité de combiner filtres traditionnels (ex. : « expérience 5 ans », « basé à Paris ») avec une similarité vectorielle (ex. : « proches sémantiquement de Data Scientist »).
— **Sécurité et conformité** : `Atlas` fournit un haut niveau de protection des données sensibles (chiffrement, authentification fine, conformité RGPD/SOC2).

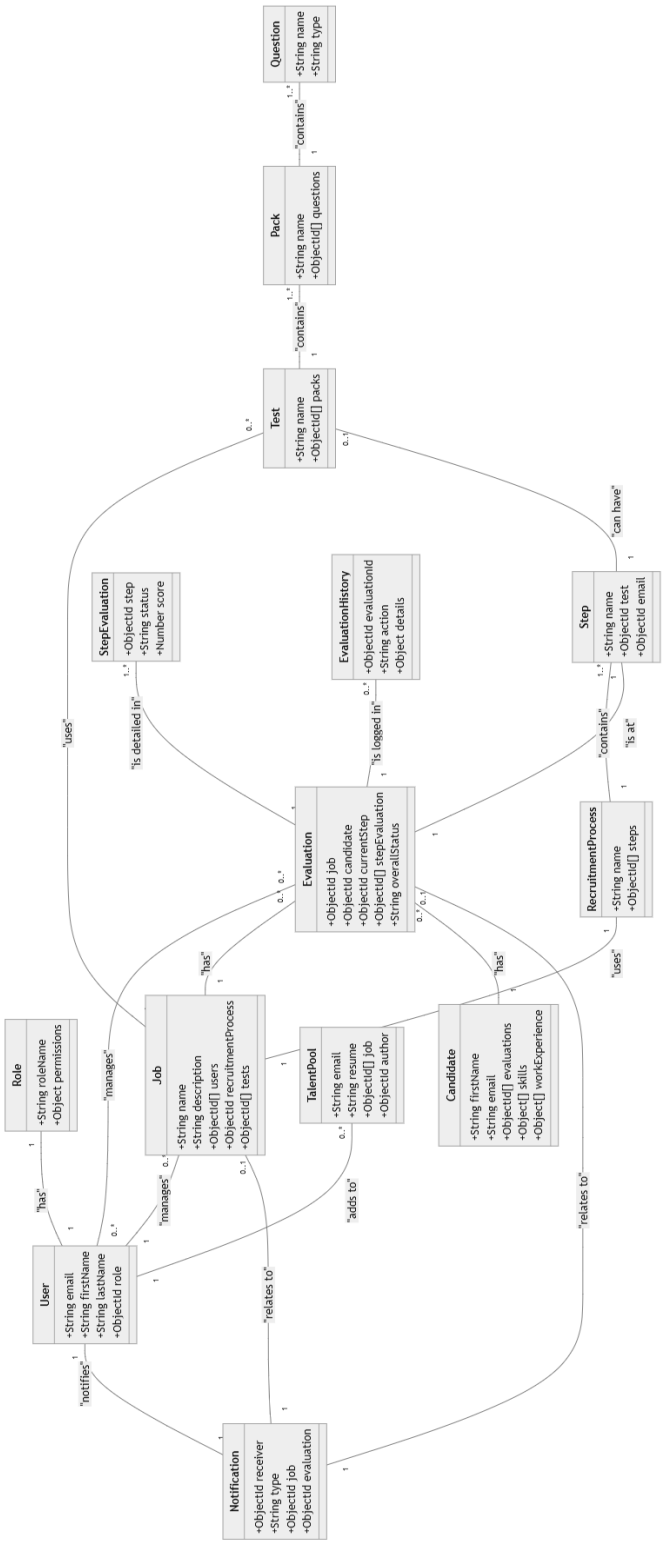



FIGURE 5.2 – Diagramme de classe - BD Talinty

### 5.3.3 Rôle central de MongoDB dans le chatbot Talinty

`MongoDB` occupe une double fonction dans l'architecture du chatbot :

— **Base transactionnelle** → stockage opérationnel de toutes les entités métiers (offres, candidats, candidatures, utilisateurs).

— **Base vectorielle** → exploitation des embeddings pour la recherche sémantique, permettant au chatbot d'aller au-delà du mot-clé et de fournir des réponses pertinentes et contextualisées.

En réunissant les données métiers et les représentations vectorielles dans une même base, `MongoDB` devient le socle unifié de **Talinty**, conciliant robustesse opérationnelle et intelligence artificielle intégrée.

## 5.4 Embeddings et Recherche Vectorielle

Afin de permettre une recherche intelligente et contextuelle des informations extraites des documents (CV et offres d'emploi), nous avons mis en place une architecture basée sur les **embeddings textuels** et la **recherche vectorielle**. Cette approche dépasse largement les méthodes classiques de recherche par mots-clés, puisqu'elle capture la sémantique du texte et permet de retrouver des éléments similaires même si les formulations diffèrent.

### 5.4.1 Génération des Embeddings

Les embeddings représentent une projection des textes dans un espace vectoriel de grande dimension, où chaque mot, phrase ou document est représenté par un vecteur numérique.

Dans notre projet, nous utilisons un modèle d'embedding textuel pré-entraîné **text-multilingual-embedding-002** fourni par **Google**. Ce choix, imposé par l'entreprise, s'explique par :

— la robustesse de ces modèles,

— leur optimisation pour la recherche sémantique,

— leur compatibilité avec des pipelines de traitement modernes.

Concrètement, chaque CV ou description d'offre est découpé en segments textuels (compétences, expériences, diplômes, etc.), et pour chacun de ces segments, le modèle génère un vecteur dense de dimension fixe (par exemple 512 ou 768 dimensions). Ces vecteurs encodent non seulement le sens littéral des mots, mais aussi les relations contextuelles entre eux.

### 5.4.2 Stockage Vectoriel dans MongoDB

Une fois générés, les embeddings sont stockés directement dans une base de données `MongoDB`. Chaque entrée dans la base contient :

— l'élément textuel original (par exemple : une compétence issue d'un CV),
— son embedding vectoriel associé,
— des métadonnées (source du document, langue, identifiant candidat/offre, type d'entité, etc.).

### 5.4.3 Collection dédiée aux embeddings

Pour introduire une recherche sémantique et conversationnelle dans le chatbot **Talinty**, une collection supplémentaire a été créée :

— **global_embeddings** : contient les représentations vectorielles (*embeddings*) générées à partir des documents stratégiques (descriptions d'offres, CV enrichis, évaluations).

Chaque entrée associe :

— l'identifiant du document source (`job_id`, `candidate_id`, etc.),
— l'embedding vectoriel (tableau de flottants haute dimension),
— des métadonnées contextuelles (type de document, date d'indexation, score de parsing ou matching éventuel).

Cette approche permet de transformer `MongoDB` en une base hybride, capable de répondre à la fois à des requêtes classiques (filtrage par localisation, expérience) et à des recherches vectorielles plus avancées.

### 5.4.4 Recherche par Similarité Cosinus

Lorsqu'un utilisateur effectue une recherche (par exemple, trouver les CV les plus proches d'une offre d'emploi donnée), le texte de la requête est lui aussi converti en embedding via le même modèle de **Google**.

La recherche s'effectue ensuite par calcul de la **similarité cosinus** entre le vecteur de la requête et les vecteurs stockés. La similarité cosinus mesure l'angle entre deux vecteurs dans l'espace vectoriel, et permet d'évaluer leur proximité sémantique indépendamment de leur magnitude.

Le système retourne les $k$ résultats les plus proches (*top-k*), classés par ordre de pertinence. Cette approche permet par exemple de retrouver un CV mentionnant « développement en Python » même si l'offre spécifie « maîtrise du langage Python », ce qui serait difficile avec une simple recherche lexicale.

Cette approche présente plusieurs bénéfices majeurs :

— **Robustesse sémantique** : surmonte les limites des synonymes et des reformulations.
— **Scalabilité** : `MongoDB` permet d'indexer et d'interroger efficacement des millions de vecteurs.
— **Personnalisation** : possibilité d'ajuster le modèle et les métriques pour des besoins spécifiques (ex. pondération plus forte des compétences techniques que des expériences).
— **Temps de réponse rapide** : l'indexation vectorielle optimise la recherche top-k en réduisant la complexité des comparaisons.

## 5.5 Résultats et cas d'utilisation

Dans cette section nous présentons les cas d'utilisation avec les resultats.

### 5.5.1 Exemples de requêtes valides

La table 5.1 illustre quelques requêtes typiques testées avec succès et le comportement attendu du système

**Remarque :** les requêtes purement « conversationnelles » ou de contrôle d'interface (ex. « bonjour », « passe en dark mode ») sont ignorées par le routeur d'outils.

### 5.5.2 Qualité des réponses observées

La qualité des réponses observées se distingue par plusieurs aspects complémentaires. La pertinence sémantique est globalement correcte pour les entités explicitement nommées, comme un intitulé de poste ou le nom d'un · e candidat · e. Grâce aux embeddings (text-embedding-005) combinés à la recherche vectorielle MongoDB, le système parvient à restituer des documents proches du sens de la requête, même lorsque le libellé diffère légèrement. Les réponses sont également contextualisées grâce à un enrichissement par recoupement entre les collections jobs, evaluations, candidates et users, ce qui permet de fournir des informations complètes incluant,

| **Requête (NL)** | **Intention comprise** | **Outil/flux activé** | **Sortie attendue** |
|---|---|---|---|
| « Détaille l'offre Senior Frontend Developer » | Récupération d'une fiche job par intitulé | Embedding → Vector Search (jobs) → Enrichissement (users, process, tests) | Titre, description, compétences, statut, managers, process associé |
| « Liste les candidats qui ont postulé au poste UI/UX Designer » | Jointure candidats ↔ évaluations ↔ job | Embedding → Vector Search (evaluations) + filtres → Enrichissement (candidates) | Tableau des candidats, statut d'évaluation, score global, date d'application |
| « Quel est le statut de la candidature de Jane Smith pour Backend Developer ? » | Recherche d'une évaluation précise | Embedding → Vector Search (evaluations) → Enrichissement (step, overallStatus) | overallStatus, currentStep, dates clés, responsable actuel |
| « Compétences et années d'expérience de Sarah Johnson » | Profilage d'un candidat | Vector Search (candidates) → Enrichissement (workExperience, skills) | Liste des compétences (niveau, années), expériences, résumé |
| « Montre-moi les jobs ouverts en Data à Paris » | Filtrage hybride sémantique + attributs | Vector Search (jobs) + filtres (location, active) | Liste des postes actifs, localisation, date d'expiration |

TABLE 5.1 – Exemples de requêtes valides et résultats attendus

par exemple, le statut, l'étape du processus ou le responsable concerné. Par ailleurs, Gemini 2.5 Pro assure une robustesse linguistique notable, avec des réponses claires, structurées et fidèles aux données disponibles. Le système montre aussi une bonne tolérance aux variations de formulation, en traitant efficacement les paraphrases et synonymes (par exemple « postulés à » et « appliqués à »). Enfin, la gestion multilingue est jugée acceptable lorsque les données sources sont homogènes, mais les performances se dégradent si les entités sont enregistrées dans plusieurs langues sans normalisation préalable.

### 5.5.3 Limites rencontrées

Malgré les résultats obtenus, certaines limites subsistent dans le fonctionnement du système. La première concerne la sélection d'outils (tool routing), qui peut parfois s'avérer incertaine : l'agent hésite ou mobilise un outil inadapté lorsque les prompts de routage sont trop génériques ou que les signaux interprétés restent ambigus. Une autre difficulté réside dans l'inclusion de

candidats inactifs parmi les résultats, due à l'absence de filtres métier suffisamment stricts en amont, ce qui conduit à prendre en compte des candidatures clôturées ou désactivées. De plus, bien que MongoDB ait permis d'intégrer efficacement la recherche vectorielle, ses performances demeurent limitées par rapport aux bases vectorielles spécialisées, entraînant parfois une latence accrue ou une qualité moindre. Enfin, le rafraîchissement des embeddings ne garantit pas toujours une prise en compte immédiate des données les plus récentes, ce qui peut réduire la précision des réponses fournies.

### 5.5.4 Résultat Final

Afin d'illustrer concrètement le fonctionnement du chatbot Talinty, nous présentons ci-dessous quelques captures d'écran de l'application.

Les figures suivantes montrent la conversation entre l'utilisateur (recruteur) et le chatbot

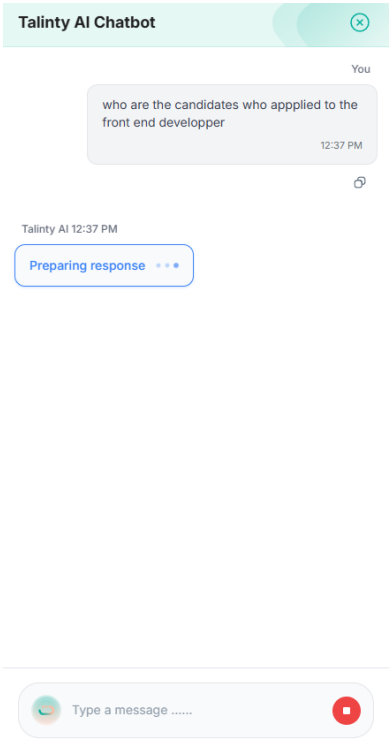



FIGURE 5.3 – Conversation Chatbot- Partie 1

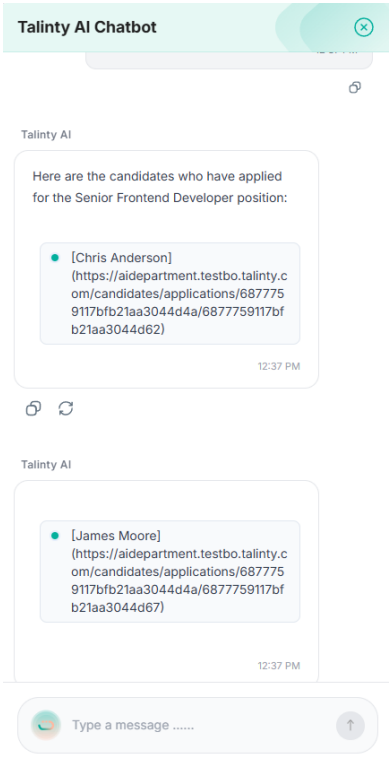



FIGURE 5.4 – Conversation Chatbot- Partie 2

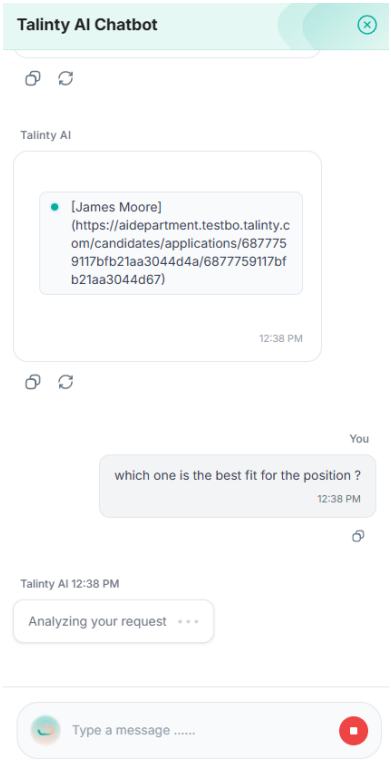



FIGURE 5.5 – Conversation Chatbot- Partie 3

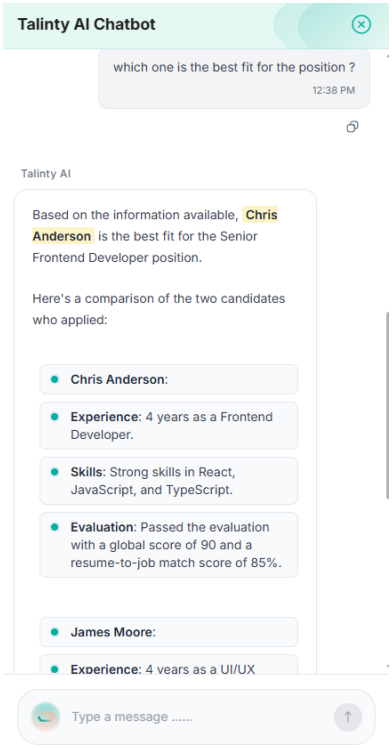



FIGURE 5.6 – Conversation Chatbot - Partie 4

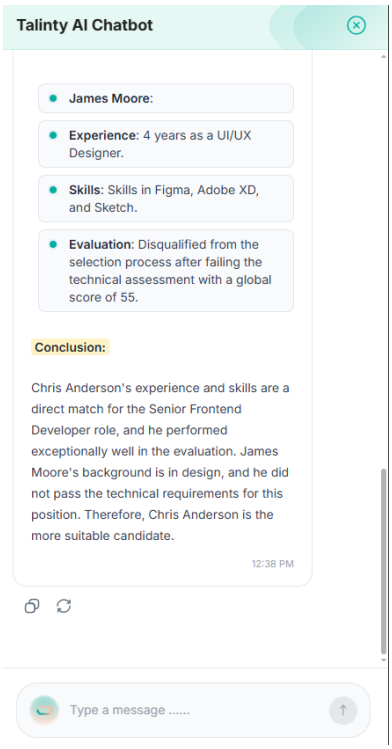



FIGURE 5.7 – Conversation Chatbot -Partie 5

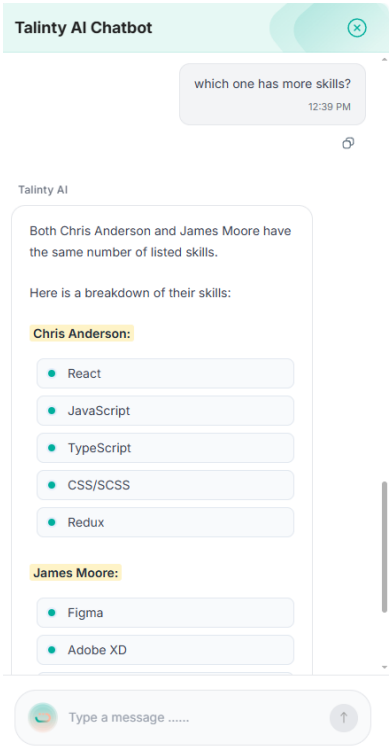



FIGURE 5.8 – Conversation Chatbot -Partie 6

# Conclusion

En conclusion, le Chatbot Talinty représente une avancée pour la plateforme de recrutement, en offrant une interaction fluide et intuitive en langage naturel. Grâce à l'intégration de modèles de langage (LLM) et de la recherche vectorielle sémantique via MongoDB Atlas, il simplifie l'accès aux informations, accélère les processus de recherche d'offres et de candidats, et améliore l'efficacité des recruteurs et des utilisateurs. Ce système, appuyé par une architecture robuste et évolutive, transforme l'expérience utilisateur en réduisant les frictions et en fournissant des réponses contextuelles pertinentes.

# Conclusion Générale et Perspectives

Le présent rapport est élaboré dans le cadre d'un projet de fin d'études pour l'obtention d'un mastère en science des données, effectué au sein de l'entreprise AstroLab Group.
L'objectif principal de ce projet était de concevoir et développer une solution innovante, intégrée à la plateforme Talinty, afin d'optimiser le processus de recrutement grâce aux avancées récentes de l'Intelligence Artificielle. La solution proposée repose sur trois modules qui permettent le parsing des documents de recrutement , la mise en correspondance entre le candidat et l'offre d'emploi et l'interaction via un chatbot conversationnel.

Dans le module de parsing, une première approche basée sur des techniques NLP traditionnelles a montré rapidement ses limites en termes de précision et de robustesse. Nous avons alors exploré des approches modernes avec des LLM orchestrés via LangChain, en expérimentant plusieurs modèles récents tels que LLaMA 3.1 8B Instant, Gemini 2.0 Flash et LLaMA 3.3 70B Versatile. Ce dernier ayant montré les meilleures performances.

Concernant le module de matching entre CV et offres de travail, une première approche a consisté à utiliser des modèles d'embeddings (Sentence Transformers) et une base vectorielle Qdrant pour calculer les scores de similarité. Bien que fonctionnelle, cette approche présentait certaines lacunes, ce qui nous a conduits à adopter des LLM dans la deuxième approche, permettant d'obtenir des résultats nettement plus précis et contextuels. Enfin,le troisème module le chatbot conversationnel a été conçu à partir d'embeddings stockés dans MongoDB Atlas et en s'appuyant sur un modèle avancé tel que Gemini 2.5 Pro, intégré avec LangChain et LangGraph, offrant ainsi une interaction fluide, personnalisée et adaptée au contexte utilisateur.

Les résultats obtenus démontrent la faisabilité technique de la solution proposée et soulignent la valeur ajoutée de l'intégration des LLM dans le processus de recrutement. L'extraction et l'enrichissement automatiques des profils candidats, la pertinence accrue du matching et le déploiement d'un chatbot capable d'accompagner les recruteurs de manière interactive montrent le potentiel transformateur de cette approche et ouvrent la voie à une modernisation profonde de l'expérience de recrutement.

Plusieurs perspectives se dessinent à l'issue de ce travail. Nous envisageons dans un premier temps d'améliorer les modules de parsing en affinant l'extraction des entités nommées et des informations clés à travers l'intégration de modèles de LLM plus spécialisés dans le traitement de documents complexes. Nous souhaitons également renforcer le module de matching en optimisant la pondération des critères de correspondance et en intégrant des techniques de reranking basées sur des LLM afin d'améliorer encore la pertinence des résultats. Après une première expérimentation avec MongoDB pour le stockage des documents et des embeddings, nous visons une migration vers des bases vectorielles spécialisées telles que Pinecone ou Weaviate, qui devraient permettre d'accroître considérablement la rapidité et la précision des recherches sémantiques. En parallèle, nous prévoyons d'explorer des modèles de génération récents et plus puissants, notamment GPT-4.1 et Mistral, afin d'améliorer les performances tant pour le parsing que pour le matching et l'interaction conversationnelle. L'évolution du chatbot Talinty constitue également une piste majeure : nous projetons de le faire évoluer vers une version personnalisée, enrichie de modules d'analytique prédictive capables de proposer de manière proactive le meilleur profil aux recruteurs, tout en fournissant une explication transparente et justifiée de la recommandation (explainable recommendation). Enfin, nous ambitionnons d'intégrer des fonctionnalités RH complémentaires telles que le suivi automatisé des entretiens, les relances programmées auprès des candidats, ainsi que la génération dynamique de rapports statistiques et de tableaux de bord décisionnels, ce qui renforcerait encore davantage la valeur ajoutée de la plateforme Talinty.

# Annexe

# Approche du module de Parsing : avec NLP classique

Avant d'intégrer des modèles avancés, une première approche a été mise en place en s'appuyant sur des méthodes classiques de NLP afin de développer un moteur d'analyse intelligent des données des CVs et offres d'emploi, capable d'extraire automatiquement et de structurer toutes les informations pertinentes

## Etapes de traitement

Le pipeline du parsing a été conçu autour de plusieurs étapes illustrées dans la figure 5.9.




FIGURE 5.9 – Pipeline du Parsing

### Étape 1 : Extraction du Texte Brut depuis les Fichiers

La première étape de notre pipeline consiste à convertir le CV, initialement au format PDF, en une chaîne de caractères brute. Cette phase, bien que paraissant simple, est en réalité l'une des plus critiques et complexes du processus.

Face aux défis de la fragmentation du texte et de la perte de l'ordre logique, le choix d'une bibliothèque d'extraction performante était primordial. D'autres bibliothèques comme

pypdf, une solution très populaire et entièrement écrite en Python, ont également été évaluées. Bien qu'elle offre une grande flexibilité et une installation simplifiée, ses performances peuvent être en retrait sur les documents à la mise en page complexe. C'est pourquoi nous avons finalement sélectionné PDFreader pour plusieurs raisons stratégiques :

- **Performance et Efficacité** : PDFreader offre une vitesse d'exécution élevée, essentielle pour traiter rapidement un grand volume de CV et garantir la réactivité de l'application.
- **Fiabilité de la Restitution de l'Ordre de Lecture** : C'est son avantage le plus déterminant. PDFreader intègre des algorithmes sophistiqués capables de reconstruire l'ordre de lecture logique d'un document. La bibliothèque analyse la disposition des blocs de texte sur la page pour les réassembler dans un ordre proche de celui d'une lecture humaine. Cette capacité à gérer correctement les mises en page complexes (multi-colonnes, etc.) est cruciale pour ne pas corrompre le sens du texte avant même de commencer l'analyse NLP.

#### Le Processus d'Extraction en Pratique

Le script d'extraction suit un processus simple mais robuste :

1. Le fichier PDF est ouvert en mémoire.
2. Le programme itère sur chaque page du document.
3. Pour chaque page, la fonction d'extraction de texte de PDFreader est appelée pour extraire le contenu en respectant le flux de lecture.
4. Le texte de toutes les pages est ensuite concaténé pour former une unique chaîne de caractères.

#### Résultat : Un Texte Brut à Prétraiter

Le produit final de cette étape est une chaîne de caractères brute qui représente l'intégralité du contenu textuel du CV. Il est important de noter que ce texte, bien que logiquement ordonné, n'est pas encore "propre". Il peut contenir des artefacts de mise en page (comme des en-têtes ou pieds de page) et des imperfections liées au format PDF. Ce texte brut constitue le point d'entrée pour la phase suivante : le nettoyage et le prétraitement.

### Étape 2 : Prétraitement et Nettoyage du Texte

Le texte brut extrait est rarement propre. Il contient des artefacts de mise en page, des incohérences et du "bruit" qui peuvent perturber les modèles NLP.

Pour cla nous avons appliquer differentes techniques afin de le rendre plus homogèneet pertinenet

— **Techniques de Nettoyage Appliquées** :
  — **Normalisation de la Casse** : le texte est entierement converti en minuscules pour que des termes comme "Python" et "python" soient traités de la même manière.
  — **Suppression des éléments superflus** : Les sauts de ligne multiples (\n\n+), les tabulations, et les espaces excessifs sont remplacés par un espace unique. Les URLs et emails sont également supprimés à ce stade, car ils seront extraits de manière ciblée plus tard.
  — **Tokenisation** : le texte nettoyé est ensuite segmenté en unités de base (mots, ponctuations) appelées tokens. Cette opération est réalisée par spaCy, qui gère intelligemment les cas complexes comme les mots composés ou les abréviations.
  — **Gestion des Mots-Vides (Stop Words)** : les mots-vides sont des mots très fréquents qui portent peu de sens (ex : "le", "de", "et", "à"). Bien que leur suppression soit courante en NLP, nous l'appliquons avec précaution. Ils sont conservés pour l'étape de NER ( reconnaissance des entités nommées) contextuelle (pour ne pas transformer "Directeur des Ventes" en "Directeur Ventes"), mais peuvent être ignorés lors de l'extraction de listes de compétences.

**Étape 3 : Extraction d'Informations**

C'est l'étape la plus complexe, où l'intelligence du système opère. Nous avons implémenté une approche hybride pour combiner la précision des règles et la flexibilité du Machine Learning. Cette approche repose sur les régles et le ML pour reconnaitre les entités nommées [57].

**Approche Basée sur des Règles (Rule-Based)** Idéale pour les informations suivant un patron prévisible et rigide.

— **Principe** : Utilisation d'expressions régulières (Regex), des motifs de recherche de texte extrêmement puissants.
— **Entités Ciblées** :
  — Informations de Contact : Emails (`\b[A-Z0-9._%+-]+@[A-Z0-9.-+\.[A-Z]{2,}\b`), numéros de téléphone (gestion des formats +33 6, 06, avec ou sans espaces), URLs de profils LinkedIn ou GitHub.

— **Avantages et Limites** : Très rapide et 100% précis quand le format correspond. Cependant, un Regex est rigide et échoue si le format est légèrement différent de ce qui était prévu.

**Approche Basée sur le Machine Learning (NER)** Indispensable pour reconnaître des concepts sémantiques dans leur contexte.

— **Principe** : la Reconnaissance d'Entités Nommées (NER) consiste à identifier et classifier des mots ou des groupes de mots dans des catégories prédéfinies (Personne, Entreprise, Compétence, etc.).
— **Outil** : nous avons utilisé la bibliothèque spaCy et son modèle pré-entraîné en français, `fr_core_news_lg`. Ce modèle reconnaît nativement des entités comme PER (Personne), ORG (Organisation) et LOC (Lieu).

### Étape 4 : Structuration par Sections et Résultat

La dernière étape de la pipeline a pour but de consolider toutes les informations collectées durant le traitement dans un format structuré unique : un objet JSON. L'approche adoptée a consisté à segmenter le CV en blocs de texte correspondant aux sections principales (expérience, formation, etc.) et à enrichir cette sortie avec diverses métadonnées issues de l'analyse NLP.

La logique de cette étape finale se déroule suivnat deux sous étapes :

1. **Segmentation** : le système tente d'isoler les blocs de texte correspondant à chaque grande partie du CV. Par exemple, le contenu textuel de la section "Expérience Professionnelle" est stocké dans la clé `experience_section`.
2. **Aggrégation** : le dictionnaire final est ensuite peuplé avec l'ensemble des données produites par la pipeline : le texte brut, le texte nettoyé, la liste des entités NER, des features NLP (mots-clés, n-grams), et les sections de texte identifiées.

Le résultat est un objet JSON complet qui agit comme une "fiche d'identité" détaillée du traitement du CV. Voici sa structure :

```
{
    // Métadonnées et données brutes
    "unique_id": "...",
    "resume_data": "Texte brut du CV...",
    "clean_data": "Texte nettoyé et normalisé...",

    // Features NLP de bas niveau
```

```
    "entities": "[('Jean Dupont', 'PER'), ('BNP Paribas', 'ORG'), ...]",
    "extracted_keywords": ["java", "gestion de projet", "sql", ...],
    "keyterms": {...},
    "bi_grams": "...",
    "tri_grams": "...",
    "pos_frequencies": {...},

    // Sections de profil extraites
    "name_section": "Jean Dupont",
    "phone_section": "0612345678",
    "location_section": "Paris, France",
    "email_section": "jean.dupont@email.com",
    "social_media_section": "linkedin.com/in/jeandupont",

    // Sections complexes
    "experience_section": "Développeur Java Senior chez BNP Paribas (2020-2022)...",
    "skills_section": "Java, Python, Gestion de projet, SQL...",
    "education_section": "Master en Informatique - Université Paris-Saclay (2019)..."
}
```

## Évaluation des Performances

Développer un parser est une chose ; prouver son efficacité en est une autre. Une évaluation rigoureuse est essentielle pour quantifier sa performance et justifier sa fiabilité.

### Méthodologie et Métriques

Pour évaluer notre modèle, nous avons constitué un jeu de données de test composé de 40 CV fourni par l'entreprise mis de côté et jamais utilisés lors de l'entraînement. Ces CV ont été annotés manuellement pour créer une "vérité terrain" (ground truth), c'est-à-dire une version parfaite du JSON attendu pour chaque CV. La sortie de notre parser est ensuite comparée à cette vérité terrain pour chaque entité. Nous Avons pris 30 CV pour l'entrainement et 10 CV pour tester. De meme pour les offres d'emploi, nous avons préparer 40 offres d'emploi dont 30 pour l'entrainement et 10 pour tester. [56] Dans cette comparaison, nous distinguons :

- **Vrai Positif (True Positive - TP)** : L'entité a été correctement identifiée par le parser. (Ex : le parser identifie "Python" comme une compétence, et c'en est une).
- **Faux Positif (False Positive - FP)** : Le parser a identifié une entité qui n'en est pas une. (Ex : le parser identifie "Anglais" comme un nom de personne).
- **Faux Négatif (False Negative - FN)** : Le parser a manqué une entité qui était présente.

(Ex : le CV mentionne "JavaScript", mais le parser ne l'a pas trouvé).

À partir de ces comptages, nous calculons trois métriques clés :

1. **La Précision** : Elle mesure la fiabilité des prédictions du parser [56].
   - **Formule** :
   $$Précision = \frac{TP}{TP + FP}$$
   - **Interprétation** : "Parmi toutes les entités que le parser a identifiées, quel pourcentage était correct ?" Une haute précision signifie que le parser n'invente pas d'informations erronées.
2. **Le Rappel (Recall)** : Il mesure l'exhaustivité du parser [56].
   - **Formule** :
   $$Rappel = \frac{TP}{TP + FN}$$
   - **Interprétation** : "Parmi toutes les entités qui existaient réellement dans les CV, quel pourcentage le parser a-t-il réussi à trouver ?" Un rappel élevé signifie que le parser manque peu d'informations importantes.
3. **Le F1-Score** : C'est la métrique la plus importante car elle représente un équilibre entre la Précision et le Rappel [56].
   - **Formule** :
   $$F1 - Score = 2 \times \frac{Précision \times Rappel}{Précision + Rappel}$$
   - **Interprétation** : Le F1-Score est la moyenne harmonique de la précision et du rappel. Un score de 1.0 est parfait, tandis que 0.0 est le pire score. Il pénalise les modèles qui sacrifient trop une métrique au profit de l'autre.

## Résultats et Analyse Critique

Afin d'évaluer la qualité du premier prototype de parsing basé sur des méthodes classiques de traitement automatique du langage (Regex, dictionnaires et modèles NER simples), une série de tests a été réalisée sur le corpus représentatif de CV et d'offres d'emploi. L'objectif était de mesurer la capacité du système à extraire automatiquement les entités essentielles pour un futur moteur de matching.

Les résultats obtenus sont résumés dans le tableau 5.2 :

Ces résultats montrent clairement les limites d'une approche classique :

| Entité Cible | Précision | Rappel | F1-Score | Analyse de la Performance |
|---|---|---|---|---|
| Nom / Prénom | 0.93 | 0.90 | 0.91 | Correct : souvent bien identifié, mais erreurs fréquentes lorsque le nom est dans l'en-tête ou stylisé. |
| Email | 0.97 | 0.94 | 0.95 | Bon : la détection par Regex fonctionne, mais des adresses incomplètes ou déformées sont parfois extraites. |
| Réseaux Sociaux | 0.85 | 0.80 | 0.82 | Acceptable : LinkedIn est bien détecté, mais d'autres réseaux (GitHub, Twitter) sont souvent manqués. |
| Compétences (Skills) | 0.65 | 0.55 | 0.59 | Mauvais : près de la moitié des compétences passent inaperçues, notamment lorsqu'elles sont décrites en phrases contextuelles. |
| Éducation | 0.70 | 0.60 | 0.64 | Médiocre : forte confusion entre écoles et entreprises ; informations de diplômes souvent tronquées. |
| Expérience | 0.68 | 0.58 | 0.62 | Mauvais : les dates sont mal alignées, les postes mal reliés aux entreprises, rendant le parcours incohérent. |

TABLE 5.2 – Résultats des performances du parsing classique par type d'entité

- **Extraction correcte mais triviale des contacts**
  Les informations de contact (Nom, Email, Réseaux sociaux) sont relativement fiables car elles suivent des formats standardisés. Cependant, ces données ne représentent qu'une fraction superficielle du profil, insuffisante pour un système de recrutement.

- **Compétences : le véritable point faible**
  L'extraction des compétences représente la principale faiblesse du système, comme en témoigne son score F1 de 0.59. Avec un rappel de seulement 55%, le système ne parvient à identifier qu'un peu plus de la moitié des compétences présentes dans les CV. C'est un échec critique, car les compétences sont essentielles pour évaluer et comparer les profils des candidats aux offres d'emploi, et leur mauvaise détection compromet l'ensemble du processus de sélection.

- **Échec de structuration de l'Expérience**
  L'expérience professionnelle est une entité composite (poste, entreprise, dates, description).

Le système identifie des fragments, mais échoue à les relier. le resultat montre des expériences incomplètes, des chevauchements de dates, voire des incohérences (un stage pris pour un CDI). La trajectoire professionnelle du candidat devient illisible.

- **Confusion sur l'Éducation**
  Le rappel de 0.60 reflète une incapacité à distinguer correctement entre formations académiques et expériences professionnelles.Le résultat illustre des universités sont classées comme employeurs, des projets étudiants sont interprétés comme emplois, ce qui dégrade fortement la fiabilité du parsing.

En résumé, si cette approche s'en sort convenablement sur des entités simples et formatées (nom, email, liens), il échoue largement sur les informations riches et déterminantes (compétences, expérience, éducation). Ces faiblesses rendent les résultats inutilisables dans un contexte réel de recrutement, où la qualité de l'information extraite conditionne directement la pertinence du matching.

Cette analyse confirme la nécessité d'abandonner une approche purement basée sur Regex et règles fixes au profit de modèles plus avancés, intégrant des LLM et un pipeline d'orchestration comme LangChain, afin de traiter la complexité et l'hétérogénéité des documents.

# Bibliographie


[1] E. Adamopoulou and L. Moussiades. An overview of chatbot technology. Artificial Intelligence Applications and Innovations, 1237 :373–383, 2020.

[2] Affinda. Affinda – ai-powered cv and document parsing, 2025.

[3] Astrolab Agency. Site web astrolab agency, 2024.

[4] Anthropic. Claude (modèle de langage) – fiche et évolutions, 2025.

[5] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. International Conference on Learning Representations (ICLR), 2015.

[6] Jack Black and Peter van Esch. Artificial intelligence in hiring : a review of current research and recommendations for future directions. International Journal of Human Resource Management, 33(12) :2461–2489, 2022.

[7] François Bournois and Jean-Marie Peretti. Gestion des ressources humaines. Economica, 2007.

[8] Tom B. Brown et al. Language models are few-shot learners. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[9] Cegid. Découvrez les 10 tendances ats en 2025 pour les rh. Cegid Blog, 2025.

[10] L. Ciechanowski, A. Przegalinska, M. Magnuski, and P. Gloor. In the shades of the uncanny valley : An experimental study of human–chatbot interaction. Future Generation Computer Systems, 92 :539–548, 2018.

[11] ClickUp. Clickup – project management platform, 2025.

[12] Google DeepMind. Introducing gemini : our largest and most capable ai model, 2023.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT, pages 4171–4186, 2019.

[14] diagrams.net. Draw.io – online diagramming tool, 2025.

[15] Docker. Docker – container platform, 2025.

[16] FastAPI Project. Fastapi – python web framework, 2025.

[17] Git. Système de contrôle de version git, 2025.

[18] GitHub. Plateforme github, 2025.

[19] Gnani.ai. How do voice bots handle languages & accents ?, 2023.

[20] Harry Guinness. Meta ai : What is llama 4 and why does it matter ?, 2025.

[21] K. Gupta, P. Goyal, and M. Bhatia. Ai powered resume parsing : A survey and future directions. International Journal of Advanced Computer Science and Applications (IJACSA), 12(5) :560–567, 2021.

[22] Rob J. Hyndman and George Athanasopoulos. Forecasting : Principles and Practice. OTexts, 2nd edition edition, 2018.

[23] IBM. What is mistral ai ?, 2024.

[24] Pinecone Systems Inc. Pinecone : Scalable vector database, 2023. Available at : https://www.pinecone.io/.

[25] Indeed. Indeed - job search engine.

[26] OpenAI / Indeed. Delivering contextual job matching for millions with openai, 2025.

[27] C. Indumathi and R. Banu. Automatic resume parsing and annotation for ranking and selection of candidates. International Journal of Pure and Applied Mathematics, 117(16) :427–433, 2017.

[28] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. IEEE Transactions on Big Data, 2019.

[29] Daniel Jurafsky and James H. Martin. Speech and Language Processing. Pearson, 3rd draft edition edition, 2023.

[30] Richa Kumari, Ranjana Singh, and Ankita Bansal. Resume parsing and matching for recruitment system using machine learning. In International Conference on Advances in Computing, Communication and Control (ICAC3), pages 1–5, 2019.

[31] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In Proceedings of NAACL-HLT, pages 260–270, 2016.

[32] LangChain. Langchain – llm orchestration framework, 2025.

[33] LangGraph. Langgraph – langchain extension, 2025.

[34] Lever. Lever - talent acquisition suite (ats + crm), 2024.

[35] LinkedIn. Linkedin - professional networking platform.

[36] Pengfei Liu et al. Pre-train, prompt, and predict : A systematic survey of prompting methods in natural language processing. ACM Computing Surveys, 2022.

[37] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta : A robustly optimized bert pretraining approach. In arXiv preprint arXiv :1907.11692, 2019.

[38] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.

[39] Jeroen Meijerink and Stefan Strohmeier. The digitalization of hrm : A research agenda and study of hrm technologies. Human Resource Management Review, 31(4) :100780, 2021.

[40] Microsoft. Visual studio code, 2025.

[41] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In Proceedings of Workshop at ICLR, 2013.

[42] MongoDB. Mongodb compass – gui for mongodb, 2025.

[43] MongoDB. Mongodb – nosql database, 2025.

[44] Monster. Monster - find jobs and careers.

[45] Andrew Morris, Viktoria Maier, and Phil Green. From wer and ril to mer and wil : improved evaluation measures for connected speech recognition. INTERSPEECH, pages 2765–2768, 2004.

[46] Jianmo Ni, Nils Reimers, and Iryna Gurevych. Sentence-t5 : Scalable sentence encoders from pre-trained text-to-text models. In Findings of the Association for Computational Linguistics (ACL), 2022.

[47] Long Ouyang et al. Training language models to follow instructions with human feedback. arXiv preprint arXiv :2203.02155, 2022.

[48] Overleaf. Overleaf – online latex editor, 2025.

[49] Pallets Projects. Flask – python web framework, 2025.

[50] Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove : Global vectors for word representation. In Proceedings of EMNLP, pages 1532–1543, 2014.

[51] Postman. Postman – api platform, 2025.

[52] Python Software Foundation. Python programming language, 2025.

[53] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. In OpenAI Technical Report, 2018.

[54] Recruitee. Recruitee - collaborative hiring software, 2024.

[55] Nils Reimers and Iryna Gurevych. Sentence-bert : Sentence embeddings using siamese bert-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 3982–3992, 2019.

[56] Claude Sammut and Geoffrey I. Webb. Evaluation measures for information extraction. Encyclopedia of Machine Learning and Data Mining, pages 433–439, 2017.

[57] A. Santos, M. Gonçalves, and P. Quaresma. A hybrid approach for information extraction from curriculum vitae. In Proceedings of the International Conference on Computational Processing of the Portuguese Language (PROPOR), pages 250–259. Springer, 2019.

[58] B. A. Shawar and E. Atwell. Chatbots : Are they really useful ? LDV Forum, 22(1) :29–49, 2007.

[59] SmartRecruiters. Smartrecruiters - modern talent acquisition suite, 2024.

[60] Greenhouse Software. Greenhouse - hiring software for structured recruiting, 2024.

[61] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mpnet : Masked and permuted pre-training for language understanding. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[62] Andreas Stolcke and Jasha Droppo. Comparing human and machine errors in conversational speech transcription. INTERSPEECH, pages 137–141, 2017.

[63] Swagger. Swagger – openapi tools, 2025.

[64] Talinty. Talinty – publications on linkedin, 2025. Consulté sur LinkedIn – flux de publications de la société.

[65] Qdrant Team. Qdrant : Vector search engine for the next generation of ai applications. 2022. Available at : https://qdrant.tech/.

[66] Qdrant Team. Qdrant : High-performance vector database for scalable ai applications, 2023.

[67] Weaviate Team. Weaviate : An open source vector search engine. 2021. Available at : https://weaviate.io/.

[68] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), pages 5998–6008, 2017.

[69] Sheng Wang, Yu Kuo, Wei Zeng, et al. Text embeddings by weakly-supervised contrastive pre-training. arXiv preprint arXiv :2212.03533, 2022.

[70] Wenhui Wang and Furu Wei. Minilm : Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[71] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

[72] Jason Wei et al. Finetuned language models are zero-shot learners. In International Conference on Learning Representations (ICLR), 2022.

[73] Workable. Workable - applicant tracking system (ats) and recruiting software, 2024.

[74] Y. Xi, R. Zhang, H. Chen, and L. Yang. Langchain : Building applications with large language models. arXiv preprint arXiv :2302.12327, 2023.

[75] Jian Xu and et al. Baai general embedding (bge) models, 2023. Beijing Academy of Artificial Intelligence (BAAI). Available at : https://huggingface.co/BAAI/bge-large-en.

[76] Zoho. Zoho cliq – team communication tool, 2025.